

Day 1 — Linux Fundamentals & Architecture

Kernel • FHS • Files, Links & Permissions Basics • Shell Navigation

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Week 2 • Day 1 — Morning 09:00-13:00 • Evening 17:00-21:00

Day 1 — Linux Fundamentals & Architecture

Starting Week 2 — from "network" to "server"

Today's study path (per the 30-day plan): Morning: Linux Overview (09:00-10:30) → Distributions & Kernel (10:45-12:15) → File System Hierarchy (12:30-13:00) Evening: Lab: install Linux and connect over SSH (17:00-18:30) → Navigation & Files (18:45-20:15) → File Operations (20:30-21:00)

Goal of the day: understand what "Linux" actually is (kernel + user space), tell distributions apart, know the file-system tree by heart, navigate fast, and perform file operations without mistakes. This day is the foundation for the next five.

0) How Week 1 connects to Week 2 (building the bridge)

In Week 1 you worked on routers and switches; in Week 2 you move to the **server that serves that network**. Same concepts, new names:

Week 1 concept (network)	Week 2 equivalent (Linux)	Note
Router IOS (proprietary OS)	Linux kernel + user space	In IOS everything lives in one binary; in Linux services are separate processes
<code>show ip route</code>	<code>ip route</code> / <code>ip addr</code>	Same concepts, new tools (Week 2, Day 6)
ACL on a router	<code>iptables</code> / <code>nftables</code> on Linux	Packet filtering at the host layer (Week 2, Day 6)
DHCP server on IOS	<code>dnsmasq</code> / <code>isc-dhcp-server</code>	Same DORA, different implementation
SSH to a router	SSH to a server (<code>sshd_config</code>)	Day 6 of this week
Flash/NVRAM/startup-config	<code>/boot</code> , <code>/etc</code> , disks and partitions	Day 2 of this week
<code>copy run start</code> (save config)	<code>systemctl</code> , systemd unit files	Day 5 of this week

Key interview line:

"The network delivers packets; the operating system turns them into services — and both must be observable, configurable, and debuggable."

1) Linux Overview (09:00-10:30)

1.1 What exactly is Linux?

Strictly speaking, **Linux is the kernel** — the core released by **Linus Torvalds** in 1991. But when people say "Linux" they usually mean the **whole operating system**:

```
Hardware
|
v
Kernel (Linux kernel: CPU scheduling, memory, file systems, networking, drivers, syscalls)
|
v
User space (C library, GNU tools, shell, services, user programs)
|
v
Application (the service you write or install)
```

Layer	Responsibility	What you see when it breaks
Kernel	CPU scheduling, memory management, VFS, networking, drivers, syscalls	Kernel panic, hangs, disks/NICs not detected
User space	Running services, command shell, libraries	Commands fail, service won't start, but the system is up

Two things that earn interview points: 1. The kernel is **monolithic** but supports **loadable modules at runtime**: `lsmod`, `modprobe`, `modinfo`. 2. Kernel and user space talk through **syscalls** (`strace` shows that conversation — Day 4).

1.2 Licensing: what is free software?

- **GPLv2**: Linux is licensed under it → if you distribute it, you must share the source (copyleft).
- **Free software ≠ zero cost**: "free" means the freedom to run/study/modify/redistribute, not a price tag.
- **Open Source** expresses the same ideas in more practical terms.
- If asked "why does Linux dominate servers?": no per-seat licensing, stability, scriptability, tooling ecosystem, code auditability, and customizability.

1.3 The 30-second history (for "what do you know about Linux?")

Year	Event
1969–70	Unix at Bell Labs (Ken Thompson, Dennis Ritchie)
1983	GNU project (Richard Stallman) — free tools, but no kernel
1991	Linux kernel 0.01 by Linus Torvalds
1993	Debian is born (first large distribution with real package management)
1994	Red Hat Linux; Linux 1.0
2004	Ubuntu (based on Debian) — Linux reaches the desktop
2011	First year Linux held the largest share of the server market
Today	~90% of public web servers, every major cloud, Android, and most IoT devices

1.4 Modern uses and the CEH link (Week 3)

- Web/database/mail servers, hypervisors (KVM), containers (Docker/K8s), supercomputers, software routers (VyOS, pfSense).
- For security: nearly every CEH tool (Nmap, Metasploit, tcpdump, Netcat) runs on Linux; understanding Linux means understanding the attack surface.

2) Distributions & Kernel (10:45–12:15)

2.1 What is a distribution?

Distro = kernel + bootloader + GNU tools + package manager + repositories + installer + support.

Family	Examples	Package manager	Package format	Support model
Debian	Debian, Ubuntu, Mint, Kali	dpkg + apt	.deb	Ubuntu LTS: 5 years
Red Hat	RHEL, Rocky , AlmaLinux , Fedora, CentOS Stream	rpm + dnf / yum	.rpm	RHCSA/RHCE; 10-year RHEL cycle
SUSE	SLES, openSUSE Leap/Tumbleweed	rpm + zypper	.rpm	YaST; strong in SAP
Arch	Arch, Manjaro	pacman	rolling release	Outstanding docs (Arch Wiki)
Gentoo	Gentoo	portage (emerge)	source builds	Maximum optimization
Others	Alpine (apk), Void (xbps), Slackware	—	—	Alpine for containers (busybox, musl)

Practical differences interviewers ask about: - **Ubuntu/Debian** dominate web servers and cloud; **RHEL/Rocky** dominate large enterprises and banks; **Alpine** dominates containers. - Install command: `apt install nginx` vs `dnf install nginx` — same mechanism, different path. - RHEL costs money but buys support and guarantees; Rocky/Alma are "binary compatible" rebuilds.

⚠ **Beginner mistake:** "I'll install Kali on the server because it has security tools." Kali is a **desktop penetration-testing** distro; for servers use Debian/Ubuntu Server or Rocky. (This answer scores well in security interviews.)

2.2 Kernel versioning

```
4 . 15 . 0 - 200 . fc42 . x86_64
| | | | |
| | | | | +-- architecture
| | | | +----- distro patch level (Fedora)
| | | +----- kernel build / ABI
| | +----- bugfix release
| +----- major / feature version
+----- (mainline)
```

- `uname -r` → running release string; `uname -a` → everything; `cat /proc/version` → same info.
- **LTS kernels** (e.g. 6.1, 6.6) for servers; mainline releases roughly every 9–10 weeks.
- On Debian/Ubuntu: `linux-image-6.8.0-31-generic`, `linux-headers-...` (headers are needed to compile drivers).
- `/boot/vmlinuz-<ver>` is the compressed kernel image; `/boot/initrd.img-<ver>` (initramfs) is the early ramdisk (full boot explanation on Day 5).

2.3 Modules, `dmesg`, `/proc` and `/sys`

```
uname -a # kernel + architecture info
lsmod | head # loaded modules
modinfo e1000 # module details (path, parameters, license)
sudo modprobe -r e1000 # unload a module
sudo modprobe e1000 # load a module
sudo insmod ./mydrv.ko # manually insert a module file
dmesg | tail -30 # kernel ring buffer (hardware, drivers)
dmesg -T | tail # human-readable timestamps
journalctl -k -b # kernel messages for this boot via systemd
cat /proc/cpuinfo # CPU details
cat /proc/meminfo # memory details
cat /proc/cmdline # kernel command line used at boot
ls /sys/class/net # network interfaces from sysfs
lspci ; lsusb # PCI / USB hardware
```

`/proc` vs `/sys` (classic interview question): - `/proc` is a **virtual** file system: files are not real, they are windows into the kernel (`/proc/<pid>/`, `/proc/meminfo`). - `/sys` (sysfs) exposes the **device/driver model** (hardware structure rather than processes). - Both live in RAM and consume no disk.

2.4 Kernel config and the bootloader (intro)

- `/boot/config-<ver>` → kernel build options (`=y` built-in, `=m` module, `is not set` disabled).
- `/boot/grub/grub.cfg` → generated GRUB file; **never edit it by hand**, change `/etc/default/grub` and run `update-grub` (Debian) or `grub2-mkconfig` (RHEL). (Full details on Day 5.)

3) File System Hierarchy (12:30-13:00)

3.1 FHS — the Filesystem Hierarchy Standard

Path	Contents	Interview note
<code>/</code>	Root	everything lives below it
<code>/bin</code> , <code>/sbin</code>	Essential binaries	on modern distros symlinks into <code>/usr/bin</code> , <code>/usr/sbin</code>
<code>/lib</code> , <code>/lib64</code>	Libraries and kernel modules	<code>/lib/modules/<ver></code>
<code>/usr</code>	Programs and data (<code>/usr/bin</code> , <code>/usr/lib</code> , <code>/usr/share</code>)	the largest part of the system
<code>/usr/local</code>	Software you compiled/installed	package manager leaves it alone
<code>/etc</code>	Configuration files	text-based; should be version-controlled
<code>/var</code>	Variable data: logs (<code>/var/log</code>), spools, caches, web	first place to fill up
<code>/tmp</code>	Temporary files, cleared on boot (tmpfs)	dangerous spot (mode 1777 + sticky)
<code>/home</code>	User home directories	<code>/root</code> is separate for root
<code>/root</code>	root's home	mode 700
<code>/dev</code>	Device files (<code>/dev/sda</code> , <code>/dev/null</code> , <code>/dev/tty</code>)	created by udev
<code>/proc</code> , <code>/sys</code>	Virtual file systems	windows into the kernel
<code>/boot</code>	Kernel, initramfs, GRUB	mounted during boot
<code>/opt</code>	Add-on (third-party) software	e.g. <code>/opt/google/chrome</code>
<code>/srv</code>	Service data (ftp, local www)	rarely used
<code>/mnt</code> , <code>/media</code>	Manual mounts / removable media	<code>/mnt</code> for admins, <code>/media</code> for USB
<code>/run</code>	Runtime data (tmpfs): PIDs, sockets	empty after reboot
<code>/lost+found</code>	Files recovered by fsck	only on ext partitions

3.2 Professional navigation

```
pwd # where am I
ls -lah # all + human sizes + details
ls -ltr # sort by time (newest last)
ls -d */ # directories only
cd /etc ; cd ~ ; cd - # go, home, back to previous
cd .. # one level up
tree -L 2 /etc # two-level tree (if tree is installed)
realpath /usr/bin/vi # canonical path after resolving symlinks
readlink -f /usr/bin/vi # same idea
which python3 # first executable in PATH
type ls # alias / function / binary? (important!)
command -v ls # path regardless of aliases
```

Absolute vs relative: `/var/log/syslog` is absolute; `../../etc/hosts` is relative. In scripts **always use absolute paths** so they don't depend on `cd`.

What is PATH? The search order for commands:

```
echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
export PATH="$PATH:/opt/tools/bin" # add a directory (temporary)
```

Security note: if `.` is at the start of PATH, a malicious file in the current directory can be executed accidentally — never do this.

4) Lab 1 — Install Linux, log in, connect over SSH (17:00–18:30)

4.1 Choosing a lab environment

Option	Best for	Why
VirtualBox / VMware + Ubuntu Server LTS	beginner-intermediate	snapshots for risky tests, NAT/Bridged networking
Multipass / WSL2	fast on Windows/macOS	lightweight, virtual network included
Docker/Podman (Debian/Alpine)	command and script practice	fast, but no systemd
Cheap VPS (1 GB)	real SSH, firewall, service practice	real-world experience; secure it properly
Kali/Parrot (separate VM)	Week 3 (CEH)	security tooling

Recommendation for this 30-day program: **one Ubuntu Server 24.04 LTS VM + one Rocky/Alma 9 VM** (to see `dnf` and `firewalld` differences) + a **Kali** VM in Week 3.

4.2 Step-by-step install (Ubuntu Server 24.04 LTS in VirtualBox)

- 1) Download ISO: ubuntu.com/download/server (ubuntu-24.04.x-live-server-amd64.iso)
- 2) VirtualBox > New:
Name: srv1 | Type: Linux | Version: Ubuntu (64-bit)
RAM: 2048 MB (minimum) | CPU: 2 | Disk: 20 GB (VDI, dynamically allocated)
- 3) Settings > Network: Adapter 1 = NAT (optionally Adapter 2 = Host-only for host access)
- 4) Settings > Storage: attach the ISO to the optical drive
- 5) Start:
Language: English | Keyboard: English (US)
Network: keep DHCP for now (we make it static on Day 6)
Storage: Use an entire disk + LVM (experience LVM here – Day 2)
Profile: your name, server name srv1, a strong username and password
SSH: tick "Install OpenSSH server"
Snaps: none (leave empty for learning)
- 6) After reboot: `sudo reboot`, then log in

4.3 Establish your identity

```
whoami # who am I
id # uid, gid, groups
hostnamectl # hostname, OS, kernel, virtualization
cat /etc/os-release # distro + version (script-friendly)
uname -r # kernel version
uptime # uptime, users, load average
who ; w # who is logged in and what they run
last | head # login history
echo $SHELL # current shell
ip -br a # interfaces and addresses, compact
```

Definitions interviewers love: - **Terminal** = a program that displays text I/O. **Shell** = the interpreter that reads and runs commands. **Console** = the physical terminal on the box. **TTY** = a virtual terminal device (`/dev/tty1`). - `tty` shows your current terminal; over SSH it looks like `/dev/pts/0` (**pts** = pseudo-terminal).

4.4 SSH from your host

```
# on the host (client)
ssh user@192.168.56.10 # log in with a password
ssh-keygen -t ed25519 -C "lab-key" # create a key (fully explained on Day 6)
ssh-copy-id user@192.168.56.10 # copy the public key to the server
ssh user@192.168.56.10 'uptime; ip -br a' # run a command without an interactive shell
scp ./file.txt user@192.168.56.10:/tmp/ # copy a file
```

```
# on the server: SSH service status
systemctl status ssh # Debian/Ubuntu service name is ssh
systemctl status sshd # RHEL/Rocky service name is sshd
ss -tlnp | grep :22 # what is listening on port 22
```

Week 1 → Week 2 link: in Week 1 you SSH'd into a router (`ssh admin@10.1.1.1`) and used an ACL so only the management subnet could connect. Now you do the same thing **on the server side**, and on Day 6 you harden it with `sshd_config` plus `firewalld/nft`.

4.5 Break & Fix drills

#	Break it	Symptom	Diagnose	Fix
1	Change a user's password to a wrong value	login fails, visible in <code>lastb</code> /journal	<code>sudo journalctl -u ssh -n 20</code>	log in via console, <code>passwd</code> , restore
2	Stop the SSH service	Connection refused	<code>systemctl status ssh</code>	<code>sudo systemctl start ssh</code>
3	Disable it and reboot	SSH is down after boot	<code>systemctl is-enabled ssh</code>	<code>sudo systemctl enable --now ssh</code>
4	<code>chmod 644</code> the key file	Permissions 0644 ... are too open	<code>ls -l ~/.ssh</code>	<code>chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys</code>

5) Navigation & Files (18:45–20:15)

5.1 Reading files — the right tool for the job

```
cat /etc/hostname # small file
head -20 /var/log/syslog # first 20 lines
tail -20 /var/log/syslog # last 20 lines
tail -f /var/log/syslog # follow live (Ctrl+C to stop)
less /etc/services # paged: /search n next G end q quit
wc -l /etc/passwd # count lines
nl file.txt # number the lines
grep -n "error" /var/log/syslog # find lines with numbers
sed -n '10,20p' file.txt # print lines 10–20 only
```

The three power columns of `less` : `/pattern` search → `n` next hit → `G` end of file → `g` start → `Ctrl+G` file info → `q` quit.

5.2 Finding files: `find` and `locate`

```
find /etc -name "*.conf" # by name
find /var -type f -size +100M # files larger than 100 MB
find /home -user ali # by owner
find /tmp -mtime +7 -type f # older than 7 days
find /var/log -type f -exec grep -l "ERROR" {} \; # run a command on results
find / -xdev -perm -4000 -type f 2>/dev/null # SUID files (security-critical)
find . -maxdepth 2 -type d -name "conf*" # limit depth
locate sshd_config # from the database (fast)
sudo updatedb # refresh the locate database
```

Tool	Speed	Freshness	When
<code>find</code>	slow	real-time and exact	disk cleanup, security, scripts
<code>locate</code>	very fast	depends on <code>updatedb</code>	quick path lookups

5.3 Pipes and redirection — the heart of Linux

```
ps aux | grep nginx | grep -v grep
journalctl -u ssh | tail -100 | grep -i "failed"
find /var/log -name "*.log" | xargs -I{} du -h {} | sort -h | tail -10
cut -d: -f1,3 /etc/passwd | sort -t: -k2 -n | head
awk -F: '$3>=1000 {print $1, $3}' /etc/passwd
sed 's/old/new/g' file.txt > new.txt
tr 'a-z' 'A-Z' < file.txt | head -5
sort -u names.txt | wc -l
uniq -c sorted.txt | sort -nr | head
diff -u a.conf b.conf
```

> and >> and 2> and &> :

```
cmd > out.txt # stdout -> file (truncate)
cmd >> out.txt # stdout -> append
cmd 2> err.txt # stderr -> file
cmd &> all.txt # stdout + stderr
cmd > out.txt 2>&1 # same (order matters!)
cmd 2>&1 > out.txt # classic mistake: stderr still goes to the terminal
cmd | tee log.txt # show AND save
```

Descriptor	Number	Meaning
stdin	0	default input (keyboard/file)
stdout	1	normal output
stderr	2	error messages

5.4 Compression and archives

```
tar -czvf backup.tar.gz /etc/nginx # create a gzip archive
tar -tzvf backup.tar.gz # list without extracting
tar -xzf backup.tar.gz -C /restore # extract into a path
tar -cjf b.tar.bz2 dir/ # bzip2 (smaller, slower)
tar --exclude='*.log' -czf web.tgz /var/www
gzip -9 big.log ; gunzip big.log.gz
xz -9 big.tar # best compression ratio
zip -r site.zip /var/www ; unzip -l site.zip
```

Comparison: `tar` archives (many files → one), `gzip/xz` compresses. `tar.gz` is both. For backups always `tar -czf` and test the extraction — an untested backup is not a backup.

5.5 Links: hard vs symbolic

```
ln target.txt hard_link # hard link: same inode, link count +1
ln -s /etc/nginx/nginx.conf /root/nginx.conf.link # symlink
ls -li target.txt hard_link # note: identical inode numbers
readlink -f /root/nginx.conf.link
```

Feature	Hard link	Symbolic (soft) link
Points to	the inode (actual data)	a path name
Across file systems	✗ impossible	✓ possible
Directories	✗ (except <code>.</code> and <code>..</code>)	✓
If target is deleted	data survives (one name fewer)	link becomes dangling
Displayed size	size of the data	length of the path string
Real example	strict backup schemes	<code>/usr/bin/python3</code> , <code>/etc/rc2.d</code>

Golden interview question: "What is the difference between a hard link and a soft link?" → inode vs path; link count in `ls -l` (2nd column) and `ln -s` for symlinks.

5.6 Searching inside content: `grep` and friends

```
grep -i "failed" auth.log # case-insensitive
grep -r "Listen 80" /etc/apache2 # recursive
grep -v "^#" nginx.conf # strip comments
grep -E "error|fail|warn" app.log # extended regex
grep -c "404" access.log # count
grep -A2 -B2 "panic" dmesg.txt # 2 lines before and after
zgrep "attack" /var/log/syslog*.gz
```

5.7 `sed` and `awk` — two multi-tools

```
sed -i 's/8080/80/g' site.conf # in-place replace (use -i.bak for safety)
sed -n '/start/,/end/p' file # print a range between patterns
sed '/^$/d' file # delete empty lines
awk '{print $1, $NF}' /var/log/x.log # first and last column
awk -F: '$3 >= 1000 {print $1}' /etc/passwd # real users
awk '{sum+=$5} END {print sum}' access.log # sum column 5
awk '/GET/ {c++} END {print c}' access.log # count GET requests
```

6) File Operations (20:30-21:00)

6.1 Create, copy, move, delete — with safe flags

```
mkdir -p /srv/app/{logs,conf,data} # build a tree in one command (brace expansion)
cp -av /etc/nginx /backup/nginx-$(date +%F) # -a keeps permissions, -v shows work
cp --preserve=all file /dest/ # explicit about ownership/timestamps
mv file1 file2 # move/rename (same command)
rm -i file.txt # ask before deleting
rm -rf /tmp/cache/* # mind the spaces! (trailing slash)
rmdir emptydir ; rm -rf dir
install -D -m 750 app.sh /usr/local/bin/app.sh # copy + create path + set mode
rsync -a --delete /var/www/ /backup/www/ # synchronization (best backup tool)
rsync -avz --progress /data/ user@10.20.0.5:/data/
```

⚠ **The scariest command in the world:** `rm -rf / --no-preserve-root`. Always verify the path with `pwd / ls` before `rm -rf`, and add `-i` if unsure.

6.2 Creating content

```
touch -t 202601011200 newfile # create / adjust timestamp
echo "hello" > f.txt # write
printf "%s\n" a b c # formatted output (better than echo in scripts)
cat > notes.txt <<'EOF' # heredoc: multi-line write without a shell
first line
second line
EOF
truncate -s 0 app.log # empty a log without deleting the file
```

6.3 Special files and devices

```
ls -l /dev/null /dev/zero /dev/random /dev/urandom
dd if=/dev/zero of=/tmp/test.img bs=1M count=100 # 100 MB file
head -c 32 /dev/urandom | od -An -tx1 # secure random bytes
stat file.txt # inode, mode, and all three timestamps
```

The three file timestamps (classic question): `atime` last access, `mtime` last content change, `ctime` last metadata change. `ls -l` shows only `mtime`; use `stat` for all of them.

6.4 Comparing and verifying files

```
diff -u old.conf new.conf # differences (patch-friendly)
vimdiff a.txt b.txt
cmp -l file1 file2 | head # byte-level differences
md5sum file ; sha256sum file # hashes (download verification, integrity)
file mystery.bin # the real file type (not the extension!)
```

7) 📅 Day 1 Self-Test (15 questions — answer first, then check the key)

1. In one sentence: kernel vs distribution?
 2. What does "monolithic kernel" mean, and what role do modules play?
 3. What is the difference between `/proc` and `/sys`?
 4. Name four subdirectories of `/var` and what lives in each.
 5. How do `/usr/local` and `/opt` differ?
 6. When does the difference between `which` and `type` matter?
 7. What does `find /var/log -type f -size +50M -exec ls -lh {} \;` do?
 8. Difference between a hard link and a symlink, plus the command to create each.
 9. In `cmd > out.txt 2>&1`, why does the order of redirections matter?
 10. Why is `less` better than `cat` for a large file? Give three shortcuts.
 11. Difference between `tar -czf b.tgz dir` and `gzip -r dir`?
 12. What is the risk in `rsync -a --delete src/ dst/` — and why is it useful for backups?
 13. Difference between `atime`, `mtime`, `ctime` and the command that shows all three?
 14. Why does `/tmp` have mode `1777`? What is that leading `1`?
 15. Difference between `apt purge nginx`, `apt remove nginx`, and `apt autoremove`?
-

8) Day 1 Interview Bank (12 questions with model answers)

- 1) What is Linux?** The Linux kernel is a monolithic kernel with loadable modules that manages CPU, memory, file systems, networking, and drivers. What we call "a Linux operating system" is kernel + GNU tools + package manager + services + configuration — a distribution.
- 2) Why is Linux the standard on servers?** High stability and uptime, small footprint, scriptability and automation, a strong permission model, zero per-seat licensing, a massive tool ecosystem, and first-class container/virtualization support.
- 3) How do you explain Ubuntu vs CentOS/Rocky for an enterprise project?** Ubuntu uses `apt` / `.deb`, ships every 6 months with 5-year LTS; Rocky/Alma are binary-compatible with RHEL using `dnf` / `.rpm`, a 10-year lifecycle, SELinux enabled, and commercial support. The choice follows organizational policy, support needs, and ecosystem.
- 4) User space vs kernel — how do they communicate?** The kernel runs privileged with direct hardware access; user processes run restricted and request work via system calls (plus `/proc` and `/sys`). `strace` makes that conversation visible.
- 5) What is FHS and why does it matter?** The Filesystem Hierarchy Standard defines where files belong. It matters for portable scripts, tools that assume standard paths, and targeted troubleshooting/backup.
- 6) If `/` is full, what are the first five commands you run?**

```
df -hT ; df -i # space and inodes
du -xh --max-depth=1 / | sort -h # biggest top-level directories
find /var -type f -size +500M -exec ls -lh {} \;
lsof +L1 | head # deleted-but-open files
journalctl --disk-usage # journal size
```

Then: rotate/trim old logs, `journalctl --vacuum-size=200M`, fix logrotate, check `/tmp` and caches.

7) `cat` vs `less` vs `tail -f` — which one when? `cat` for small files, `less` for interactive browsing with search, `tail -f` for live log watching. On servers I reach for `less` first (so the terminal buffer survives) and `tail -f / journalctl -f` when I need to watch events as they happen.

8) How do you find which config file in `/etc` contains "Listen"?

```
grep -RIn --exclude-dir=.git "Listen" /etc
find /etc -type f -name "*.conf" -exec grep -Hn "Listen" {} +
```

9) Difference between `>` and `>>` and their main danger? `>` truncates, `>>` appends. The danger: redirecting a command onto its own input file (`sort file > file`) destroys the file before it is read.

10) How do you create a directory tree with specific permissions in one command?

```
mkdir -p /srv/app/{conf,logs,data} && chmod -R 750 /srv/app && chown -R app:app /srv/app
```

11) When do you choose `find` over `locate`? `locate` is fast but database-based and possibly stale; `find` is real-time and exact with complex criteria (size, time, owner, permissions). For sensitive scripts and backups I always use `find`.

12) How do you know a backup is healthy? I hash it (`sha256sum`), extract it into a scratch path and compare file counts/sizes with the source (`tar -tzf | wc -l`), and when possible run a restore drill on a test system. An untested backup is not a backup.

9) ☐ Day 1 Completion Checklist

- [] I installed a Linux VM and can SSH into it from my host
- [] I ran `uname -r`, `hostnamectl`, `cat /etc/os-release` and understand the output
- [] I inspected `lsmod`, `modinfo`, `dmesg`, `/proc/cpuinfo`, `/proc/meminfo`
- [] I can recite the FHS tree and give one example file per branch
- [] I ran `find` with size/time/owner filters and refreshed `locate`'s database
- [] I practiced pipes and redirection (`|`, `>`, `>>`, `2>`, `&>`, `tee`)
- [] I created, listed, and extracted a `tar` archive and hashed it
- [] I created a hard link and a symlink and observed inode equality with `ls -li`
- [] I completed the four Break & Fix drills in the lab
- [] I answered all 12 interview questions **out loud**

All questions in this day, their full answers, and the week's exams are collected in the key and final-exam chapter at the end of this pack.

Day 2 — Commands, Permissions & File Systems

Text Processing • Users & Permissions • ACL • inodes • LVM

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Week 2 • Day 2 — Morning 09:00-13:00 • Evening 17:00-21:00

Day 2 — GNU/Linux Commands, Permissions & File Systems

Today's study path (per the 30-day plan): **Morning:** Text processing (09:00-10:30) → File permissions (10:45-12:15) → Links: hard & soft (12:30-13:00) **Evening:** Lab: permissions and users (17:00-18:30) → `find / locate / which` (18:45-20:15) → File systems, partitions and LVM basics (20:30-21:00)

Goal of the day: professional text processing (`grep/sed/awk/cut/sort/uniq/tr`), a deep understanding of the permission model (rwx, octal, special bits, umask, ACL), and solid knowledge of file systems, partitioning, and LVM.

0) 3-minute review of Day 1

Yesterday	Where it shows up today
FHS (<code>/etc</code> , <code>/var</code> , <code>/home</code>)	Where users (<code>/etc/passwd</code>), logs (<code>/var/log</code>), and permissions live
<code>find / grep</code> /pipes	Today's text-processing toolkit; they combine constantly
hard/symlink	Going deeper: <code>inode</code> , link count, <code>stat</code>
File operations	<code>cp -a</code> and <code>rsync</code> preserve permissions — today you learn why that matters
<code>ls -l</code>	Column one = the permission bits you unlock today

The connecting sentence: yesterday you found and moved files; today you **process their content** and **control who may do what** — two skills that make up ~80% of daily admin work.

1) Text Processing (09:00-10:30)

1.1 The family of tools

Tool	Main job	Key example
grep	search with patterns	grep -RIn "error" /var/log
cut	slice columns/characters	cut -d: -f1 /etc/passwd
sort	sort lines	sort -t: -k3 -n /etc/passwd
uniq	collapse/count adjacent duplicates	uniq -c
tr	translate/delete characters	tr '[:lower:]' '[:upper:]'
sed	stream-edit text	sed -Ei 's/x/y/g' f
awk	column processing and math	awk -F: 'NR<=5{print \$1}'
head/tail	start/end of a file	tail -n +2
wc	count lines/words/bytes	wc -l
paste/join/split	combine, join, split	split -l 1000 big.txt part_
tee	write to a file and the screen at once	at the end of any pipeline (see §2.3)
column	turn dense output into readable columns	column -t applied to mount

1.2 grep in depth

```
grep -RIn --include="*.conf" "Listen" /etc # recursive + line numbers + only *.conf
grep -v "^#" nginx.conf | grep -v "^$" # strip comments and blank lines
grep -E "(GET|POST) /admin" access.log # extended regex
grep -P "\b\d{1,3}(\.\d{1,3}){3}\b" file # PCRE: IP pattern
grep -oE "[0-9]+\.[0-9]+" log | sort -u # print only the match
grep -L "permit" *.conf # files that do NOT contain the pattern
grep -c "" file # line count (same as wc -l)
```

Performance note: for large files and literal strings, `grep -F` (fixed string, no regex) is much faster; `rg` (ripgrep) is the modern, faster choice for large trees.

1.3 sed in depth — the three everyday uses

```
sed -i.bak -E 's/^#?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config # edit config +
keep a backup
sed -n '100,120p' app.log # line range
sed -n '/ERROR/,+3p' app.log # every ERROR plus the next 3 lines
sed -e 's/\t/ /g' -e 's/ */ /g' messy.txt # multiple edits in sequence
sed '/^#/d; /^$/d' file # delete comments and empty lines
sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\3\/\2\/\1/' dates.txt # swap capture groups
```

⚠ **Golden rule:** `-i` without `-i.bak` on a config file is how you lose a service. And always validate afterwards with `nginx -t`, `sshd -t`, or `named-checkconf`.

1.4 `awk` in depth — a small language that does a lot

```
awk -F: '{print $1}' /etc/passwd # first column
awk -F: '$3>=1000 && $1!="nobody" {print $1,$3}' /etc/passwd
awk '{s+=$10} END {printf "Total bytes: %d\n", s}' access.log
awk '$9==404 {c++} END {print "404s:", c}' access.log
awk 'NR==1{print} /CRITICAL/{print NR: "$0"}' app.log # header + critical lines
awk -F'[|]' '{print $2}' logs | sort | uniq -c | sort -nr | head
awk '{a[$1]++} END {for (k in a) print a[k], k}' ips.txt
```

Roles, separated for interviews: `grep` selects lines, `sed` edits lines, `awk` selects and organizes **fields** and computes. Those three plus pipes give you data processing without any programming language.

1.5 A real example: a web-log report (the serious exercise for this block)

```
# top 10 consumer IPs
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head -10

# top 10 requested paths
awk '{print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head -10

# 5xx errors per hour
awk '$9 ~ /^5/ {print $4}' access.log | cut -d: -f1 | sort | uniq -c

# requests from inside the network (RFC1918 only)
grep -E "^(10\.|192\.168\.|172\.(1[6-9]|2[0-9]|3[01])\.)" access.log | wc -l
```

Week 1 link: the same analysis you did on a router with ACL counters, you now do on a server log; in Week 3 (CEH) this exact pattern becomes attack-event analysis.

1.6 The important text files of the system

```
cat /etc/passwd # name:password:UID:GID:GECOS:home:shell
sudo cat /etc/shadow # name:hash:password policy dates (root only)
cat /etc/group # name:password:GID:members
sudo cat /etc/gshadow
cat /etc/fstab # what gets mounted at boot
cat /etc/hosts /etc/hostname /etc/resolv.conf
cat /etc/os-release /etc/shells /etc/services
ls -l /etc/passwd /etc/shadow /etc/group # compare the permissions!
```

Know the `/etc/passwd` layout by heart:

```
ali : x : 1000 : 1000 : Ali A , Room 12 , 555 , 555 : /home/ali : /bin/bash
^ ^ ^ ^ ^ ^ ^
| | | | +-- GECOS (description) | +-- login shell
| | | +----- primary GID +-- home directory
| | +----- UID
| +----- password placeholder (x = stored in /etc/shadow)
+----- login name
```

2) File Permissions (10:45-12:15)

2.1 Decoding `ls -l`

```
-rwxr-xr-- 1 ali devs 4096 Sep 26 10:00 script.sh
||| ||| |||
||| ||| +++---- others: r--
||| +++----- group: r-x
+++----- owner: rwx
+----- type: - regular | d directory | l symlink | c/b device | s socket | p pipe
```

Character	On a file	On a directory
<code>r</code> (4)	read contents	list names (<code>ls</code>)
<code>w</code> (2)	change contents	create/delete/rename entries
<code>x</code> (1)	execute the file	enter (<code>cd</code>) and traverse

The single most important idea here: deleting a file depends on the permissions of the **parent directory**, not on the file's own mode. If you have `w` and `x` on the directory, you can delete a read-only file.

2.2 Octal mode and how it adds up

rwX	octal		rwX	octal
---	0		r-x	5
--X	1		rw-	6
-W-	2		rwX	7
r--	4			

```
chmod 644 file # rw-r--r--
chmod 600 id_rsa # rw----- (private key!)
chmod 755 /usr/local/bin/app
chmod 640 secret.cfg
chmod u+x,g-w,o-rwx script.sh # symbolic mode
chmod -R 750 /srv/app # recursive (careful)
chmod --reference=ref.txt target.txt
```

Mode	Typical use
644	normal files, documents, readable config
600	private data: private SSH keys, secrets in config
755	executables, public directories
700	private home dirs, sensitive scripts
664/775	group collaboration
1777	/tmp

2.3 Ownership: `chown` / `chgrp`

```
sudo chown ali file
sudo chown ali:devs file
sudo chown -R www-data:www-data /var/www/html
sudo chgrp devs /srv/project
sudo chown --from=olduser:oldgroup newuser file
```

Numeric UID/GID: `chown 1000:1000 file` — needed in container scenarios where users exist by number only.

2.4 `umask` — where default permissions come from

```
umask # e.g. 0022
umask -S # u=rwx,g=rx,o=rx
umask 027 # for the current shell only
```

- New file: `666 - umask`. With `022` → `644`. New directory: `777 - umask` → `755`.
- `umask 077` for private data (directories `700`, files `600`).
- Permanent location: `/etc/profile`, `/etc/bash.bashrc`, or `~/.bashrc` (Day 3).

Practice table (fill it yourself):

umask	directory	file
022	755	644
027	750	640
077	700	600
002	775	664

2.5 Special bits: SUID, SGID, sticky (very important)

```
-rwsr-xr-x root root /usr/bin/passwd <- SUID
drwxrwsr-x root devs /srv/shared <- SGID on a directory
drwxrwxrwt root root /tmp <- sticky
```

Bit	Value	On a file	On a directory
SUID	4000	runs with the owner's UID (like <code>passwd</code> writing to <code>shadow</code>)	—
SGID	2000	runs with the file group's GID	new files inherit the directory's group
Sticky	1000	—	only the owner/root may delete others' files (<code>/tmp</code>)

```

chmod u+s app # add SUID
chmod 4755 app # SUID + 755
chmod g+s /srv/shared # SGID on a shared directory
chmod 1777 /tmp ; chmod +t /tmp # sticky
find / -xdev -perm -4000 -type f 2>/dev/null # hunt system SUID binaries

```

Security angle (bridge to Week 3): audit the SUID list regularly and diff it against a baseline. An unexpected SUID binary (`nmap`, `vim`, `find`, `cp`) is a serious **privilege-escalation** signal. `sudo -l` is the other half. Both are standard SOC/pentest interview questions: `bash find / -perm -4000 -type f 2>/dev/null | sort > /root/suid-baseline.txt # baseline diff baseline.txt <(find / -perm -4000 -type f 2>/dev/null | sort) # weekly compare`

2.6 ACLs — when three classes are not enough

```

mount | grep acl # does the fs support ACLs?
getfacl /srv/project # view ACLs
setfacl -m u:mary:rw /srv/project/report.doc # a specific user gets rw
setfacl -m g:audit:r-x /srv/project # a specific group
setfacl -m d:u:mary:rw /srv/shared # default ACL for new files
setfacl -x u:mary /srv/project/report.doc # remove one entry
setfacl -b /srv/project/report.doc # remove all ACLs
getfacl --all-effective /srv/project | head

```

`getfacl` keywords: `user::rw`, `group::r-x`, `other::r-x`, `mask::rw` (the ceiling for extended entries), `default:...`.

Interview note: the **mask** caps effective named-user/named-group permissions. If an ACL change "does not apply", check the mask first.

2.7 sudo and access policy

```

sudo visudo # safe editing of /etc/sudoers (test before you exit!)
sudo -l # what may I do?
sudo -u www-data whoami # run as another user
sudo -k # invalidate the cached credential

```

```

# example entry in /etc/sudoers.d/ops
%devs ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
deploy ALL=(root) NOPASSWD: /usr/bin/rsync --server *
Defaults env_reset, timestamp_timeout=5, logfile="/var/log/sudo.log"

```

su vs sudo (RU in the plan): **su** gives a full shell as another identity and requires the target's password; **sudo** runs specific commands under an auditable policy. In modern servers: disable direct root SSH, use per-user keys, and grant granular, logged **sudo**.

3) Links: a deeper look at inodes (12:30–13:00)

3.1 What is an inode?

Every file on ext4 has **one inode**: metadata (owner, mode, timestamps, size, link count, pointers to data blocks) — but **the name is not inside the inode**; the directory maps names to inodes.

```
stat file.txt | head -8
ls -li file.txt hard link # same inode -> hard link
ls -li file.txt symlink # different inode, type l -> symlink
ln file.txt h2.txt ; ls -l file.txt h2.txt | awk '{print $2, $9}' # link counts
rm file.txt ; cat h2.txt # data survives until the last name is gone
```

Operation	Hard	Soft
<code>ls -l</code>	increments the link count	creates a new file of type <code>l</code>
Delete the target	data stays; one name is removed	link dangles (<code>readlink</code> still shows the path)
On a directory	forbidden (infinite loops)	allowed
Across file systems	forbidden (inodes are local)	allowed
Real-world use	strict backups, identical files without copies	<code>python3 -> python3.12</code> , <code>/etc/rc2.d/S01nginx</code>

3.2 Links in practice: version switching

```
ls -l /usr/bin/python3 # python3 -> python3.12
sudo ln -sf /opt/app-2.0 /opt/app-current # switch versions without touching the service
readlink -f /opt/app-current
find / -xtype l 2>/dev/null # find broken symlinks across the system
```

4) Lab 1 — Permissions, users and groups (17:00–18:30)

4.1 Scenario

A company named **ParsData** has three teams: `devs` , `ops` , `audit` . A shared project directory `/srv/project` must follow this policy: - `devs` read/write; `ops` read/write; `audit` read-only. - New files must inherit the `devs` group (SGID). - User `ali` (a developer) may use `sudo` only to restart `nginx`.

4.2 Step by step

```
# 1) groups and users
sudo groupadd devs ; sudo groupadd ops ; sudo groupadd audit
sudo useradd -m -G devs -s /bin/bash ali
sudo useradd -m -G ops -s /bin/bash sara
sudo useradd -m -G audit -s /bin/bash reza
sudo passwd ali ; sudo passwd sara ; sudo passwd reza

# 2) project directory + SGID + access
sudo mkdir -p /srv/project
sudo chown root:devs /srv/project
sudo chmod 2775 /srv/project # rwxrwsr-x (SGID)
setfacl -m g:audit:rx /srv/project # audit: read-only

# 3) test: log in as each user and see what they can do
su - ali -c 'touch /srv/project/a.txt && ls -l /srv/project'
su - sara -c 'touch /srv/project/s.txt'
su - reza -c 'touch /srv/project/r.txt' # must fail: Permission denied
ls -l /srv/project ; getfacl /srv/project

# 4) a private file + SGID behaviour
echo "secret" | sudo tee /srv/project/report.txt >/dev/null
sudo chmod 640 /srv/project/report.txt
sudo chown root:devs /srv/project/report.txt

# 5) limited sudo for ali
echo 'ali ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx' | sudo tee /etc/sudoers.d/ali
sudo chmod 440 /etc/sudoers.d/ali
sudo -l -U ali
su - ali -c 'sudo systemctl restart nginx' # (if nginx is not installed, expect a service error, not a sudo error)
su - ali -c 'sudo cat /etc/shadow' # must be refused
```

4.3 Understanding checks (answer, then verify)

1. With `umask 027`, create a file and a directory. What are the modes? (`640`, `750`)
2. What does `chmod 1777` on `/tmp` mean? What would break without the sticky bit? (anyone could delete anyone's files)
3. A `600` file inside a `755` directory — can another user delete it? Why? (No: no `w` on the directory.)
4. Difference between `chmod u+s f` and `chmod g+s f`? (SUID vs SGID — the first on a binary, the second on files/directories.)
5. Run `find / -xdev -perm -4000 -type f 2>/dev/null` — how many entries, and do you recognize all of them?

4.4 Break & Fix drills

#	Break	Symptom	Diagnose/fix
1	<code>chmod 000 ~/.ssh/id_rsa</code>	SSH rejects the key	<code>chmod 600 ~/.ssh/id_rsa</code>
2	<code>chmod -R 777 ~/.ssh</code>	key login refused (too open)	<code>chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys</code>
3	delete a user but keep the home dir	locked home, <code>userdel</code> complaints	<code>sudo userdel -r ali</code> or <code>find / -nouser</code>
4	<code>umask 077</code> on a web server	the web server cannot read files	<code>umask 022</code> or a targeted <code>chmod</code>
5	<code>chown -R user:user /</code> (catastrophe)	the system will not boot	restore from snapshot/backup — and never run this as a test

5) `find` / `locate` / `which` — file management at scale (18:45-20:15)

5.1 `find` in depth

```
find / -xdev -type f -size +1G # bigger than 1 GB (no NFS)
find /var -type f -mtime -1 # changed in the last 24 h
find /var/log -type f \( -name "*.log" -o -name "*.gz" \) -size +10M
find /home -type d -perm -002 # world-writable dirs (security risk)
find . -type f -exec chmod 640 {} + # efficient exec (one invocation)
find /data -name "*.tmp" -delete
find /etc -newer /etc/passwd # newer than a reference file
find / -nouser -o -nogroup 2>/dev/null # orphaned ownership
find /tmp -type f -mtime +3 -exec rm -f {} \; # cleanup (test with -print first!)
```

Safe pattern: always start with `-print` and inspect the list before switching to `-delete` / `-exec rm`.

`-exec {} \;` vs `-exec {} +`: the first spawns one process per file (slow), the second batches them into one invocation (fast). In scripts, prefer `+` or `xargs`.

5.2 Quick log rotation (a real use of `find`)

```
find /var/log -name "*.log" -mtime +30 -exec gzip {} \;
find /var/log -name "*.gz" -mtime +365 -delete
du -sh /var/log/* | sort -h | tail -5
```

(The standard mechanism is `logrotate`, configured under `/etc/logrotate.d/`.)

5.3 `which`, `whereis`, `type`, `command -v`

```
which nginx # first match in PATH
whereis nginx # binary, source, man page
type -a ls # every meaning: alias/builtin/function/binary
command -v ls # path or type – safe inside scripts
hash -r # clear the shell's command path cache
```

6) File Systems, Partitions and LVM — the basics (20:30–21:00)

6.1 File system types

File system	Traits	Where
ext4	default on most distros, journaling, stable	general servers
XFS	journaling, excellent on large files, cannot shrink	RHEL/Rocky, big LVs, databases
Btrfs	snapshots, CoW, compression, subvolumes	openSUSE, NAS, rollbacks
ZFS	pools, checksums, snapshots, RAID-Z	storage/virtualization servers
vfat/exFAT	universally compatible	USB sticks, EFI partition
swap	disk-backed memory extension	swap partition/file
tmpfs	lives in RAM, very fast	<code>/tmp</code> , <code>/run</code> (not persistent)
NFS/CIFS	network file systems	sharing between servers/Windows

Journaling (ext4) vs copy-on-write (Btrfs): journaling guarantees metadata consistency (fast recovery after power loss); CoW never overwrites live data, which enables cheap snapshots.

6.2 Inspecting and creating

```
lsblk -f # block-device tree + file systems + UUIDs
blkid # UUID and LABEL for every device
df -hT ; df -i # space and inode usage
du -sh /var/* | sort -h | tail
mount | column -t
findmnt -T /var/www # which device backs this path
mount /dev/sdb1 /mnt/data
umount /mnt/data # if busy: lsof +D /mnt/data
```

Creating a file system and partitions (always on a scratch disk):

```

sudo fdisk -l /dev/sdb
sudo fdisk /dev/sdb # n create, t type, w write (MBR: max 4 primaries; GPT: 128)
sudo partprobe /dev/sdb
sudo mkfs.ext4 -L data /dev/sdb1
sudo mkfs.xfs /dev/sdb2
sudo tune2fs -l /dev/sdb1 | head -20 # ext4 feature inspection
sudo fsck -y /dev/sdb1 # check the fs (must be unmounted!)
sudo e2label /dev/sdb1 data ; sudo xfs_admin -L data /dev/sdb2

```

6.3 `/etc/fstab` — persistent mounts at boot

```

# <device> <mount point> <fstype> <options> <dump> <pass>
UUID=8f4e...c2 / ext4 defaults 0 1
UUID=1a2b...9d /data xfs defaults,noatime 0 2
UUID=77aa...11 none swap sw 0 0
//srv/share /mnt/share cifs credentials=/root/.smb,uid=1000 0 0

```

Option	Meaning
<code>defaults</code>	rw,suid,dev,exec,auto,nouser,async
<code>noatime</code>	do not update atime (performance)
<code>noexec/nosuid/nODEV</code>	hardening for <code>/tmp</code> , <code>/home</code>
<code>ro</code>	read-only
<code>nofail</code>	do not fail boot if the device is absent (USB)
<code>_netdev</code>	depends on the network (mount after networking)

```

sudo findmnt --verify # validate fstab (before rebooting!)
sudo mount -a # test without rebooting
sudo systemctl daemon-reload

```

⚠ **The classic beginner disaster:** a wrong UUID in `fstab` → the system boots into emergency mode. Always run `findmnt --verify` and `mount -a`.

6.4 LVM — the three layers

```

PV (Physical Volume: /dev/sdb1, /dev/sdc1, ...)
|
v
VG (Volume Group: a storage pool, e.g. vg_data)
|
v
LV (Logical Volume: vg_data/lv_www => ext4/xfs => /var/www)

```

```

sudo pvcreate /dev/sdb1 /dev/sdc1
sudo pvdisplay ; sudo pvs
sudo vgcreate vg_data /dev/sdb1 /dev/sdc1
sudo vgs ; sudo vgdisplay
sudo lvcreate -L 20G -n lv_www vg_data
sudo mkfs.ext4 /dev/vg_data/lv_www
sudo mount /dev/vg_data/lv_www /var/www

# grow (elasticity is the main reason to use LVM)
sudo lvextend -L +10G /dev/vg_data/lv_www
sudo resize2fs /dev/vg_data/lv_www # ext4 (for xfs: xfs_growfs /var/www)
sudo lvextend -l +100%FREE -r /dev/vg_data/lv_www # -r resizes in one step
# shrinking: resize2fs smaller first, then lvreduce -L -5G (risky; take a backup)

```

LVM advantage	Why it matters
Online growth	add capacity without downtime or reboots
Snapshots	consistent backups before upgrades/patches
Shared pool	several disks act as one logical space
Meaningful names	<code>vg_data/lv_www</code> instead of <code>/dev/sdb3</code>

6.5 swap

```

free -h ; swapon --show
sudo mkswap /dev/sdb2 ; sudo swapon /dev/sdb2
sudo fallocate -l 2G /swapfile ; sudo chmod 600 /swapfile
sudo mkswap /swapfile ; sudo swapon /swapfile
cat /proc/sys/vm/swappiness # default 60; DB servers often use 10

```

Rule of thumb: 1× RAM up to 4 GB, then 2 GB — or a small swap if you do not need hibernation.

6.6 Quotas (intro)

```

sudo mount -o remount,usrquota,grpquota /
sudo quotacheck -cugm / ; sudo quotaon -vug /
sudo edquota -u ali
sudo repquota -a

```

7) ☐ Day 2 Self-Test (15 questions)

1. In one short phrase each: `grep` , `sed` , `awk` ?
2. What does `awk -F: '$3>=1000 {print $1}' /etc/passwd` do?
3. Difference between `find -exec {} \;` and `find -exec {} +` ?
4. Why does deleting a file depend on directory permissions rather than the file's own mode?
5. With `umask 027` , what are the modes of a new file and a new directory?
6. What is SUID and why does `passwd` need it?
7. Difference between `chmod g+s` on a file and on a directory?

8. What problem does the sticky bit solve in `/tmp`?
9. Difference between `setfacl -m u:x:rw` and `setfacl -m d:u:x:rw`?
10. What role does the `mask` play in an ACL?
11. Difference between `userdel ali` and `userdel -r ali`?
12. When would you pick XFS over ext4? What is XFS's important limitation?
13. In `fstab`, when are `nofail` and `_netdev` needed?
14. Three reasons to use LVM instead of raw partitions?
15. What does `lvextend -l +100%FREE -r /dev/vg0/lv0` do?

8) Day 2 Interview Bank (12 questions with model answers)

1) What does mode `764` mean? Owner `rw` (7), group `rw-` (6), others `r--` (4). It is common for group-shared executables, though `755` / `750` are safer — `other` read access is fine for public content but wrong for sensitive data.

2) How do you make a file writable by one specific user without changing ownership? With ACLs: `setfacl -m u:mary:rw file` (and `-m d:u:mary:rw` on the directory for inheritance). Alternative: create a private group and use `chown :grp` plus `chmod 660`.

3) How do you find and manage SUID binaries?

```
find / -xdev -perm -4000 -type f 2>/dev/null | sort
```

Then compare against a baseline, neutralize anything unexpected with `chmod u-s`, and investigate the owning package. Typical legitimate SUIDs: `su`, `passwd`, `mount`, `sudo`, `pkexec`. Every extra SUID is privilege-escalation risk.

4) Compare `sudo` and `su`; which one do you recommend in a company? `su` hands over a full shell as another identity and asks for the target's password; `sudo` runs specific commands under an auditable policy. Recommendation: disable direct root login (`PermitRootLogin no`), use per-user SSH keys, and grant granular `sudo` rules in `/etc/sudoers.d` with central logging.

5) Users say "I cannot enter the shared folder". How do you troubleshoot? `namei -l /srv/shared/sub` (permission chain of every parent), `getfacl -p -R`, group membership with `id user` (and the need to **re-login** after `usermod -aG`), SELinux/AppArmor (`ausearch -m avc -ts recent`, `dmesg | grep -i denied`), and the parent directory's mode. In ~90% of cases a missing `x` on an intermediate directory.

6) What is the difference between `chmod 2775` and `775` on a project directory? `2775` enables the SGID bit: files created inside inherit the **directory's group** instead of the user's primary group. That is essential for team shares; without it, files get personal groups and teammates lose access.

7) `df -h` shows free space but I get "No space left on device". Why? Usually **inodes** are exhausted (`df -i`) — millions of small files or a mail queue. Fixes: locate the heavy directory, clean `/var/spool`, use `systemd-tmpfiles` / `tmpwatch`. Other causes: a deleted-but-open file (`ls -l +L1`), quota limits, or a full journal.

8) What does `find / -size +100M` do? What other `find` parameters matter? It lists files larger than 100 MB (512-byte blocks for unit-less numbers). Key predicates: `-type f/d/l`, `-mtime/-mmin`, `-user/-group`, `-perm`, `-newer`, `-maxdepth`, `-iname`, `-exec`, `-delete`, `-xdev`.

9) Difference between `rm` and `unlink`; is there any recovery? `rm` is the friendly interface (`-i/-r/-f`); `unlink` removes exactly one name (one file, no recursion). Data is truly freed when the last process closes it and the link count hits zero. Recovery: copy from `/proc/<pid>/fd/N` if a process still holds it; otherwise `extundelete`, `testdisk`, or `photorec` on an unmounted filesystem. The real answer is backups and snapshots.

10) Explain `inode` in one sentence and why it matters for storage. The metadata structure for a file that holds its data location, owner, mode, and timestamps; the filename is not part of it. Because the inode count is fixed at `mkfs` time, "many small files" workloads can exhaust inodes while space remains free.

11) In `/etc/fstab`, do you use UUIDs or `/dev/sdb1`? Why? UUIDs (or LABELs, or `/dev/mapper/...` for LVM). Kernel device names (`/dev/sdX`) can shuffle between boots when disks/USB devices change, breaking `fstab`; UUIDs are stable. For network shares use `//server/share` with `_netdev`, and `nofail` for removable devices.

12) Explain LVM's three layers and give a work scenario. PV on disks/partitions, VG as the shared pool, LV as the logical volume the file system sits on. Scenario: root filesystem is nearly full; add a disk, `pvccreate`, `vgextend`, then `lvextend -r` — the filesystem grows online with no downtime. Additional benefit: snapshots before package upgrades.

9) ☐ Day 2 Completion Checklist

- [] I built a log report with `grep/sed/awk/cut/sort/uniq`
- [] I can write octal and symbolic modes without consulting a table
- [] I changed `umask` and observed file vs directory differences
- [] I created, tested, and documented SUID/SGID/sticky
- [] I set ACLs for one user and one group and read the result with `getfacl`
- [] I granted limited `sudo` to a user and verified with `sudo -l`
- [] I completed the three-team lab (`devs/ops/audit`) end to end
- [] I practiced `find` with `-perm`, `-mtime`, `-size`, `-exec +`
- [] I partitioned a scratch disk, created a filesystem, and registered it by UUID in `fstab`
- [] I built a VG from two disks, created an LV, and grew it with `lvextend -r`
- [] I answered all 12 interview questions out loud

Day 3 — Shells & Bash Scripting

Variables • Conditionals • Loops • Error Handling • Automation

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Day 3 — Shells & Bash Scripting

Today's study path (per the 30-day plan): Morning: Shell types (09:00–10:30) → Variables (10:45–12:15) → Conditionals: if / test (12:30–13:00) **Evening:** Lab: Bash scripting (17:00–18:30) → Loops (18:45–20:15) → Script debugging (20:30–21:00)

Goal of the day: shell types, variables and environment, conditionals, loops, I/O handling, error handling (`set -euo pipefail`), and turning repetitive work into a script you can trust. This day provides the tooling for the rest of the week.

0) 3-minute review of Days 1-2

Topic	Where it reappears today
Pipes and redirection (Day 1)	Scripts manage I/O exactly the same way
<code>grep/sed/awk</code> (Day 2)	The building blocks of real scripts
Mode <code>755</code> and shebang	A script must be executable and declare an interpreter
<code>PATH</code> (Day 1)	Personal scripts live in <code>~/bin</code> or <code>/usr/local/bin</code>
<code>/etc/fstab</code> , LVM (Day 2)	Backup and disk-monitoring scripts (today's exercise)

The connecting sentence: for two days you typed commands; today you make the same commands **repeatable, documented, and safe** — that is the leap from "Linux user" to "admin/automation engineer", and it is exactly what interviews probe.

1) Shell Types and Environment (09:00–10:30)

1.1 What a shell is, and the family tree

A shell is a **command interpreter**: it reads a line, expands it (variables, globs, arguments), runs it, and reports the exit status.

Shell	Path	Traits
sh / dash	<code>/bin/sh</code>	POSIX, tiny, used by installers and initrd (<code>dash</code> on Debian)
bash	<code>/bin/bash</code>	default on most distros: history, job control, arrays, <code>[[]]</code>
zsh	<code>/bin/zsh</code>	smart completion and plugins (macOS default)
ksh	<code>/bin/ksh</code>	commercial Unix heritage
tcsh/csh	<code>/bin/csh</code>	C-like, mostly retired
fish	<code>/usr/bin/fish</code>	user-friendly, different syntax (limited scripting)

```
cat /etc/shells # shells allowed on this system
echo $SHELL # login shell
chsh -s /bin/bash ali # change a shell (sudo for others)
exec bash # replace the current shell without logging out
bash --posix script.sh # run a script in POSIX mode
```

Login vs non-login vs non-interactive: a login shell reads `/etc/profile` and one of `~/.bash_profile`, `~/.bash_login`, `~/.profile`; an **interactive non-login** shell (a new terminal, or `bash`) reads `~/.bashrc`; a **non-interactive** shell (a script) reads neither unless `BASH_ENV` is set. This is the root cause of "why are my variables missing in cron?"

```
login -> /etc/profile -> ~/.bash_profile -> ~/.bashrc -> /etc/bash.bashrc
terminal -> ~/.bashrc -> /etc/bash.bashrc
script -> inherited environment only
```

1.2 Variables: define, expand, export

```
name="Ali" # no spaces around = (spaces => "command not found")
echo "$name" # double quotes: expansion happens
echo '$name' # single quotes: literal
echo "${name}jan" # braces separate the name from adjacent text
readonly PI=3.14 # constant
unset name # remove
export PATH="$PATH:$HOME/bin" # environment variable for children
env | sort | head -20 # current environment
declare -p name # type and value
```

Shell variable vs environment variable: a shell variable lives only in the current shell; `export` puts it into the environment so child processes inherit it. Services and cron do **not** have your interactive shell — set the environment explicitly in scripts.

```
# variables worth knowing
echo "$USER $HOME $PWD $OLDPWD $SHELL $HOSTNAME"
echo "$PATH"
echo "$RANDOM" # random number
echo "$((5*3))" # integer math
date +"%F %T" # formatted time in scripts
echo "$$" # PID of this shell (useful for lock files)
echo "$?" # exit code of the last command
echo "$# $@" # count / all / first argument
echo "${1:-default}" # fallback if empty
echo "${file%.txt}" # strip the shortest suffix
echo "${path##*/}" # basename only
echo "${path%/*}" # dirname only
echo "${str//a/b}" # replace all
echo "${#str}" # string length
```

Positional and special parameters — memorize this table:

Parameter	Meaning
<code>\$0</code>	script/shell name
<code>\$1..\$9</code> , <code>\${10}</code>	arguments
<code>\$#</code>	number of arguments
<code>\$@</code>	all arguments, each quoted separately — use this in loops
<code>\$*</code>	all arguments as one string
<code>\$?</code>	exit status of the last command (0 = success)
<code>\$\$</code>	PID of the shell
<code>\$_</code>	PID of the last background job
<code>\$-</code>	active shell flags

More powerful expansions:

```
echo {1..5} # 1 2 3 4 5
echo {a..c}{1..2} # a1 a2 b1 b2 c1 c2
mkdir -p /tmp/{in,out,done} # create several directories
for f in *.log; do echo "$f"; done
$((RANDOM%100)) # arithmetic
$(command) # command substitution (avoid backticks)
```

1.3 Quoting, globbing and escaping — the source of 90% of script bugs

Form	Behaviour	Example
'...'	nothing is expanded	'\$HOME' stays \$HOME
"..."	variables and \$() expand, globs do not	"\$HOME" becomes /root
unquoted	word splitting + globbing	\$HOME can break apart
\x	neutralize a special meaning	\\$HOME stays \$HOME
\$(...)	command substitution	\$(date +%F)

```
# The deadly difference in scripts:
rm -rf $dir # if dir is empty -> rm -rf / !!!
rm -rf "$dir" # quoted, plus an empty check
[ -n "$dir" ] || { echo "dir is empty"; exit 1; }
```

✓ **Rule #1 of shell scripting: always write "\$var"**. It preserves spaces in filenames and prevents glob explosions.

1.4 Exit codes and short-circuit operators

```
ping -c1 10.20.0.1 >/dev/null 2>&1
if [ $? -eq 0 ]; then echo "UP"; else echo "DOWN"; fi

cmd1 && cmd2 # cmd2 only if cmd1 succeeded (exit 0)
cmd1 || cmd2 # cmd2 only if cmd1 failed
cmd1 ; cmd2 # both, regardless
false || echo "ran"
mkdir -p /a && cd /a && touch ok
```

Meaningful exit codes (interview material): 0 success, 1 general error, 2 misuse of a command, 126 not executable (permission), 127 command not found, 128+n killed by signal n (130 = Ctrl+C, 143 = SIGTERM), 137 = SIGKILL/OOM.

2) Variables and User Input in Scripts (10:45-12:15)

2.1 The skeleton of a professional script

```
#!/usr/bin/env bash
# name: backup.sh | purpose: back up a web directory | author/date
set -euo pipefail # -e stop on error, -u unset vars are errors, -o pipefail fail pipes
IFS=$'\n\t' # a safe word separator
shopt -s nullglob # empty globs expand to nothing

readonly SCRIPT_NAME="$(basename "$0")"
readonly BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

log() { printf '%s [%s] %s\n' "$(date +%F %T)" "${SCRIPT_NAME}" "$*"; }
die() { log "ERROR: $*" >&2; exit 1; }
trap 'log "error at line $LINENO"' ERR
trap 'log "finished with code $?"' EXIT
```

Option	Meaning and why it matters
<code>set -e</code>	stop on the first failing command (without it, the script keeps going and causes damage)
<code>set -u</code>	referencing an undefined variable is an error (prevents <code>rm -rf /</code>)
<code>set -o pipefail</code>	a failure anywhere in the pipeline fails the whole pipeline
<code>trap</code>	cleanup on exit (<code>trap cleanup EXIT</code>), capture errors and signals

2.2 Arguments and validation

```
if [ "$#" -lt 2 ]; then
    echo "usage: $0 <source-dir> <dest-dir>" >&2
    exit 2
fi
src="$1" ; dst="$2"
[ -d "$src" ] || die "source path does not exist: $src"
mkdir -p "$dst" || die "cannot create destination: $dst"

while [ "$#" -gt 0 ]; do # parse flags by hand
    case "$1" in
        -v|--verbose) VERBOSE=1 ;;
        -n|--dry-run) DRY=1 ;;
        -*) die "unknown flag: $1" ;;
        *) files+=("$1") ;;
    esac
    shift
done

getopts "vh" opt # the POSIX-standard approach
usage() { echo "usage: $0 [-v] [-h]"; }
```

If asked "how do you read user input?": `read` for interactive, `$1/$@` for parameters, `getopts` for flags — and **never** use input without validation (command injection: `${...}` inside input executes if you pass it to `eval`).

2.3 read and I/O

```
read -r -p "service name: " svc # -r keeps backslashes intact
read -r -s -p "password: " pass # -s silent
while IFS= read -r line; do echo ">> $line"; done < file.txt # the golden line-by-line loop
ls -l /var/log | head -3 | while read -r f; do echo "$f"; done
printf '%s\n' "$var" # safe printing (unlike echo with -n and %)
echo "text" >&2 # errors to stderr
exec > "/var/log/${SCRIPT_NAME}.log" 2>&1 # send everything to a log
```

3) Conditionals: `test`, `[ ]`, `[[ ]]` (12:30–13:00)

3.1 Common tests

File test	Meaning	String test	Meaning
<code>-e f</code>	exists	<code>-z "\$s"</code>	empty string
<code>-f f</code>	regular file	<code>-n "\$s"</code>	non-empty string
<code>-d f</code>	directory	<code>"\$a" = "\$b"</code>	equal
<code>-r/-w/-x f</code>	readable/writable/executable	<code>"\$a" != "\$b"</code>	not equal
<code>-s f</code>	size > 0	<code>"\$a" \< "\$b"</code>	lexicographic
<code>f1 -nt f2</code>	newer than	<code>[[\$a =~ ^[0-9]+\$]]</code>	regex (only in <code>[[]]</code>)

Numeric test	Meaning
<code>-eq</code> , <code>-ne</code>	equal, not equal
<code>-lt</code> , <code>-le</code>	less than, less or equal
<code>-gt</code> , <code>-ge</code>	greater than, greater or equal

```
if [[ -f /etc/nginx/nginx.conf && -x /usr/sbin/nginx ]]; then ... fi
if [[ "$(id -u)" -ne 0 ]]; then die "run this script as root"; fi
if [[ "$os" == "ubuntu" || "$os" == "debian" ]]; then ... fi
if (( $(echo "$load > 5.0" | bc -l) )); then echo "high load"; fi # float compare
```

`[ ]` vs `[[ ]]`: `[[ ]]` is shell syntax: no word splitting, it supports `&&` / `||`, regex (`=~`), and globs, and it will not raise "unary operator expected". Inside `[ ]` everything must be quoted and there is no `&&`. In bash always prefer `[[ ]]`; use `[ ]` only for POSIX `sh` scripts.

3.2 `if/elif/else`, `case`, and precise string handling

```
case "$1" in
  start) systemctl start nginx ;;
  stop) systemctl stop nginx ;;
  restart|reload) systemctl restart nginx ;;
  status) systemctl status nginx ;;
  *) echo "usage: $0 {start|stop|restart|status}" >&2; exit 2 ;;
esac

case "$(file -b --mime-type "$f")" in
  text/*) echo "text" ;;
  image/*) echo "image" ;;
  *) echo "other" ;;
esac
```

3.3 Security-minded patterns in conditionals

```
# strip dangerous characters from an argument
sanitize() { printf '%s' "$1" | tr -cd 'A-Za-z0-9._-/'; }
# safe temporary file
tmp="$(mktemp)" ; trap 'rm -f "$tmp"' EXIT
# single-instance lock for cron jobs
exec 9>/var/lock/myscript.lock ; flock -n 9 || die "another instance is running"
```

4) Lab 1 — Build a real script (17:00-18:30)

4.1 Scenario: a backup script with rotation and health checks

Requirements: source and destination come from arguments; create the destination if missing; timestamped `tar.gz`; keep only the last 7 copies; log to `/var/log/backup.log`; meaningful exit codes; safe to run from cron.

```
#!/usr/bin/env bash
# /usr/local/bin/backup.sh – rotating backup, keeps 7 copies
set -euo pipefail
IFS=$'\n\t'

readonly KEEP=7
readonly LOG="/var/log/backup.log"
readonly NAME="$(basename "$0")"

log() { printf '%s [%s] %s\n' "$(date +%F %T)" "$NAME" "$*" | tee -a "$LOG"; }
die() { log "ERROR: $*" >&2; exit 1; }

[[ $# -eq 2 ]] || die "usage: $NAME <source-dir> <dest-dir>"
src="$1"; dst="$2"
[[ -d "$src" ]] || die "missing source: $src"
mkdir -p "$dst" || die "cannot create destination: $dst"

stamp="$(date +%Y%m%d-%H%M%S)"
base="$(basename "$src")"
archive="${dst}/${base}-${stamp}.tar.gz"

log "starting backup: $src -> $archive"
tar -czf "$archive" -C "$(dirname "$src")" "$base" || die "tar failed"
[[ -s "$archive" ]] || die "archive is empty!"

# health check: compare source file count with archive contents
n_src=$(find "$src" -type f | wc -l)
n_arc=$(tar -tzf "$archive" | grep -cv '/$' || true)
log "source files=$n_src archived=$n_arc"
[[ "$n_arc" -ge "$n_src" ]] || die "file count mismatch!"

# rotation: drop the oldest copies
mapfile -t olds <<(ls -lt "${dst}/${base}-*.tar.gz" 2>/dev/null | tail -n +$(KEEP+1))
for f in "${olds[@]:-}"; do [[ -n "$f" ]] && rm -f -- "$f" && log "removed old: $f"; done

log "backup OK. size: $(du -h "$archive" | cut -f1)"
exit 0
```

```
sudo install -m 750 backup.sh /usr/local/bin/backup.sh
sudo /usr/local/bin/backup.sh /etc /var/backups
sudo crontab -e
# 0 2 * * * /usr/local/bin/backup.sh /etc /var/backups
```

4.2 The tests professionals run

```
bash -n backup.sh # syntax check only
shellcheck backup.sh # static analysis (apt install shellcheck) – always run it!
sudo bash -x backup.sh /etc /var/backups | head -40 # trace execution (debug)
/usr/local/bin/backup.sh /no/such/dir /tmp ; echo "exit=$?" # error path -> exit 1
for i in {1..10}; do sudo /usr/local/bin/backup.sh /etc /tmp/bk >/dev/null; done # test rotation
ls -lt /tmp/bk | wc -l # must be 7
```

4.3 Break & Fix drills

#	Break	Symptom	Diagnose/fix
1	remove the shebang (<code>#!/usr/bin/env bash</code>)	runs under <code>sh</code> : <code>[[: not found</code>	restore the shebang, <code>chmod +x</code>
2	<code>chmod 644 backup.sh</code>	<code>Permission denied</code> on execution	<code>chmod 750</code>
3	drop the quotes around <code>\$src</code> and use a path with a space	half path, <code>tar: ... not found</code>	<code>"\$src"</code>
4	comment out <code>set -e</code> and break <code>tar</code>	rotation runs against an empty archive	restore <code>set -euo pipefail</code>
5	<code>KEEP=0</code>	every backup is deleted	validate input: <code>[[\$KEEP -ge 1]]</code>
6	CRLF line endings (edited on Windows)	<code>bad interpreter: ^M</code>	<code>dos2unix</code> or <code>sed -i 's/\r\$//'</code>

5) Loops (18:45-20:15)

5.1 `for` — four forms

```
for i in 1 2 3; do echo "$i"; done
for i in {1..5}; do echo "$i"; done
for i in $(seq 10 -2 2); do echo "$i"; done
for f in /var/log/*.log; do echo "== $f =="; done
for ((i=0; i<5; i++)); do echo "counter: $i"; done # C style
for host in 10.20.0.{1..5}; do ping -c1 -W1 "$host" >/dev/null && echo "$host UP" || echo "$host DOWN"; done
while read -r user uid; do echo "$user:$uid"; done <<(awk -F: '{print $1, $3}' /etc/passwd)
```

5.2 `while` and `until`

```
i=0 ; while [[ $i -lt 3 ]]; do echo "$i"; ((i++)); done
until systemctl is-active --quiet nginx; do echo "waiting for nginx..."; sleep 2; done
while :; do echo "forever (Ctrl+C to stop)"; sleep 1; done
while true; do
    if ping -c1 -W1 10.20.0.1 >/dev/null; then break; fi
    sleep 2
done
```

Counters in bash: `((i++))`, `((i+=5))`, `let i++`, `i=$((i+1))`. Careful: `((i++))` returns **exit status 1** when `i` is 0 (because the previous value was zero), and with `set -e` that kills the script — a famous trap: use `i=$((i+1))` or `((i++)) || true`.

5.3 Loop control: `break`, `continue`

```
for u in alice bob carol; do
  [ "$u" == "bob" ] && continue # skip this one
  [ "$u" == "carol" ] && break # stop the loop
  echo "checking $u"
done
```

5.4 A real job: a daily server health report

```
#!/usr/bin/env bash
# health.sh – quick server health report (daily cron at 07:00)
set -uo pipefail
printf '%-28s %s\n' "=== health report $(hostname) $(date +%F %T) ==="
printf '%-28s %s\n' "Uptime" "$(uptime -p)"
printf '%-28s %s\n' "Load(1/5/15)" "$(cut -d' ' -f1-3 /proc/loadavg)"
printf '%-28s %s\n' "RAM" "$({free -h | awk '/Mem:/{print $3"/"$2" used, \"$7\" avail\"}'}"
printf '%-28s %s\n' "Swap" "$({free -h | awk '/Swap:/{print $3"/"$2\"}'}"
echo "--- file systems above 80% ---"
df -hP | awk 'NR>1 && int($5) >= 80 {print $6, $5}'
echo "--- inodes above 80% ---"
df -iP | awk 'NR>1 && int($5) >= 80 {print $6, $5}'
echo "--- failed services ---"
systemctl --failed --no-legend | awk '{print $1}' || true
echo "--- last 5 sshd failures ---"
journalctl -u ssh -n 200 --no-pager 2>/dev/null | grep -i "failed" | tail -5
echo "--- listening sockets ---"
ss -tulnp | grep -v "127.0.0.1\|:::1" | head -10
```

6) Error Handling and Debugging (20:30–21:00)

6.1 Debugging tools

```
bash -n script.sh # syntax check without executing
bash -x script.sh # trace line by line (customize with PS4: export PS4='+$LINENO: ')
set -x ; ... ; set +x # trace just a section
shellcheck -S style script.sh # style advice too
time script.sh # measure runtime
strace -f -e trace=openat,execve script.sh 2>&1 | tail -30 # syscalls
bash -c 'echo "$BASH_VERSION"'
```

```
trap 'echo "error at line $LINENO, code $?" >&2' ERR
trap 'echo "cleaning up..."; rm -f "$tmp"' EXIT
```

6.2 `exec`, child processes and environment

```
exec ls -l /tmp # replace the shell with the command (no child shell)
( cd /tmp && ls ) # subshell: the cd does not leak out
{ cd /tmp && ls; } # current shell: the cd persists
( umask 077; touch secret.txt ) # temporary setting without polluting the environment
```

`( )` vs `{ }` is a favourite interview question: a subshell isolates the environment; a brace group runs in the current shell (and needs `;` before the closing brace and a space after the opening one).

6.3 Advanced I/O

```
command > out.txt 2> err.txt # separate
command &> all.txt # merged
command 2>/dev/null # suppress errors (only when you are sure!)
exec 3< file.txt # custom descriptor for reading
while read -r l <&3; do echo "$l"; done ; exec 3<&-
cmd | tee -a log.txt | grep -i error
printf '%s\n' "${array[@]}" # print an array
declare -A map=( [a]=1 [b]=2 ) ; echo "${map[a]}" # associative array (bash 4+)
readarray -t lines < file.txt ; echo "${#lines[@]}"
```

7) ☐ Day 3 Self-Test (15 questions)

1. Difference between `'single'`, `"double"`, and unquoted?
2. Why is `rm -rf $dir` dangerous and what is the safe form?
3. Difference between `$@` and `$*`, and which one belongs in loops?
4. What do exit codes 127, 126 and 137 mean?
5. What does each part of `set -euo pipefail` do?
6. Difference between `[ ]` and `[[ ]]`, and which one works in `sh`?
7. Difference between `while read -r line; do ... done < f` and `cat f | while read ...`?
8. Difference between `( cd /tmp; ls )` and `{ cd /tmp; ls; }`?
9. Difference between `$(...)` and `${...}`?
10. Write a `case` for `start|stop|restart`.
11. What problem does `mktemp` solve?
12. Why is `flock` needed in a cron script?
13. What is the trap with `((i++))` under `set -e`?
14. What do `bash -x` and `shellcheck` each give you?
15. Why does a script work when run by hand but fail in cron? (three causes with fixes)

8) Day 3 Interview Bank (14 questions with model answers)

1) What is a shell, and why bash? The interpretation layer between the user and the kernel: read, expand, execute, manage jobs. bash is the default on most distributions, POSIX-compatible, with arrays, `[[ ]]` and job control plus abundant documentation; for portable scripts I write `#!/bin/sh` with POSIX syntax.

2) Environment variable vs shell variable? A shell variable exists only in that shell instance; `export` moves it into the environment so children inherit it. Services and cron do not inherit your interactive environment — define variables explicitly in the unit file or the crontab.

3) How do you validate input safely? Check the argument count, use `getopts` for flags, verify paths exist, `case` over allowed values, reject dangerous characters (`tr -cd 'A-Za-z0-9._-'`), quote everything, use `--` before paths to end option parsing, and never `eval` user input. It is the same principle in PowerShell/Python: allow-list, never deny-list.

4) Write a script that warns when a disk exceeds 85% (key lines).

```
df -hP | awk 'NR>1 && int($5) >= 85 {print $6, $5}' | while read -r mnt use; do
  printf 'WARN %s usage %s\n' "$mnt" "$use" | mail -s "Disk ${use} on $(hostname)" ops@example.com
done
```

Notes: use `df -P` (POSIX output), filter out `tmpfs` mounts, control log noise in cron with `MAILTO=`, and in a mature setup feed a monitoring system (Zabbix/Prometheus node_exporter) rather than raw email.

5) Explain trap with a real use case. `trap` registers handlers for signals or the pseudo-signals `EXIT`, `ERR`, `DEBUG`. Typical use: remove temp files and release locks on exit — `tmp=$(mktemp); trap 'rm -f "$tmp"' EXIT` — essential for long-running and cron scripts so failures do not leave garbage behind.

6) How do you run a script on every server? A loop over SSH (`for h in $(cat hosts); do ssh "$h" 'bash -s' < script.sh; done`), Ansible (`ansible all -m script -a script.sh`) which is the modern standard, or configuration-management tools (Puppet/Chef). Scripts for a handful of hosts, Ansible for hundreds.

7) Difference between `cmd > f 2>&1` and `cmd 2>&1 > f`? In the first, stderr is attached to wherever stdout points (the file). In the second, stderr is attached to the current terminal and stdout is then moved to the file. Order matters because redirections are evaluated left to right.

8) Difference between `bash script.sh`, `./script.sh`, and `source script.sh`? `bash script.sh`: a new interpreter, no execute bit needed. `./script.sh`: the shebang chooses the interpreter, needs `+x`. `source /.`: runs in the current shell, so variables and `cd` persist (that is why you `source ~/.bashrc`).

9) Write a parallel script that waits for all jobs.

```
for h in 10.20.0.{1..20}; do ping -c1 -W1 "$h" >/dev/null && echo "$h UP" & done
wait
echo "all probes finished"
```

With concurrency limits: `xargs -P 10 -I{} sh -c 'ping -c1 -W1 {} >/dev/null && echo {} UP' < hosts`, or GNU `parallel` with an explicit job count.

10) How do you make `sed -i` on config files safe inside a script? Anchor the pattern precisely instead of a loose match, use `-i.bak`, create a timestamped copy beforehand, validate the service after the change (`nginx -t`, `sshd -t`), and roll back automatically on failure with a `trap`. Never run recursive `sed -i` across a whole directory of configs.

11) Why do exit codes matter in automation? Automation tools (Ansible, Jenkins, cron, systemd) judge success/failure **only** from the exit code. A script that always returns 0 hides failures and produces no alerts; a professional script returns distinct codes for "bad input", "execution failure", and "dependency failure".

12) `$()` vs backticks — why prefer the former? Both do command substitution; `$()` nests cleanly (`$ (echo $(date +%F))`) and is clear with quoting, while backticks become unreadable and fragile once nested or embedded in strings.

13) A script should be "idempotent" — what does that mean and how do you achieve it?

Running it repeatedly produces the same result without damage. Techniques: check preconditions before acting (`[[ -f f ]] || touch f`), use `mkdir -p` instead of `mkdir`, use `systemctl enable --now` (idempotent triggers), timestamped backups, and rewrite whole config files instead of appending blindly.

14) What does exit code 137 mean for a service, and how do you debug it? $128 + 9 = \text{SIGKILL}$; the most common cause is the **OOM killer**. Check `dmesg | grep -i -E "oom|killed process"`, `journalctl -k | grep -i oom`, container/cgroup limits (`systemctl show -p MemoryMax`), measure real usage with `ps_mem` / `smem`, then fix by limiting, adding RAM, or repairing the leak.

9) 📅 Day 3 Completion Checklist

- [] I tested login vs non-login behaviour with `~/.bash_profile` and `~/.bashrc`
- [] I practiced `export`, `unset`, `readonly`, `declare` and `${var...}` expansions
- [] I built `backup.sh`, ran it, and verified 7-copy rotation
- [] I used `set -euo pipefail` and `trap` in a script
- [] I parsed flags with `getopts` and `case`
- [] I wrote `for` loops (C style, list, glob) and `while read -r`
- [] I ran `health.sh` and scheduled it daily in `crontab`
- [] I debugged a script with `bash -x`, `bash -n` and `shellcheck`
- [] I used `flock` to prevent overlapping runs
- [] I completed the six Break & Fix drills for this day

Day 4 — Process Management & Job Control

ps/top • Signals • nice/ionice • Job Control • cron, at & systemd Timers

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Week 2 • Day 4 — Morning 09:00–13:00 • Evening 17:00–21:00

Day 4 — Process Management & Job Control

Today's study path (per the 30-day plan): Morning: Processes overview (09:00–10:30) → `ps`, `top`, `pstree`, `kill` (10:45–12:15) → `nice`, `renice`, `nohup` (12:30–13:00) Evening: Lab: job control (17:00–18:30) → `nice`, `renice` (18:45–20:15) → `cron` and scheduling (20:30–21:00)

Goal of the day: understand how processes are created and scheduled, observe and prioritize them, master signals and the difference between `SIGTERM` / `SIGKILL`, drive job control in a terminal, and schedule work with `at` / `cron` / `systemd timers`.

0) 3-minute review of Days 1-3

Topic	Where it shows up today
<code>/proc/<pid></code> (Day 1)	Every process has a directory in <code>/proc</code> ; the source of <code>ps</code> , <code>top</code>
Permissions and ownership (Day 2)	Only the owner/root may signal a process → <code>kill</code> returns <code>EPERM</code>
<code>#!</code> , <code>&</code> and <code>wait</code> (Day 3)	Same job control, deeper: <code>jobs</code> , <code>fg</code> , <code>bg</code>
<code>cron</code> in the backup example (Day 3)	Today in full: <code>crontab</code> , <code>anacron</code> , <code>at</code> , <code>systemd timer</code>
<code>\$?</code> and exit codes (Day 3)	A process exit code is the language of monitoring tools

The connecting sentence: until yesterday you wrote what should run; today you learn **what is running right now, how much it consumes, and when it must die** — the daily work of troubleshooting "the server is slow".

1) What Is a Process? (09:00–10:30)

1.1 Process, program, thread

Concept	Definition
Program	A file on disk; static and passive
Process	A running program with a PID, address space, owner, and state
Thread	A parallel execution stream inside a process; threads share memory
Daemon	A background process with no controlling terminal (<code>sshd</code> , <code>nginx</code> , <code>systemd</code>)

`fork` and `exec` — the two fundamental syscalls: 1. The shell calls `fork` to create a copy of itself (the child process). 2. The child calls `exec` to replace its memory image with a new program. 3. The parent waits with `wait` for the exit code.

```
strace -f -e trace=clone,execve,wait4 bash -c 'ls /tmp' 2>&1 | head -20 # see the flow live
```

1.2 Process states, zombies and orphans

State	Meaning	Remedy
Running (R)	On CPU or queued for it	—
Sleeping (S/D)	Waiting for an event; D is uninterruptible I/O wait	check disk/mounts
Stopped (T)	Suspended (Ctrl+Z or SIGSTOP)	<code>fg / bg / kill -CONT</code>
Zombie (Z)	Finished, but the parent never <code>wait</code> ed	fix the parent; restart it if broken
Orphan	Parent died; <code>init</code> /systemd (PID 1) adopts it	nothing to do

```
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/ {print}' # hunt zombies
ps -eo pid,ppid,user,stat,cmd --sort=-pcpu | head -10 # top CPU consumers
pstree -p 1234 # process tree
```

A zombie consumes nothing except a process-table entry; when they pile up, the **parent** is broken, not the child. (Classic interview question.)

1.3 PID and namespaces

```
echo $$ # the current shell's PID
pgrep -a nginx # PID + command line
pidof nginx # PID only
ls -d /proc/[0-9]* | wc -l # current process count
cat /proc/$$/status | head -20 # details of this process
cat /proc/1/cmdline | tr '\0' ' ' ; echo # what PID 1 is (systemd/init/container main)
systemctl status <svc> | head -5
```

PID 1 is `systemd` (or `init`) on traditional servers and usually the main application in a container. If PID 1 does not handle signals, `docker stop`'s `SIGTERM` is ignored and after the timeout the container is `SIGKILL` ed (a direct bridge to Day 5 and container work).

2) Seeing Processes: `ps`, `top`, `htop`, `uptime` (10:45-12:15)

2.1 `ps` — the essential combinations

```
ps aux # BSD style: user, CPU, MEM, start time, command
ps -ef # System V style: PPID plus different columns
ps aux --sort=-%mem | head # top memory consumers
ps -ef | grep -v grep | grep nginx
ps -eo pid,ppid,user,stat,%cpu,%mem,etime,cmd
ps -eLf # show threads (LWPs)
ps -C nginx -o pid,etime,%cpu
ps -p 1234 -o pid,cmd,lstart # when did this process start
watch -n2 'ps aux --sort=-%cpu | head -8'
```

Decoding STAT: **R** running, **S** sleeping, **D** disk sleep, **T** stopped, **Z** zombie, **<** high priority, **N** low priority (positive nice), **s** session leader, **l** multi-threaded, **+** foreground process group.

2.2 **top** — live view and the keys that matter

```
top -d 2 -o %CPU -n 1 -b > topdump.txt # batch mode (for logs/scripts)
```

Key	Action
l	per-CPU display
P / M	sort by CPU / memory
k	kill a process (asks for the PID)
r	renice
c	full command line
u	filter by user
H	show threads
f	choose columns
W	save settings to <code>~/.toprc</code>

Reading the **top header (a golden interview question):**

```
top - 10:04:12 up 3 days, 2:14, 2 users, load average: 1.20, 0.85, 0.42
Tasks: 178 total, 2 running, 176 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.9 us, 2.0 sy, 0.0 ni, 91.1 id, 0.5 wa, 0.5 hi, 0.0 si, 0.0 st
MiB Mem : 3934.0 total, 412.3 free, 1190.5 used, 2331.2 buff/cache
MiB Swap: 2048.0 total, 1900.0 free, 148.0 used. 2210.9 avail Mem
```

Field	Meaning	Warning sign
load average	runnable/uninterruptible tasks averaged over 1/5/15 min	higher than core count = queue
us/sy	user/kernel time	high sy = syscalls/drivers or an attack
wa	I/O wait	high wa = slow disk or storage fault
st	time stolen by the hypervisor	high st = noisy neighbour
buff/cache	kernel cache (reclaimable)	low free but healthy avail = normal
si/so	swap in/out	anything non-zero = memory pressure

Load-average rule: divide by the CPU count; **< 1** healthy, **1–2** busy, **> 2** trouble. Or automate it: `echo "load: $(cut -d' ' -f1-3 /proc/loadavg) cores: $(nproc)".`

2.3 htop, atop, iotop, vmstat, pidstat

```
htop -u nginx # interactive with bars and a tree (apt install htop)
atop 5 3 # system + per-process with history
iostat -xz 2 5 # disk pressure and latency (sysstat)
iotop -oPa # which process is doing I/O
vmstat 2 5 # memory/CPU/swap summary over time
sar -u 1 5 ; sar -r 1 5 # historical system stats (sysstat)
pidstat -d 2 5 # per-process I/O
free -h ; cat /proc/meminfo | head
```

Troubleshooting map for "the server got slow":

Symptom	Tools	Likely cause
High load + high CPU	<code>top</code> , <code>ps --sort=-%cpu</code>	runaway process, bad code, miner
High load + low CPU + high <code>wa</code>	<code>iostat</code> , <code>iotop</code>	slow/failing disk, NFS, heavy backup
High memory + swap	<code>free</code> , <code>vmstat</code> , <code>smem</code>	leak, insufficient RAM
Many processes + zombies	<code>ps</code> , <code>pstree</code>	app bug in child handling
Saturated network	<code>ss</code> , <code>iftop</code> , <code>nload</code> , <code>dstat -n</code>	traffic/attack (bridge to Day 6)

3) Controlling Processes: Signals, `kill`, `pkill`, `nice` (12:30-13:00 and into the evening)

3.1 Signals — the language you speak to processes

Number	Name	Default	Use
1	SIGHUP	terminate	reload configuration (daemons)
2	SIGINT	terminate	that is <code>Ctrl+C</code>
3	SIGQUIT	core dump	quit with a core
9	SIGKILL	terminate	cannot be caught or ignored — last resort
15	SIGTERM	terminate	polite request to stop (the <code>kill</code> default)
18/19	SIGCONT/SIGSTOP	continue/stop	hard suspend and resume
20	SIGTSTP	stop	that is <code>Ctrl+Z</code>
10/12	SIGUSR1/2	free	application-defined (e.g. reopen logs)
28	SIGWINCH	ignored	terminal resize

```
kill -l # list all signals
kill 1234 # default = SIGTERM (15)
kill -TERM 1234 ; kill -15 1234 # the same
kill -9 1234 # force kill (SIGKILL)
kill -HUP $(cat /run/nginx.pid) # software reload
pkill -u ali -f "python worker" # by user and command-line pattern
pkill -TERM -x nginx # exact name match
killall nginx # every instance of a name
pgrep -a -f "celery" # find first, then kill (the professional way)
ss -tlnp | grep 8080 # who holds the port
fuser -v 8080/tcp ; fuser -k 8080/tcp # who holds it / kill them (careful)
lsof -i :8080
```

⚠ **Golden rule:** send **SIGTERM** (15) first so the process can clean up and close files; keep **SIGKILL** (9) for genuinely stuck processes. **kill -9** on a database risks corrupting data files.

kill -0 PID sends no signal and merely checks that the process **exists and that you are allowed to signal it** — the standard check inside scripts:

```
if kill -0 "$pid" 2>/dev/null; then echo "still running"; else echo "gone"; fi
```

3.2 Priorities: **nice** and **renice**

Range **-20** (highest priority) to **19** (lowest); default **0**. Only root may assign negative nice values.

```
nice -n 10 tar -czf big.tgz /data # run at low priority
sudo nice -n -5 /usr/bin/important-task # high priority (needs root)
renice -n 5 -p 1234 # change a running process
sudo renice -n -10 -u www-data # all processes of a user
ionice -c3 -p 1234 # I/O priority (idle class) – complements nice
chrt -f 10 ./realtime-task # real-time scheduling (extreme caution)
ps -eo pid,ni,cmd | head
```

Interview nugget: **nice** influences CPU but **not** I/O; for heavy backups that stall a disk use **ionice -c3** or **ionice -c2 -n7**. On database servers, the professional pattern is a nightly backup + **ionice** + **nice**.

3.3 Process environment, **nohup** / **setsid** / **disown**

```
nohup long-task > /var/log/long.log 2>&1 & # survives SSH disconnects
disown -h %1 # detach a job from the shell
setsid long-task </dev/null >/tmp/t.log 2>&1 & # new independent session
sleep 1000 & jobs ; fg %1 ; bg %1 # job control
Ctrl+Z -> bg # suspend then continue in background
```

For long jobs on a server, **screen** or **tmux** is the practical standard: **bash tmux new -s work ; tmux ls ; tmux attach -t work ; Ctrl+B , D (detach) ; tmux kill-session -t work**

4) Lab 1 — Job Control and Signal Behaviour (17:00-18:30)

4.1 Job control, step by step

```
sleep 300 & # start in the background
sleep 400 &
jobs -l # jobs with PIDs and state
fg %1 # bring job 1 to the foreground
# press Ctrl+Z
jobs # now Stopped
bg %2 # continue job 2 in the background
kill %1 # kill by job number (not PID)
jobs ; wait # wait for everything
```

4.2 Experiment with signal behaviour

```
#!/usr/bin/env bash
# /tmp/sig-demo.sh — shows SIGTERM vs SIGKILL and signal handling
trap 'echo "[$(date +%T)] got SIGTERM, cleaning up and exiting"; exit 0' TERM
trap 'echo "[$(date +%T)] got SIGINT (Ctrl+C)"' INT
trap 'echo "[$(date +%T)] got SIGHUP, reloading"' HUP
trap 'echo "[$(date +%T)] exiting now"' EXIT
echo "PID=$$ running; send a signal: kill -TERM $$"
while true; do sleep 2; done
```

```
chmod +x /tmp/sig-demo.sh ; /tmp/sig-demo.sh & # run it
demo=$!
kill -HUP $demo # should print the reload message
kill -TERM $demo # should print cleanup and exit
# second experiment: no trap, compare 15 vs 9
python3 -c "import time; time.sleep(300)" & p=$!
kill -9 $p ; sleep 0.5 ; ps -p $p -o pid,stat,cmd # gone immediately
```

4.3 Simulate "the server is slow" (the golden exercise for this day)

```
# 1) create CPU load
sudo nice -n 5 bash -c 'while ;; do ;; done' & # saturate one core
uptime ; top -b -n1 | head -12
ps -eo pid,ppid,user,%cpu,cmd --sort=-%cpu | head -5

# 2) create memory/swap pressure
python3 - <<'PY' &
a=[]
while True: a.append(' ' * 50_000_000) # keeps going until OOM; stop it early with Ctrl+C/kill PY
free -h ; vmstat 2 3

# 3) clean up
pkill -f 'while : ' ; pkill -f '50_000_000'
```

After each phase, answer out loud: 1. What did load average become, and what does it say relative to `nproc`? 2. Which `top` columns jumped (`us`, `wa`, `si/so`)? 3. Which tool gave the fastest diagnosis? 4. On a real server, what is the **containment** step, and what report/actions follow?

4.4 Break & Fix drills

#	Break	Symptom	Diagnose/fix
1	<code>kill -9</code> a daemon	service will not start / stale lock file	<code>systemctl start</code> , remove the lock if needed, investigate why
2	suspend a <code>sleep</code> with <code>kill -STOP</code>	<code>ps</code> shows state <code>T</code> , zero CPU	<code>kill -CONT <pid></code>
3	occupy port 80 with <code>python3 -m http.server 80</code>	nginx fails: <code>Address already in use</code>	<code>ss -tlnp</code> filtered on <code>:80</code> , then kill the owner
4	create a zombie (child without <code>wait</code>)	state <code>Z</code> in <code>ps</code> , parent sleeping	kill/restart the parent
5	forget <code>nohup</code> and drop the SSH session	work stops half-way, takes <code>SIGHUP</code>	<code>nohup</code> / <code>tmux</code> / <code>systemd-run</code>

5) Scheduling Work: `cron`, `at`, `systemd timers` (18:45-20:15)

5.1 `cron` — the five time fields

```
* * * * * command
| | | | |
| | | | +---- day of week (0-7, 0 and 7 = Sunday; names: sun,mon,...)
| | | +----- month (1-12; names: jan,feb,...)
| | +----- day of month (1-31)
| +----- hour (0-23)
+----- minute (0-59)
```

Example	Meaning
<code>0 2 * * *</code>	daily at 02:00
<code>*/10 * * * *</code>	every 10 minutes
<code>0 */4 * * *</code>	every 4 hours
<code>30 6 * * 1-5</code>	Monday-Friday at 06:30
<code>0 0 1 * *</code>	first of the month at midnight
<code>@reboot</code>	at boot
<code>@daily</code> , <code>@weekly</code> , <code>@monthly</code> , <code>@hourly</code>	named shortcuts

```

crontab -l # my jobs
crontab -e # edit
sudo crontab -u www-data -l
ls -l /etc/cron.{d,daily,hourly,weekly,monthly}/
cat /etc/crontab ; ls /etc/cron.d/
sudo systemctl status cron # Debian/Ubuntu | crond on RHEL
journalctl -u cron --since "1 hour ago" ; grep CRON /var/log/syslog | tail

```

The six classic reasons "cron did not run" (a scoring interview answer):

1. **Cron's limited PATH** (`/usr/bin:/bin`) → use absolute paths or set `PATH=` in the crontab.
2. **Different environment** → set `HOME` , `SHELL` , `LANG` explicitly.
3. **Nowhere for output/errors to go** → `>> /var/log/job.log 2>&1` or set `MAILTO=`.
4. **Missing execute bit** on the script or a wrong shebang (`+x` , CRLF line endings).
5. **% has special meaning in crontab** (e.g. `date +%F`) → escape as `\%` or move it into the script.
6. **The cron service is stopped/disabled** → `systemctl enable --now cron`.

✓ **Professional pattern:** put the logic in a script and keep the crontab to a single line: `*/5 * * * * /usr/local/bin/check.sh >> /var/log/check.log 2>&1`

5.2 `at` — one-shot execution

```

sudo systemctl enable --now atd # the service must be running
echo "/usr/local/bin/backup.sh /etc /var/backups" | at 02:00
at -f /tmp/script.sh 10:30 2026-10-01
at -l ; atq # list jobs
at -c 3 | tail -20 # show job #3
atrm 3 # remove job #3
at now + 3 hours <<< 'wall "meeting in 3 hours"'
batch < /tmp/script.sh # run when load drops

```

cron vs at : cron is recurring, `at` is one-shot; cron's finest granularity is one minute, `at` accepts `now + 30 seconds` (approximately).

5.3 `systemd timers` — modern, logged and precise

```

# /etc/systemd/system/backup.service
[Unit]
Description=Backup the /etc directory
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh /etc /var/backups
Nice=10
IOSchedulingClass=idle

```

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Timer for the daily backup
[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true # run after boot if the host was down at that time
RandomizedDelaySec=300 # spread load across many servers
Unit=backup.service
[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers --all ; systemctl status backup.timer
systemd-analyze calendar 'Mon..Fri *-*-* 09:00' # test a calendar expression
journalctl -u backup.service -n 30
```

Timer advantage over cron	Why
Complete logs	<code>journalctl -u <unit></code>
Dependencies and ordering	<code>After=</code> , <code>Requires=</code>
Catch-up execution	<code>Persistent=true</code>
Resource control	<code>Nice</code> , <code>IOSchedulingClass</code> , <code>MemoryMax</code>
Load spreading	<code>RandomizedDelaySec</code>

5.4 Hands-on exercise for this block

```
# 1) a cron job that writes the time every minute (quick test)
( crontab -l 2>/dev/null; echo "* * * * * date +\%F\ \%T >> /tmp/cron-test.log" ) | crontab -
sleep 120 ; tail -3 /tmp/cron-test.log
crontab -l | grep -v cron-test | crontab - # cleanup

# 2) build a timer with OnCalendar=minutely and watch it in journalctl
# 3) use `at now + 1 minute` to write a message and observe the result
```

6) 📅 Day 4 Self-Test (15 questions)

1. In one sentence each: program, process, thread?
2. What do `fork` and `exec` do?
3. What is a zombie and does it consume CPU? How is it cleared?
4. What does `load average: 8.0` mean on a machine with 4 cores?
5. In `top` , what does a high `wa` tell you, and which tool goes deeper?
6. Why is `buff/cache` not "bad memory usage"?
7. Difference between `SIGTERM` and `SIGKILL` , and the correct order of use?
8. What does `kill -HUP` do to a daemon?
9. What is the practical use of `kill -0 PID` ?

10. What is the `nice` range and who may assign negative values?
 11. Why is `ionice` necessary if you already set `nice`?
 12. Difference between `bg` and `disown`?
 13. Write the five cron fields and interpret `15 3 * * 1-5`.
 14. Why does a script work interactively but not from cron? Three causes.
 15. Three advantages of `systemd timers` over `cron`?
-

7) Day 4 Interview Bank (14 questions with model answers)

1) How is a process created on Linux? Via `fork` from a parent process (or `clone` / `vfork`), followed by `execve` to load the new program; the parent collects the exit code with `wait`. `fork` is the only way to create a process on Linux — this question comes up constantly.

2) "The server is slow" — what is your first window? `uptime` (load relative to `nproc`), `top` / `htop` for CPU, memory, and `wa`, `vmstat 2 5` for memory/swap pressure, `iostat -xz 2 5` / `iotop` for disk, `ss -s` for network. Then identify the culprit with `ps --sort=-%cpu|-%mem` and `pidstat`, and go deeper with `strace -p` or `perf top`.

3) Difference between `ps aux` and `ps -ef`? Both list all processes; BSD-style `aux` shows `%CPU/%MEM/VSZ/RSS` and `COMMAND`, System V `-ef` shows `PPID` and `STIME`. You pick based on the output you need and distro habits.

4) How do you stop a process completely and safely? First identify it correctly with `pgrep/pstree`, then send `SIGTERM` for a clean exit (closing files/connections), wait a few seconds and check with `kill -0`, then `SIGKILL` only if necessary. For `systemd` services I use `systemctl stop`, which manages that same sequence with a timeout. I never start with `-9`.

5) Create a zombie and explain how you destroy it. A zombie is a child that exited without the parent calling `wait`; it holds only a process-table slot. The real fix is correcting the parent (`wait` / `waitpid`, handling `SIGCHLD`) or restarting it; you cannot kill a zombie directly because it is already dead. `init` / `systemd` reaps orphaned zombies.

6) What does `nohup` do and why is it a lifesaver? It ignores `SIGHUP` and detaches standard I/O from the terminal so an SSH disconnect does not kill the job: `nohup cmd > log 2>&1 &`. For long jobs on servers I prefer `tmux` / `screen` because I can reattach interactively, and for serious workloads `systemd-run --unit`, which adds restart, logging, and resource controls.

7) A heavy backup is slowing the database server — what do you do? Run the backup with `nice -n 19` plus `ionice -c3` (or `-c2 -n7`), cap I/O with `ionice/cgroup IOWeight`, move the backup to a low-traffic window, use LVM/disk snapshots instead of reading live files, compress with `--rsyncable` or `pigz` with limited threads, and watch `iostat` during the run.

8) Difference between `kill`, `pkill`, `killall`? `kill` takes a PID or job spec; `pkill` matches by name/user/command-line pattern (risk of over-matching); `killall` targets every instance of an exact name. The most professional path is `pgrep -a` to confirm, then an explicit `kill`.

9) How do you find what is listening on port 80?

```
ss -tlnp 'sport = :80' # or ss -tlnp | grep :80
lsof -i :80 # PID and process
fuser -v 80/tcp
systemctl status <owner> # finally, identify the owning service
```

If you get **Permission denied** for the PID, re-run with **sudo** (process details are restricted).

10) Difference between Ctrl+C, Ctrl+Z and Ctrl+D? Ctrl+C sends **SIGINT** (terminate); Ctrl+Z sends **SIGTSTP** (suspend and return to the prompt); Ctrl+D sends end-of-input (**EOF**), which in an empty shell means logout. The interview-worthy nuance: **Z** does not kill the process, it only suspends it.

11) cron or systemd timer — which do you recommend in production? Both: timers give centralized journal logs, **Persistent=true**, resource controls, and dependencies, and they are my default on systemd hosts; cron remains useful for very simple jobs, legacy scripts, and systems without systemd. What matters is that the job is idempotent, logged, and monitored (e.g. exit code pushed to Zabbix/Prometheus).

12) Where does at fit? Timed one-shot work: tonight's maintenance, a scheduled service restart, reminders/notifications. **at -l**, **atrm**, and **batch** (run when idle). Recurring work belongs in cron or a timer.

13) When do you reach for strace, and how do you get to an answer fast? When a program hangs "for no reason" or fails without logs:

```
strace -f -p <pid> -e trace=network -tt # network/connection trouble
strace -f -e trace=openat,stat ./app 2>&1 | grep -i "ENOENT|EACCES" # missing files/permissions
strace -c -f ./app # syscall time summary
```

Use it carefully in production (overhead) and only for short windows.

14) Three important uses of lsof? Viewing a process's open files (**lsof -p <pid>**), finding deleted-but-open files that still hold space (**lsof +L1**), and identifying the owner of a port or connection (**lsof -i :443**, **lsof -i @10.20.0.5**). It also shows who is blocking an **umount** (**lsof +D /mnt**).

8) 📅 Day 4 Completion Checklist

- [] I ran **ps aux**, **ps -ef**, **ps -eo ... --sort** and can explain every column
- [] I practiced **top** keys **l P M c u k r H f**
- [] I can read the **top** header field by field (load, us/sy/wa/st, cache, swap)
- [] I observed system pressure with **vmstat**, **iostat**, **iotop**, **free**
- [] I exercised the important signals and saw the 9 vs 15 difference
- [] I applied **nice** / **renice** / **ionice** to test processes
- [] I used **nohup**, **disown** and **tmux** for long-running work
- [] I built the "slow server" scenario, diagnosed it, and cleaned up
- [] I scheduled a correct **crontab** entry (PATH and logging handled) and saw it run
- [] I created a **systemd timer** with **Persistent=true** and verified it in **journalctl**
- [] I answered all 14 interview questions out loud

Day 5 — System Services & the Boot Process

systemd Units • Targets • GRUB • initramfs • journald & logrotate

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Week 2 • Day 5 — Morning 09:00-13:00 • Evening 17:00-21:00

Day 5 — System Services & the Boot Process

Today's study path (per the 30-day plan): Morning: systemd overview (09:00-10:30) → Managing services (10:45-12:15) → Boot process & GRUB (12:30-13:00) Evening: Lab: systemctl (17:00-18:30) → GRUB configuration (18:45-20:15) → Logs with journalctl (20:30-21:00)

Goal of the day: know the boot chain from BIOS/UEFI to systemd, write a unit and manage a service, control GRUB and kernel parameters, and search logs like a professional with journald.

0) 3-minute review of Days 1-4

Topic	Where it shows up today
/boot/vmlinuz, initramfs (Day 1)	The exact place of each piece in the boot chain
File systems and fstab (Day 2)	A broken fstab stops boot in emergency mode
trap and exit codes (Day 3)	Type=oneshot units judge success by the exit code
Processes, signals, PID 1 (Day 4)	systemd is PID 1 and cycles services with signals
kill -HUP reload (Day 4)	systemctl reload and daemon-reload are the same idea
cron (Day 4)	Compared with systemd timers, and used for boot recovery

The connecting sentence: for four days you have managed what is running; today you learn **how those processes got there** (the boot chain) and **how they must always come up correctly** (services and dependencies).

Direct bridge to Week 1: the Linux boot chain mirrors a router's start-up: power-on self test ↔ router POST, bootloader ↔ BootROM/ROMMON, kernel ↔ IOS image, services ↔ startup-config. Know the mapping — and be ready to name the differences (Linux is multi-service and modular; IOS is monolithic).

1) systemd Overview (09:00-10:30)

1.1 What systemd is and why it replaced SysV init

Feature	SysV init (<code>/etc/init.d</code>)	systemd
Start-up model	numbered shell scripts (<code>S01nginx</code>)	declarative units
Parallelism	sequential (slow)	parallel with dependency resolution
Dependencies	coarse (runlevel numbers)	explicit (<code>After=</code> , <code>Wants=</code> , <code>Requires=</code>)
On-demand activation	none	socket/DBus activation
Logging	plain text	structured journal with timestamps and priorities
cgroups / resource limits	none	per-service resource control
Tooling	<code>service</code> , <code>chkconfig</code> , <code>update-rc.d</code>	<code>systemctl</code> , <code>journalctl</code> , <code>systemd-analyze</code>

Core concepts: a `unit` is the smallest managed object; important types are `.service` , `.socket` , `.timer` , `.target` , `.mount` , `.path` , `.device` . A `target` replaces the old runlevel (`multi-user.target` $\approx$ runlevel 3, `graphical.target` $\approx$ 5).

```
systemctl list-units --type=target --all
systemctl get-default ; sudo systemctl set-default multi-user.target
```

1.2 The three "states" of a service — the golden interview question

Question	Command	Meaning
Is it running now?	<code>systemctl is-active nginx</code>	active / inactive / failed
Will it start at boot?	<code>systemctl is-enabled nginx</code>	enabled / disabled / static / masked
Everything at once	<code>systemctl status nginx</code>	both plus PIDs and recent logs
Why did it fail?	<code>systemctl --failed</code> , <code>journalctl -u nginx</code>	the actual error

```
# the standard, safe way to enable + start
sudo systemctl enable --now nginx

systemctl list-units --type=service --state=running
systemctl list-unit-files --state=enabled
systemctl cat nginx # the main unit plus any drop-ins
systemctl show nginx -p MainPID -p ExecStart -p Restart
systemctl edit nginx # create a drop-in at /etc/systemd/system/nginx.service.d/override.conf
systemctl daemon-reload # after ANY unit change
systemctl restart|reload|try-restart nginx
systemctl mask nginx # prevent any start, even as a dependency
```

⚠ **disable** vs **mask**: **disable** removes the boot symlinks (you can still start it manually); **mask** symlinks the unit to `/dev/null` and blocks every path to starting it. Use **mask** for "must never run".

1.3 A service unit template with the important keywords

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application Service
Documentation=man:myapp(8) https://example.com/docs
After=network-online.target
Wants=network-online.target
Requires=mysql.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple # simple | forking | oneshot | notify | dbus | idle
User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
EnvironmentFile=-/etc/myapp/env # - tolerates a missing file
Environment=APP_MODE=production PORT=8080
ExecStartPre=/usr/bin/test -f /opt/myapp/config.yml
ExecStart=/opt/myapp/bin/myapp --config /opt/myapp/config.yml
ExecReload=/bin/kill -HUP $MAINPID # support for reload
ExecStop=/bin/kill -TERM $MAINPID
Restart=on-failure # no | on-failure | on-abnormal | always
RestartSec=5
TimeoutStopSec=20
KillMode=mixed
# security hardening (bridge to Week 3)
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/var/lib/myapp /var/log/myapp
# resource control
LimitNOFILE=65535
MemoryMax=1G
CPUQuota=150%
Nice=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Type values — a table that earns points:

Type	When	Example
<code>simple</code> (default)	the main process stays alive from <code>ExecStart</code> onward	nginx, myapp
<code>forking</code>	the process forks and the parent exits	legacy services
<code>oneshot</code>	does one job and finishes; <code>RemainAfterExit=yes</code> keeps it "active"	mounts, configuration scripts
<code>notify</code>	the app signals readiness via <code>sd_notify</code>	modern services
<code>idle</code>	run after everything else	deferred work

1.4 Know the essential services

```
systemctl status sshd|ssh # remote access
systemctl status cron|crond # scheduling
systemctl status NetworkManager # networking (desktop/modern server)
systemctl status systemd-networkd, systemd-resolved, systemd-timesyncd
systemctl status firewalld|ufw # firewall (Day 6)
systemctl status rsyslog|systemd-journald
systemctl list-dependencies multi-user.target # dependency tree
```

2) Managing Services in Practice (10:45-12:15)

2.1 The anatomy of a professional troubleshooting loop (memorize this pattern)

```
sudo systemctl status myapp -l --no-pager # 1) state and recent logs
journalctl -u myapp -n 50 --no-pager # 2) service log
journalctl -u myapp --since "10 min ago" -p err
systemctl cat myapp # 3) unit contents
sudo myapp --config /opt/myapp/config.yml # 4) run by hand: see the error
sudo -u myapp /opt/myapp/bin/myapp # 5) test as the service user (perms/home)
systemctl show myapp -p ExecStart -p User -p MainPID
ss -tlnp | grep 8080 # 6) port/dependency check
systemctl list-dependencies myapp # 7) dependencies
```

Five common causes of service failure in production:

- File/directory permissions** (the service user cannot read logs/data) → `ls -l`, `sudo -u user test -r ...`
- Port already in use** (`Address already in use`) → `ss -tlnp`
- Missing environment variable** → `EnvironmentFile`, test with `sudo -u user env`
- Dependency not ready** → `After=`, `Wants=` and `network-online.target`
- SELinux/AppArmor restriction** → `auresearch -m avc -ts recent`, `journalctl -k | grep -i denied`

2.2 Compact command set

```
systemctl start|stop|restart|reload|status <svc>
systemctl is-active|is-enabled|is-failed <svc>
systemctl enable|disable|mask|unmask <svc>
systemctl list-units --failed ; sudo systemctl reset-failed
sudo systemctl daemon-reexec # re-exec systemd itself (after systemd upgrades)
systemctl list-unit-files --type=service | grep enabled | wc -l
systemd-analyze # total boot time
systemd-analyze blame | head -15 # slowest services
systemd-analyze critical-chain # the critical boot chain
systemd-analyze verify /etc/systemd/system/myapp.service # unit validation
```

3) The Boot Process and GRUB (12:30-13:00)

3.1 The complete boot chain (recite it from memory)

```
[1] Firmware: BIOS or UEFI
    BIOS: POST, read the MBR (first 446 bytes), chain to the bootloader
    UEFI: POST, read the ESP (FAT32 partition) and the EFI bootloader
    |
    v
[2] Bootloader: GRUB2 (or systemd-boot)
    grub.cfg -> choose kernel + initramfs + kernel command line
    |
    v
[3] Kernel: loads vmlinuz into memory, then initramfs
    initramfs = a small ramdisk with the drivers needed to reach the real root
    |
    v
[4] Kernel: mounts root (or LVM/LUKS) and runs /sbin/init (systemd) as PID 1
    |
    v
[5] systemd: reaches default.target, resolving dependencies in parallel
    |
    v
[6] login: getty/display-manager -> user login
```

The three tables that go with this chain:

Stage	File/component	Symptom when broken
Firmware	MBR / ESP	disk will not boot
Bootloader	<code>/boot/grub/grub.cfg</code>	no GRUB menu, or the wrong kernel boots
Kernel + initramfs	<code>/boot/vmlinuz-*</code> , <code>/boot/initrd.img-*</code>	Panic: <code>VFS: Unable to mount root fs</code>
Root mount	<code>/etc/fstab</code> , LVM/LUKS	emergency mode (<code>Give root password for maintenance</code>)
systemd	units and targets	a service fails while the system boots fine
Login	<code>getty</code> , <code>sshd</code>	console/remote login impossible

3.2 initramfs: what lives inside?

```
ls -l /boot/initrd.img-$(uname -r)
lsinitramfs /boot/initrd.img-$(uname -r) | head -30 # list contents
lsinitramfs /boot/initrd.img-$(uname -r) | grep -E "crypt|raid|lvm|ext4|dm-" # critical modules
sudo update-initramfs -u -k $(uname -r) # rebuild after critical changes
sudo dracut -f # RHEL/Fedora
```

If you add or change an LVM or LUKS driver (`/etc/crypttab`, `/etc/fstab`), **rebuilding initramfs is mandatory** — otherwise the next boot ends in a panic. This is one of the most common rookie-admin mistakes.

3.3 GRUB — real control over boot

```
cat /etc/default/grub # the main settings (edit here)
# example:
# GRUB_TIMEOUT=5
# GRUB_DEFAULT=0 # or "1>2" for nested menus, or an entry name
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
# GRUB_CMDLINE_LINUX=""
# GRUB_DISABLE_RECOVERY="false"

sudo update-grub # Debian/Ubuntu: regenerate /boot/grub/grub.cfg
sudo grub2-mkconfig -o /boot/grub2/grub.cfg # RHEL BIOS
sudo grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg # RHEL UEFI
ls /etc/grub.d/ # 00_header, 10_linux, 40_custom, ...
```

```
# add a kernel parameter temporarily: press 'e' in the GRUB menu
# - single-user mode for root password recovery
# - or: GRUB_CMDLINE_LINUX="systemd.unit=multi-user.target" to skip graphical
# - or: init=/bin/bash for a direct shell (carefully, then remount rw)
```

Useful kernel parameters: | Parameter | Effect | |---|---| | `quiet splash` | fewer messages (remove for debugging!) | | `ro` / `rw` | mount root read-only / read-write | | `init=/bin/bash` | run a shell instead of init (rescue) | | `systemd.unit=rescue.target` | boot into rescue | | `single` / `1` | single-user mode | | `nomodeset` | work around graphics/driver issues | | `net.ifnames=0` | legacy interface naming (`eth0`) | | `acpi=off`, `noapic` | work with faulty hardware |

3.4 Boot ordering, generators and `systemd-analyze`

```
systemd-analyze ; systemd-analyze blame ; systemd-analyze critical-chain  
systemd-analyze plot > /tmp/boot.svg # boot chart (very useful)  
systemctl list-dependencies --reverse sshd  
systemctl list-jobs
```

4) Lab 1 — Build and manage a service with `systemctl` (17:00-18:30)

4.1 Scenario: turn a small Python app into a service

```

# 1) the sample app
sudo mkdir -p /opt/hello/bin /var/log/hello /var/lib/hello
sudo tee /opt/hello/bin/hello.py >/dev/null <<'PY'
#!/usr/bin/env python3
import http.server, socketserver, os
PORT = int(os.environ.get("PORT", "8080"))
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"hello from systemd\n"
        self.send_response(200); self.send_header("Content-Type","text/plain")
        self.send_header("Content-Length", str(len(body))); self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass
socketserver.TCPServer.allow_reuse_address = True
with socketserver.TCPServer(("0.0.0.0", PORT), H) as httpd:
    httpd.serve_forever()
PY
sudo chmod 755 /opt/hello/bin/hello.py

# 2) a service user without a login shell
sudo useradd --system --home /opt/hello --shell /usr/sbin/nologin helloapp
sudo chown -R helloapp:helloapp /opt/hello /var/log/hello /var/lib/hello

# 3) the unit file
sudo tee /etc/systemd/system/hello.service >/dev/null <<'UNIT'
[Unit]
Description=Hello HTTP demo service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=helloapp
Group=helloapp
WorkingDirectory=/opt/hello
Environment=PORT=8080
ExecStart=/usr/bin/python3 /opt/hello/bin/hello.py
Restart=on-failure
RestartSec=3
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=full
ProtectHome=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
UNIT

# 4) validate, enable, and test
sudo systemd-analyze verify /etc/systemd/system/hello.service
sudo systemctl daemon-reload
sudo systemctl enable --now hello.service
systemctl status hello.service --no-pager
ss -tlnp | grep :8080
curl -sS http://127.0.0.1:8080 ; echo
journalctl -u hello -n 20 --no-pager

```

4.2 Professional experiments on the same service

```
# a) resilience: kill the process and watch systemd restart it
sudo pkill -f hello.py ; sleep 5 ; systemctl is-active hello # active (Restart)
journalctl -u hello -n 10 | tail -5

# b) change the port with a drop-in and reload
sudo systemctl edit hello # in the override: [Service] Environment=PORT=9090
sudo systemctl daemon-reload ; sudo systemctl restart hello
curl -sS http://127.0.0.1:9090 ; sudo ss -tlnp | grep 909

# c) resource limits and observation
sudo systemctl set-property hello.service MemoryMax=64M CPUQuota=20%
systemctl show hello -p MemoryMax -p CPUQuota
systemctl status hello | head -12 # cgroup consumption

# d) reload support (HUP)
# add to the unit: ExecReload=/bin/kill -HUP $MAINPID
sudo systemctl reload hello ; systemctl show hello -p ExecReload

# e) mask and forced documentation
sudo systemctl mask hello ; sudo systemctl start hello # error: masked
sudo systemctl unmask hello ; sudo systemctl start hello

# f) boot test: enable the service, reboot
sudo systemctl enable hello ; sudo reboot
# after boot: systemctl is-active hello ; systemd-analyze blame | head
```

4.3 Break & Fix drills

#	Break	Symptom	Diagnose/fix
1	wrong <code>ExecStart</code> path	<code>status: 203/EXEC</code>	<code>journalctl -u</code> ; fix the absolute path
2	<code>User=wronguser</code> (does not exist)	<code>Failed to determine user credentials</code>	create the user or fix the unit
3	<code>WorkingDirectory</code> missing	<code>200/CHDIR</code>	create the directory or drop the key
4	port already in use	<code>Address already in use</code>	<code>ss -tlnp</code> , kill the owner
5	forgetting <code>daemon-reload</code>	changes have no effect; "unit modified" warning	<code>systemctl daemon-reload</code>
6	endless restart loop	<code>start-limit-hit</code>	<code>StartLimitIntervalSec / Burst</code> , fix the bug
7	<code>ProtectHome=yes</code> while reading <code>/home</code>	unexplained <code>Permission denied</code>	<code>ProtectHome=read-only</code> or move the data
8	service does not start at boot	inactive after reboot	<code>is-enabled</code> → <code>systemctl enable</code>

5) GRUB and Boot Recovery (18:45-20:15)

5.1 Changing GRUB settings, step by step

```
# 1) back up
sudo cp /etc/default/grub /etc/default/grub.bak.$(date +%F)
sudo cp /boot/grub/grub.cfg /boot/grub/grub.cfg.bak.$(date +%F)

# 2) settings (example: 10-second menu + no quiet, so you can see boot messages)
sudo sed -i 's/^GRUB_TIMEOUT=.*GRUB_TIMEOUT=10/' /etc/default/grub
sudo sed -i 's/^GRUB_CMDLINE_LINUX_DEFAULT=.*GRUB_CMDLINE_LINUX_DEFAULT=""/' /etc/default/grub

# 3) apply
sudo update-grub # Debian/Ubuntu
sudo grub-mkconfig -o /boot/grub/grub.cfg
grep -n "menuentry\|linux" /boot/grub/grub.cfg | head
```

Adding a custom boot entry (the **40_custom** pattern):

```
sudo tee /etc/grub.d/40_custom >/dev/null <<'GRUB'
#!/bin/sh
exec tail -n +3 $0
menuentry "Debian (rescue mode)" {
    insmod ext2
    set root=(hd0,gpt2)
    linux /vmlinuz root=/dev/vg0/lv_root ro systemd.unit=rescue.target
    initrd /initrd.img
}
GRUB
sudo chmod +x /etc/grub.d/40_custom ; sudo update-grub
```

5.2 Recovering a lost root password (a favourite interview question)

Method A — from GRUB (**init=/bin/bash):**

```
1) At the GRUB menu, press 'e' on the boot entry
2) On the linux line, append: rw init=/bin/bash
3) Ctrl+X to boot
4) In the unrestricted shell: passwd root
5) remount and restart:
mount -o remount,ro /
exec /sbin/reboot -f
```

Method B — rescue mode:

```
1) In GRUB: systemd.unit=rescue.target (or single)
2) Log in with the existing root password, then run passwd
```

Method C — live media and **chroot :**

```
sudo mount /dev/vg0/lv_root /mnt && sudo mount /dev/sda1 /mnt/boot
sudo mount --bind /dev /proc /sys /run /mnt/{dev,proc,sys,run}
sudo chroot /mnt passwd root
```

5.3 Recovering from a broken `fstab` (a real exercise)

```
# 1) log in via emergency mode (root password available)
mount -o remount,rw / # root is usually read-only here
journalctl -xb | grep -i -A3 "failed to mount"
nano /etc/fstab # fix the wrong UUID/options
findmnt --verify ; mount -a # test immediately
systemctl daemon-reload ; systemctl reboot
# 2) emergency shortcuts: systemd.unit=emergency.target in GRUB, or add
# nofail / x-systemd.automount to the offending entry
```

5.4 Reinstalling GRUB, and fixing "the machine will not boot"

```
# BIOS (MBR)
sudo grub-install /dev/sda ; sudo update-grub
# UEFI
sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=debian
efibootmgr -v # UEFI boot entry order
sudo efibootmgr -o 0001,0002 # change the order
# from rescue media (chroot)
sudo mount /dev/sda2 /mnt && sudo mount --bind /dev /proc /sys /run /mnt/{dev,proc,sys,run}
sudo chroot /mnt grub-install /dev/sda && sudo chroot /mnt update-grub
```

6) Logs with `journalctl` and `rsyslog` (20:30-21:00)

6.1 `journald` — structured logging

```
journalctl # everything from the start (paged)
journalctl -b # this boot only (-b -1 = previous boot, vital for debugging)
journalctl -u nginx -f # follow a service live
journalctl -u nginx --since "today" --until "1 hour ago"
journalctl -p err -b # errors and above, this boot
journalctl -p warning..err # priority range
journalctl -k -b # kernel messages this boot (same as dmesg)
journalctl _PID=1234 # by PID
journalctl /usr/sbin/sshd # by executable
journalctl --disk-usage
sudo journalctl --vacuum-size=500M # trim to a size
sudo journalctl --vacuum-time=7d # keep 7 days
journalctl -o short-precise|json-pretty|cat
journalctl -u ssh --grep "Failed password" --since "24 hours ago" | tail -20
```

Priorities (`-p`): 0 emerg , 1 alert , 2 crit , 3 err , 4 warning , 5 notice , 6 info , 7 debug .

6.2 Persistent journald configuration

```
# /etc/systemd/journald.conf
[Journal]
Storage=persistent # keep in /var/log/journal (default on-disk: auto)
SystemMaxUse=1G
SystemKeepFree=500M
MaxRetentionSec=1month
MaxFileSec=1week
ForwardToSyslog=yes
Compress=yes
```

```
sudo mkdir -p /var/log/journal ; sudo systemd-tmpfiles --create --prefix /var/log/journal
sudo systemctl restart systemd-journald
```

6.3 rsyslog and /var/log — classic text logs

File	Contents
/var/log/syslog (Debian) / /var/log/messages (RHEL)	general system
/var/log/auth.log (Debian) / /var/log/secure (RHEL)	authentication, sudo, SSH
/var/log/kern.log	kernel messages
/var/log/dpkg.log , /var/log/apt/	package install/upgrade
/var/log/nginx/access.log , error.log	web server
/var/log/faillog , btmp , wtmp , lastlog	logins (last , lastb , lastlog)
/var/log/cron	scheduled jobs

```
logger -p user.notice "test message for rsyslog" # generate a log line (for testing)
sudo grep -i "Failed password" /var/log/auth.log | tail
sudo lastb | head ; last | head ; lastlog | head
```

Logrotate — keep the disk from filling:

```
ls /etc/logrotate.d/ ; cat /etc/logrotate.d/nginx
sudo logrotate -d /etc/logrotate.conf # dry run
sudo logrotate -f /etc/logrotate.conf # forced run
```

```
/var/log/myapp.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 myapp adm
    sharedscripts
    postrotate
        systemctl reload myapp >/dev/null 2>&1 || true
    endscript
}
```

6.4 Security hunting in logs (bridge to Week 3)

```
# failed SSH attempts and top offending IPs
journalctl -u ssh --since "24 hours ago" --grep "Failed password" | awk '{print $(NF-3)}' | sort |
uniq -c | sort -nr | head
# successful root logins
grep "Accepted" /var/log/auth.log | grep "root" | tail
# user and sudo changes
grep -E "useradd|usermod|userdel|sudo:" /var/log/auth.log | tail -20
# package changes
grep -E "install | upgrade " /var/log/dpkg.log | tail -20
```

7) ☐ Day 5 Self-Test (15 questions)

1. Name the six stages of the boot chain.
2. What is the practical difference between BIOS and UEFI at boot?
3. What is `initramfs` and when must it be rebuilt?
4. Difference between `is-active` and `is-enabled`?
5. Difference between `disable` and `mask`?
6. `Type=simple` vs `forking` vs `oneshot`?
7. How do `After=` and `Requires=` differ?
8. What does `WantedBy=multi-user.target` mean?
9. When is `daemon-reload` required, and why does `restart` not replace it?
10. Under what conditions does a service hit `start-limit-hit`?
11. Difference between `GRUB_CMDLINE_LINUX_DEFAULT` and `GRUB_CMDLINE_LINUX`?
12. Why is `update-grub` needed after editing `/etc/default/grub`, and which file must never be edited by hand?
13. What happens at boot if `/etc/fstab` is wrong, and how do you recover?
14. What does `journalctl -b -1 -p err` show?
15. What problem does `logrotate` solve, and what does `postrotate` do?

8) Day 5 Interview Bank (14 questions with model answers)

1) Walk me through the Linux boot process. Firmware (UEFI/BIOS) POSTs and finds a bootloader; GRUB2 loads the kernel and `initramfs` from `/boot`; the kernel uses `initramfs` to load the drivers it needs and mount root; `systemd` starts as PID 1 and builds `default.target` by resolving dependencies; finally `getty` /the display manager allows login. Depending on where it breaks you get a panic (kernel/root), emergency mode (`fstab`), or simply a failed service (`systemd`).

2) What does `systemd` give you over `SysV init`? Parallel start-up, explicit dependency management, socket/DBus activation, cgroup-based resource control, structured logging via `journald`, and analysis

tooling (`systemd-analyze`). Its weakness is scope creep and complexity, which is a legitimate design criticism.

3) Write a unit file for a service. (Use the template in section 1.3 with `Type` , `User` , `ExecStart` , `Restart` , `WantedBy` , and hardening like `NoNewPrivileges` , `ProtectSystem` .) I would explain that `ExecReload` enables HUP-based reloads, `EnvironmentFile` carries secrets, and `MemoryMax` / `CPUQuota` contain the service.

4) A server will not boot and shows Give root password for maintenance . Diagnosis and fix?

Usually an `fstab` mount failure (changed UUID, detached disk, corrupt filesystem). Log in to emergency mode, `mount -o remount,rw /` , read `journalctl -xb` , fix `fstab` (correct UUID, or `nofail`) or `fsck` the device, verify with `mount -a` and `findmnt --verify` , then reboot. Prevention: `nofail` for removable media and always testing `fstab` before rebooting.

5) systemctl status says the service failed . Your steps? Read the status output and exit code/signal, `journalctl -u <svc> -n 50` , `systemctl cat` for `ExecStart`/`User`, run it manually as the same user, check file permissions (`sudo -u user test -r`), occupied ports, environment variables, and SELinux/AppArmor. Then fix, `daemon-reload` , `reset-failed` , `restart` , and add monitoring/alerting so it does not silently recur.

6) How do you disable a service at boot but keep it running now? `sudo systemctl disable nginx` (and `stop` if required). To block any start at all: `mask` . For "run it just this once": `systemd-run --unit` or a `nohup` script.

7) Difference between reload and restart ? `reload` sends a signal (usually HUP) and re-reads configuration without dropping service — it requires `ExecReload` and app support; `restart` kills and starts the process (brief outage). In production, prefer `reload` for allowed config changes and `restart` for code/binary changes.

8) "Why does my service start manually but not at boot?" Usually it is not `enabled` ; or it is not wired into `multi-user.target` (`WantedBy` missing and never enabled); or it depends on something that is not ready at boot (`After=network-online.target` needed); or conditions (`ConditionPathExists` , `ConditionUser`) are false in the boot environment. Diagnose with `systemctl list-dependencies` and `journalctl -b` .

9) How do you filter journalctl to a specific service and time window? `journalctl -u nginx --since "2026-09-20 08:00" --until "2026-09-20 12:00" -p err -o short-precise` . For the previous boot use `-b -1` , for kernel events `-k` , for an executable the full path, and to locate a pattern `--grep` .

10) Where are logs stored and how do you prevent the disk filling? `journald` keeps them in `/var/log/journal` (persistent) or `/run/log/journal` (volatile) bounded by `SystemMaxUse` ; `rsyslog` writes `/var/log/*.log` . Prevention: `journald` quotas, `logrotate` for apps, monitoring space **and** inodes, shipping to a central log server (`rsyslog/syslog-ng/ELK/Graylog`), and an alert on low free space in `/var` .

11) How do you reinstall GRUB on a UEFI system? From live media/UEFI shell: mount the ESP at `/boot/efi` and the root filesystem, `chroot` , then `grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=...` and `update-grub` ; verify with `efibootmgr -v` and fix order with `efibootmgr -o` . If NVRAM entries were wiped, recreating the entry is mandatory.

12) How does systemd handle service signals differently from init? `systemd` puts each service in its own cgroup and on stop signals every process in that cgroup according to `KillMode` , which is why services that leak child processes still get fully cleaned up. It also manages the stop timeout (`TimeoutStopSec`) and the escalation to SIGKILL.

13) How do you modify a unit without touching the vendor file? With a drop-in: `systemctl edit myapp`, which creates `/etc/systemd/system/myapp.service.d/override.conf` and survives package upgrades; then `daemon-reload` and `restart`. This is the best practice and scores points in interviews.

14) If boot is slow, how do you find what is holding it up? `systemd-analyze` (total), `systemd-analyze blame` (slowest units), `systemd-analyze critical-chain` (critical path), and `systemd-analyze plot > boot.svg` for a chart. Common fixes: disable unused services, remove unnecessary `network-online.target` waits, `random` / `systemd-random-seed` delays, lower `GRUB_TIMEOUT`, and address the real culprit such as waiting on NFS or a failing disk.

9) ☐ Day 5 Completion Checklist

- [] I can explain the boot chain from UEFI to login without notes
- [] I listed initramfs contents and know when a rebuild is required
- [] I built `hello.service`, enabled it, and tested `Restart` by killing it
- [] I created and applied a drop-in with `systemctl edit`
- [] I set `MemoryMax` / `CPUQuota` on a service and observed the effect
- [] I tested the `disable` / `mask` / `enable --now` differences
- [] I edited `/etc/default/grub`, ran `update-grub`, and saw the menu
- [] I created a `40_custom` boot entry
- [] I practiced the root-password recovery scenario (on a test VM)
- [] I simulated and recovered from a broken `fstab`
- [] I practiced `journalctl` with `-b`, `-u`, `-p`, `--since`, `--vacuum-size`
- [] I wrote a custom `logrotate` policy for my own app
- [] I answered all 14 interview questions out loud

Day 6 — Linux Networking Essentials

ip/ss • DNS • tcpdump • SSH Hardening • nftables/ufw/firewalld • NAT

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Week 2 • Day 6 — Morning 09:00–13:00 • Evening 17:00–21:00

Day 6 — Linux Networking Essentials

Today's study path (per the 30-day plan): Morning: IP and hostname (09:00–10:30) → Network tools: `ping`, `netstat` / `ss` (10:45–12:15) → SSH and SCP (12:30–13:00) **Evening:** Lab: network configuration (17:00–18:30) → SSH key login (18:45–20:15) → Firewall basics (20:30–21:00)

Goal of the day: see and set addressing/routing from the Linux command line (`ip` , `nmcli` , `resolvectl`), use diagnostic tools professionally, harden SSH including key-based login, and close the host firewall with `nftables` / `firewalld` / `ufw` . This day is where Week 1 (networking) and Week 2 (Linux) meet.

0) 3-minute review of Days 1-5 + the Week 1 bridge

Topic	Where it shows up today
<code>/etc/fstab</code> , mounts (Day 2)	<code>/etc/network/interfaces</code> , <code>netplan</code> , and configuration persistence
The <code>health.sh</code> script (Day 3)	Today we add <code>ss</code> / <code>ip</code> network sections to it
systemd units (Day 5)	<code>NetworkManager</code> , <code>systemd-networkd</code> , <code>firewalld</code> are systemd services
<code>journalctl</code> (Day 5)	Network logs: <code>journalctl -u NetworkManager -b</code>
Week 1: subnetting (Day 3)	IP address/mask and network math inside <code>ip addr</code>
Week 1: routing/static (Day 4)	<code>ip route</code> , default gateway, longest prefix match
Week 1: OSPF (Day 5)	Running FRR/quagga on Linux for dynamic routing
Week 1: DHCP/NAT/ACL (Day 6)	<code>dnsmasq</code> (DHCP), <code>nftables</code> (NAT), <code>nft</code> / <code>ufw</code> (ACL)

The connecting sentence (a golden interview answer):

"Everything I did in IOS with `interface` , `ip address` , `ip route` , and `access-list` , I do on Linux with `ip addr add` , `ip route add` , and `nft add rule` ; the concepts are identical, only the implementation layer differs."

1) Addressing, Hostnames and DNS (09:00-10:30)

1.1 Viewing and setting addresses with `ip`

```
ip a # all interfaces and addresses
ip -br a # brief: UP/DOWN plus addresses
ip -br link # link state only
ip addr show dev ens33
ip -4 a ; ip -6 a
ip link set ens33 up
ip addr add 10.20.0.10/24 dev ens33
ip addr del 10.20.0.10/24 dev ens33
ip link show ens33 | head -2 # MAC and state
cat /sys/class/net/ens33/address # MAC (second source)
ethtool ens33 | head -20 # speed/duplex/driver
ethtool -S ens33 | head -20 # driver counters (CRC errors/drops)
```

⚠ **Temporary vs persistent:** `ip addr add` survives until reboot. For persistence use `nmcli` , `netplan` , `systemd-networkd` , or `/etc/network/interfaces` (section 5).

1.2 Routing with `ip route`

```
ip route # IPv4 routing table
ip -6 route
ip route get 8.8.8.8 # the path actually chosen + source address
ip route show default
ip route add 10.99.0.0/24 via 10.20.0.1 dev ens33
ip route del 10.99.0.0/24
sudo ip route add default via 10.20.0.1
ip neigh # ARP/neighbour table
ip neigh flush all # clear the ARP cache
arp -a # the legacy way (net-tools)
```

Direct mapping to Week 1:

IOS	Linux
<code>show ip interface brief</code>	<code>ip -br a</code>
<code>show ip route</code>	<code>ip route</code>
<code>ip route 10.99.0.0 255.255.255.0 10.20.0.1</code>	<code>ip route add 10.99.0.0/24 via 10.20.0.1</code>
<code>ip route 0.0.0.0 0.0.0.0 <gw></code>	<code>ip route add default via <gw></code>
<code>show arp</code>	<code>ip neigh</code>
<code>clear arp-cache</code>	<code>ip neigh flush all</code>

1.3 Hostname, resolvers and DNS services

```
hostnamectl # identity + hostname
sudo hostnamectl set-hostname srv2.lab.local
cat /etc/hostname
sudo tee -a /etc/hosts >/dev/null <<'H'
10.20.0.10 srv2.lab.local srv2
10.20.0.11 srv1.lab.local srv1
H
getent hosts srv1 ; getent passwd root # getent consults every NSS source

resolvectl status # systemd-resolved: DNS servers and domains
resolvectl query example.com
cat /etc/resolv.conf # a symlink on systemd-resolved hosts
ls -l /etc/resolv.conf
nmcli dev show | grep -i dns
```

```
# DNS tools (dnsutils/bind-utils)
dig example.com A +short
dig @1.1.1.1 example.com MX
dig +trace example.com
dig -x 8.8.8.8 +short # reverse / PTR
nslookup example.com
host example.com
getent ahostsv4 example.com
```

Name resolution order (NSS): `/etc/hosts` first (per `/etc/nsswitch.conf`: `hosts: files dns`), then DNS. That is why you can test without DNS by editing `/etc/hosts`. The question that always follows: "Why does `ping google.com` fail while `ping 8.8.8.8` works?" → it is DNS, not the network.

1.4 Ports, sockets and services

```
cat /etc/services | grep -E "^(ssh|http|https|dns|smtp|ntp|ldap|mysql|postgres)\b"
ss -tuln # TCP+UDP, listening, no name resolution
ss -tlnp # with PID/process (root sees all)
ss -tan state established
ss -s # socket summary
ss -tp dport = :443
netstat -tulnp # legacy (net-tools, usually absent now)
lsof -i :22 # which process holds port 22
nc -zv 10.20.0.10 22 # test whether a port is open
```

Important port	Service
22	SSH
25/587/465	SMTP/Submission
53 (udp/tcp)	DNS
67/68 udp	DHCP
80/443	HTTP/HTTPS
123 udp	NTP
389/636	LDAP/LDAPS
3306/5432	MySQL/PostgreSQL
161/162 udp	SNMP (Week 1 link — snmp-server community)

2) Network Troubleshooting Tools (10:45-12:15)

2.1 Layer by layer — the standard order

#	Layer	Tools	Question you are answering
1	Link	<code>ip -br link</code> , <code>ethtool</code>	Is the NIC up? Cable/speed?
2	Address	<code>ip -br a</code>	Correct IP and mask?
3	Neighbour	<code>ip neigh</code> , <code>arping</code>	Does the gateway's MAC resolve?
4	Route	<code>ip route get</code> , <code>traceroute</code>	Correct path and gateway?
5	Name	<code>dig</code> , <code>getent</code>	Does DNS answer?
6	Port/service	<code>ss</code> , <code>nc</code> , <code>curl</code>	Is the service listening? Is a firewall blocking?
7	Application	<code>curl -v</code> , <code>tcpdump</code>	What does the app layer say?

```

ping -c4 8.8.8.8
ping -c4 -W2 -I ens33 10.20.0.1 # bind to a source interface
ping6 -c2 2001:4860:4860::8888
ping -M do -s 1472 10.20.0.1 # MTU test (1472+28=1500)
traceroute -n 8.8.8.8 # path (UDP by default)
traceroute -T -p 443 8.8.8.8 # TCP/443 (crosses more firewalls)
mtr -rwzc 10 8.8.8.8 # live ping+traceroute with a report (best tool)
arping -c3 -I ens33 10.20.0.1 # ARP test
curl -sS -o /dev/null -w "%{http_code} %{time_total}s %{remote_ip}\n" https://example.com
curl -v --resolve srv.lab.local:443:10.20.0.10 https://srv.lab.local/ # test without DNS
wget -S --spider http://10.20.0.10/ # headers only
nc -zv 10.20.0.11 1-1024 2>&1 | grep -i succeeded # port scan (our own lab only!)

```

2.2 `tcpdump` — the network microscope (the most important tool today)

```

sudo tcpdump -i any -nn -c 20 # 20 packets on all interfaces
sudo tcpdump -i ens33 -nn port 53 # DNS
sudo tcpdump -i any -nn host 10.20.0.11
sudo tcpdump -i any -nn 'tcp and port 22 and not src host 10.20.0.1'
sudo tcpdump -i any -nn -A 'port 80' # ASCII output (HTTP headers)
sudo tcpdump -i any -nn -w /tmp/cap.pcap port 443 # save for Wireshark
sudo tcpdump -r /tmp/cap.pcap -nn | head -20 # read a capture
sudo tcpdump -i any -nn 'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0' # SYN only

```

Real scenario: "the web server is slow" → `tcpdump` on the server reveals heavy retransmissions (RST/duplicate ACK) or that MTU/firewall rules are getting in the way. (Week 3 link: the same skill becomes attack-traffic analysis.)

2.3 Bandwidth and link quality

```

iftop -i ens33 # live traffic per connection
nload ens33 ; bmon # bandwidth graphs
dstat -tnf 2 # network + CPU + disk
speedtest-cli # internet speed test
iperf3 -s (server) | iperf3 -c 10.20.0.10 -t 10 -P 4 (client) # real throughput measurement
nmap -sn 10.20.0.0/24 # host discovery (deeper in Week 3)
ss -s ; cat /proc/net/dev # socket and interface counters

```

3) SSH and SCP (12:30-13:00 and into the evening)

3.1 Connection and keys — the professional workflow

```

# on the client
ssh-keygen -t ed25519 -a 100 -C "ali@srv1-2026" -f ~/.ssh/id_ed25519_srv1
ssh-copy-id -i ~/.ssh/id_ed25519_srv1.pub ali@10.20.0.11
ssh -i ~/.ssh/id_ed25519_srv1 ali@10.20.0.11
# ~/.ssh/config — shortcuts and defaults

```

```
# ~/.ssh/config
Host srv1
  HostName 10.20.0.11
  User ali
  Port 2222
  IdentityFile ~/.ssh/id_ed25519_srv1
  ServerAliveInterval 30
  ForwardAgent no
```

```
ssh srv1 # with the config, just this
ssh -J bastion srv1 # jump through a bastion host (ProxyJump)
ssh -L 8080:127.0.0.1:80 srv1 # local tunnel: server port 80 on my 8080
ssh -R 9000:127.0.0.1:3000 srv1 # reverse tunnel (server reaches my local service)
ssh -D 1080 srv1 # SOCKS5 proxy
ssh -N -f -L 5432:db.internal:5432 srv1 # persistent tunnel with no shell (-f backgrounds it)
```

3.2 Copying and transferring files

```
scp file.txt ali@10.20.0.11:/tmp/
scp -r ./site ali@10.20.0.11:/var/www/
scp -P 2222 ali@10.20.0.11:/var/log/syslog ./
rsync -avz --progress ./site/ ali@10.20.0.11:/var/www/site/
rsync -avz --delete --exclude='*.log' ./data/ ali@10.20.0.11:/data/
rsync -e "ssh -p 2222" -avz ./f/ ali@10.20.0.11:/f/ # with a non-standard port
sftp ali@10.20.0.11 # interactive transfer
```

scp or **rsync**? **rsync** transfers only differences, supports `--delete`, `--exclude`, `--link-dest`, and resuming interrupted transfers — better for backups/synchronization; **scp** is simpler for a single small file.

3.3 SSH hardening — the security checklist (very important in interviews)

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%F)
sudo sshd -t # test the config before restarting (skip this and you lock yourself out!)
sudo systemctl reload sshd
```

```
# /etc/ssh/sshd_config – key settings
Port 2222
AddressFamily inet
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers ali deploy@10.20.0.0/24
DenyUsers test
X11Forwarding no
AllowTcpForwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
Protocol 2
```

```
# brute-force mitigation
sudo apt install fail2ban # or dnf install fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo grep -i "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort
-nr | head
```

Standard answer to "how do you secure SSH?" key-only auth with passwords disabled, no root login, restricted users and source addresses (`AllowUsers` , `Match`), a non-default port (reduces noise, not real security), `fail2ban` , continuous patching, logging and monitoring, a bastion with `ProxyJump` for management access, disabled forwarding, and 2FA (TOTP) where required.

4) Lab 1 — Network config, `sshd` and firewall (17:00-18:30)

4.1 Lab topology

Two Linux VMs on the same host-only/NAT network:

Host	Address	Role
<code>srv1</code>	10.20.0.10/24	server (web/SSH), firewall enabled
<code>srv2</code>	10.20.0.11/24	test client
NAT gateway	10.20.0.1	internet access (VirtualBox NAT/Host-only adapter)

4.2 Step 1 — addressing and persistence (three methods)

```
# a) Debian/Ubuntu with netplan (/etc/netplan/01-lab.yaml)
sudo tee /etc/netplan/01-lab.yaml >/dev/null <<'YAML'
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [10.20.0.10/24]
      routes:
        - to: default
          via: 10.20.0.1
      nameservers:
        addresses: [1.1.1.1, 9.9.9.9]
        search: [lab.local]
YAML
sudo netplan try # auto-rollback after 120 seconds if you do not confirm
sudo netplan apply ; ip -br a ; ip route

# b) Rocky/RHEL with nmcli
sudo nmcli con mod "System ens33" ipv4.method manual ipv4.addresses 10.20.0.10/24 \
  ipv4.gateway 10.20.0.1 ipv4.dns "1.1.1.1 9.9.9.9" connection.autoconnect yes
sudo nmcli con up "System ens33" ; nmcli dev show ens33 | head -20

# c) classic Debian: /etc/network/interfaces
# auto ens33
# iface ens33 inet static
# address 10.20.0.10/24
# gateway 10.20.0.1
# dns-nameservers 1.1.1.1
sudo systemctl restart networking # or ifup ens33
```

4.3 Step 2 — test connectivity between the two machines

```
# on srv2
ping -c2 10.20.0.10 ; ip neigh | grep 10.20.0.10
ss -tn state established
nc -zv 10.20.0.10 22
sudo tcpdump -i any -nn host 10.20.0.10 -c 10 # run simultaneously with ping on srv1
tracert 10.20.0.10 ; mtr -rwzc3 10.20.0.10
dig @10.20.0.10 lab.local
```

4.4 Step 3 — SSH hardening and key-based login

```
# on srv2 (client)
ssh-keygen -t ed25519 -C "srv2-to-srv1" -f ~/.ssh/id_srv1
ssh-copy-id -i ~/.ssh/id_srv1.pub ali@10.20.0.10

# on srv1 (server)
sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sshd -t # always test before reloading
sudo systemctl reload ssh # sshd on RHEL
# test from a *different* session (not the one you are working in!)
ssh -i ~/.ssh/id_srv1 ali@10.20.0.10 'hostname; id'
```

The sysadmin golden rule: keep a second SSH session open until you are sure the new configuration works. A bad config plus `systemctl reload` can lock you out of the server.

4.5 Step 4 — host firewall (three implementations, one concept)

a) `ufw` (Debian/Ubuntu — simple and readable):

```
sudo ufw default deny incoming ; sudo ufw default allow outgoing
sudo ufw allow from 10.20.0.0/24 to any port 22 proto tcp
sudo ufw allow 80/tcp ; sudo ufw allow 443/tcp
sudo ufw limit 22/tcp # rate limiting (anti brute force)
sudo ufw enable ; sudo ufw status verbose ; sudo ufw status numbered
sudo ufw delete 3 # delete rule number 3
```

b) `firewalld` (RHEL/Rocky — zone-based):

```
sudo systemctl enable --now firewalld
sudo firewall-cmd --get-active-zones ; sudo firewall-cmd --list-all
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="10.20.0.0/24"
service name="ssh" accept'
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload ; sudo firewall-cmd --list-all
```

c) `nftables` (modern; also backs `iptables-nft`):

```
sudo nft list ruleset
sudo nft add table inet filter
sudo nft add chain inet filter input '{ type filter hook input priority 0; policy drop; }'
sudo nft add rule inet filter input ct state established,related accept
sudo nft add rule inet filter input iif lo accept
sudo nft add rule inet filter input ip saddr 10.20.0.0/24 tcp dport 22 accept
sudo nft add rule inet filter input tcp dport { 80, 443 } accept
sudo nft add rule inet filter input icmp type echo-request accept
sudo nft add rule inet filter input counter log prefix "DROP-IN: " drop
sudo nft list ruleset | head -30
```

Week 1 → host firewall mapping (an interview gold table):

IOS	nftables
<code>access-list 10 permit 10.20.0.0 0.0.0.255</code>	<code>ip saddr 10.20.0.0/24 accept</code>
<code>access-list 101 permit tcp any host X eq 443</code>	<code>tcp dport 443 accept</code>
<code>ip access-group 101 in</code>	<code>hook input priority 0</code>
<code>implicit deny</code>	<code>policy drop</code>
<code>ip nat inside source ... overload</code>	<code>table ip nat { chain postrouting { oifname "eth0" masquerade } }</code>
<code>show access-lists</code>	<code>nft list ruleset</code>

NAT/masquerade on Linux (a direct bridge to Week 1, Day 6):

```
sudo sysctl -w net.ipv4.ip_forward=1
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-ipforward.conf
sudo nft add table ip nat
sudo nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; }'
sudo nft add rule ip nat postrouting oifname "ens33" masquerade
```

This is exactly `ip nat inside source list 1 interface g0/0 overload`, but on Linux.

4.6 Step 5 — DHCP on Linux (optional but valuable)

```
sudo apt install dnsmasq
sudo tee /etc/dnsmasq.d/lab.conf >/dev/null <<'CONF'
interface=ens34
dhcp-range=10.30.0.100,10.30.0.200,12h
dhcp-option=3,10.30.0.1
dhcp-option=6,1.1.1.1
dhcp-authoritative
CONF
sudo systemctl restart dnsmasq
sudo journalctl -u dnsmasq -n 20
# on the client: sudo dhclient -v ens33 or sudo nmcli con up ...
cat /var/lib/misc/dnsmasq.leases
```

Interview line: "I ran DHCP on an IOS router and on Linux with dnsmasq; DORA and lease semantics are identical."

4.7 Break & Fix drills

#	Break	Symptom	Diagnose/fix
1	wrong subnet mask	ping: connect: Network is unreachable	ip -br a , fix address/mask
2	no default gateway	LAN works, internet does not	ip route get 8.8.8.8 → default via
3	wrong DNS	ping 8.8.8.8 fine, ping google.com fails	resolvectl query , /etc/resolv.conf
4	firewall blocks port 80	nc -zv times out, curl hangs	nft list ruleset , ufw status , add the rule
5	broken sshd_config + reload	you cannot log in	from the other open session: sshd -t and fix; console as a fallback
6	PermitRootLogin no + user without sudo	cannot administer	repair via console/recovery
7	changed SSH port without opening the firewall	access lost after reload	console: add the new port rule
8	wrong MTU in a tunnel	connection works, big transfers hang	ping -M do -s 1472 , ip link set mtu 1400

5) Wrapping Up Day 6 (18:45-20:15 and 20:30-21:00)

5.1 Persistent network configuration per distribution

Distribution	Tool/path
Debian 12+/Ubuntu 18+	netplan (/etc/netplan/*.yaml) → networkd/NetworkManager
Older Debian	/etc/network/interfaces + ifupdown
RHEL/Rocky/Fedora	NetworkManager + nmcli / nmtui , files in /etc/NetworkManager/system-connections/
systemd-networkd	/etc/systemd/network/*.network
Modern DNS	systemd-resolved + resolvectl , or local dnsmasq / unbound

5.2 Extending `health.sh` with a network section (bridge to Days 3 and 5)

```
# additions to the Day 3 script
echo "--- network: interfaces and addresses ---" ; ip -br a
echo "--- default route ---" ; ip route show default
echo "--- DNS ---" ; resolvectl status | grep -A2 "DNS Servers" | head -6
echo "--- listening sockets (non-local) ---"
ss -tulnp | awk 'NR==1 || $5 !~ /^(127\.|\\[::1\\])/ '
echo "--- recent NetworkManager errors ---"
journalctl -u NetworkManager -b -p warning --no-pager | tail -5
```

5.3 `sysctl` — network-related kernel tuning (admin level)

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl net.ipv4.tcp_syncookies=1 # SYN flood defence
sudo sysctl net.ipv4.conf.all.rp_filter=1 # anti-spoofing
sudo sysctl -w net.ipv4.tcp_max_syn_backlog=4096
sysctl -a | grep -E "ipv4.tcp_(fin_timeout|keepalive_time)"
sudo tee /etc/sysctl.d/99-network-hardening.conf >/dev/null <<'SYS'
net.ipv4.ip_forward = 1
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv6.conf.all.accept_ra = 0
SYS
sudo sysctl --system
```

Setting	Purpose
<code>ip_forward</code>	route between interfaces (Linux router/NAT)
<code>tcp_syncookies</code>	resilience against SYN floods
<code>rp_filter</code>	reverse path filtering — reduces spoofing
<code>accept_redirects=0</code>	ignore ICMP redirects (prevents route manipulation)
<code>accept_ra=0</code>	disable router advertisements on statically addressed servers

6) □ Day 6 Self-Test (15 questions)

1. Temporary vs persistent addressing: what is the difference, and what is the persistent method on Ubuntu and on RHEL?
2. Name five core `ip` commands with their purpose.
3. What is the IOS equivalent of `ip route add 10.99.0.0/24 via 10.20.0.1`?
4. `ping 8.8.8.8` works but `ping google.com` fails — why, and how do you troubleshoot?
5. How do `ss -tlnp` and `netstat -tulnp` differ on modern distributions?
6. Write a `tcpdump` command that shows DNS traffic only.

7. Difference between `scp` and `rsync`, and which one for backups?
 8. Name four essential SSH hardening actions.
 9. If you cannot log in after editing `sshd_config` and reloading, what do you do?
 10. Compare `ufw`, `firewalld`, and `nftables` — which layer do they share?
 11. What is the `nftables` equivalent of `ip access-group 101 in`?
 12. Write the masquerade rule and the kernel setting required for NAT.
 13. What problem does `ProxyJump` solve?
 14. Write the `sysctl` settings that enable forwarding and harden against SYN floods.
 15. What does `mtr` give you that `traceroute` does not?
-

7) Day 6 Interview Bank (14 questions with model answers)

1) How do you configure and persist networking on Linux? With `ip` for temporary/debug work, and for persistence per distribution: `netplan` on Ubuntu, `nmcli` / NetworkManager on RHEL, or `systemd-networkd`. I apply the change through the service, verify with `systemctl status NetworkManager/networkd` and `ip -br a`, and for risky changes I keep `netplan try` or console access as a safety net.

2) `ip` vs `ifconfig` — why `ip`? `ifconfig` comes from the legacy net-tools package and does not cover modern functionality (interface naming, advanced routes, VRF, policy rules). `iproute2` (`ip`, `ss`, `bridge`, `tc`) is the current kernel standard: faster, more complete, and safe from removal. Show that you know both but use `ip`.

3) A server "has no network". Your troubleshooting path? Bottom-up: `ip -br link` (up?), `ethtool` (cable/speed/CRC), `ip -br a` (address), `ip neigh` (gateway ARP), `ip route get 8.8.8.8` (route/source), ping the gateway → internet → DNS (`dig`), `ss -tlnp` and `nc -zv` for the service, and finally `tcpdump` to see whether packets leave and whether a firewall drops them. Each step confirms or eliminates one OSI layer.

4) `ss` or `netstat`? Why? `ss` reads kernel state directly via netlink and is dramatically faster on busy systems, with rich filters (`dport`, `state`, `src`). `netstat` reads `/proc`, is slower, and is no longer installed by default on many distributions (net-tools is deprecated).

5) Why does `-p` in `ss` sometimes show nothing for a normal user? Displaying the owning PID/process requires access to process information; a non-root user sees only their own sockets and PID data is suppressed for the rest. `sudo ss -tlnp` shows everything. The same logic applies to `lsof -i`.

6) What is a SYN flood and how do you defend a Linux host? An attack that fills the half-open backlog with SYNs. Defences: `tcp_syncookies=1`, raise `tcp_max_syn_backlog` and `somaxconn`, reduce `tcp_synack_retries`, enable `SYNPROXY` in `nftables`, rate limiting in the firewall/ `fail2ban` /WAF, and upstream ACL/rate limits on routers and middleboxes (Week 1 link).

7) Difference between `iptables` and `nftables`? `nftables` is the modern replacement: one engine for IPv4/IPv6/ARP/bridge, uniform syntax, sets/maps, and better performance with large rule sets. `iptables` on new kernels is usually emulated by `iptables-nft`. On fresh installs I choose `nft`; on legacy systems I must know both, because tools like Docker still program rules through the `iptables` interface.

8) How do you prove a firewall or the network is at fault rather than the application? From both ends: on the server `ss -tlnp` (is the service listening?), from the client `nc -zv host port` and `curl -v` (which step fails?), and simultaneously `tcpdump` on the server (does the packet arrive? does a reply leave?). If the SYN arrived and no SYN/ACK left, it is policy/firewall; if the SYN never arrived, routing/network; if TCP completes but no application data comes back, it is the application.

9) How do you manage SSH keys? Generate with `ed25519` plus a passphrase, use `ssh-agent` (`ssh-add -t 8h`), keep hosts in `~/.ssh/config`, distribute with `ssh-copy-id`, rotate keys periodically and remove old ones from `authorized_keys`, use `Match / ForceCommand` to restrict, and at scale adopt an SSH CA or a key manager (Vault, Teleport). On the server: `chmod 700 ~/.ssh`, `600 authorized_keys`, correct ownership, and `PasswordAuthentication no`.

10) How is NAT/masquerade done on Linux, and what is required? `net.ipv4.ip_forward=1` (persisted in `/etc/sysctl.d/`), a `nat` table with a `postrouting` chain and `masquerade` on the egress interface (`oifname`). The iptables equivalent is `-t nat -A POSTROUTING -o eth0 -j MASQUERADE`. You must also ensure `FORWARD` permits the traffic, and that a return path exists for routed (non-NAT) designs. Same concept as IOS `ip nat inside source ... overload`.

11) Explain a real `tcpdump` line. For example `IP 10.20.0.11.51234 > 10.20.0.10.443: Flags [S], seq ...` is a SYN from the client to port 443 with its sequence number; if no `[S.]` comes back, the destination never saw the packet or a firewall dropped it; seeing `[R]` means the packet arrived but nothing is listening on that port. That simple reading solves ~80% of network troubleshooting.

12) MTU problems — how do you detect and fix them? Symptom: the connection establishes but large transfers hang (e.g. the TLS handshake or a download). Test with `ping -M do -s 1472 host` ($1472 + 28 = 1500$) and step down until it succeeds. Fixes: correct the interface/tunnel MTU (`ip link set dev tun0 mtu 1400`), clamp TCP MSS in the firewall (`nft ... tcp flags syn tcp option maxseg size set rt mtu`), and check any path with VPN/GRE/PPPoE.

13) Explain `ProxyJump` and bastions. In a secure architecture servers live in a private network and only one public host (the bastion) allows SSH. With `ssh -J user@bastion target` or `ProxyJump` in `~/.ssh/config`, the connection tunnels through the bastion, keys never travel over the open internet, and access logs are centralized; the benefit is a single hardened entry point to monitor. File transfers work too: `scp -o ProxyJump=...`

14) How do you find which process is talking to a given external IP?

```
ss -tnp | head -20 # TCP connections + process
ss -tnp dst 203.0.113.5
lsof -i @203.0.113.5
sudo tcpdump -nn host 203.0.113.5 -c 20 # confirm the traffic
sudo netstat -plan | grep ESTABLISHED | head
```

In a security scenario (Week 3) this is step one of C2 beacon detection — followed by checking `/proc/<pid>/exe`, `lsof -p`, and the process's parent chain for incident response.

8) ☐ Day 6 Checklist and Week 2 Wrap-up

- [] I configured addressing temporarily with `ip` and persistently with `netplan / nmcli`
- [] I can read `ip route get`, manage the default route, and add/remove routes by hand

- [] I practiced DNS with `resolvectl`, `dig`, `getent`, and `/etc/hosts`
- [] I used `ss -tlnp` to find which service holds a port
- [] I captured DNS/HTTP with `tcpdump` and read the flows
- [] I used `mtr` and `traceroute -T` to trace paths
- [] I set up key-based login and disabled password authentication
- [] I built a `~/.ssh/config` with a Host entry and a `ProxyJump`
- [] I closed the host firewall with **all three** approaches (`ufw`, `firewalld`, `nftables`) and inspected the rules
- [] I implemented Linux NAT/masquerade including `ip_forward`
- [] I tested `dnsmasq` as a DHCP server (optional but recommended)
- [] I reviewed network logs with `journalctl -u NetworkManager -b`
- [] I answered all 14 interview questions out loud

Week 2 complete. The next block (Week 3 — CEH) builds directly on this: scanning with `nmap` on the network you built in Week 1, sniffing with the `tcpdump` /Wireshark skills you gained today, and analyzing the Linux server whose administration you practiced all week.

Week 2 Wrap-Up — Review, Final Exams & Interview Bank

Golden Tables • 30 Short + 50 MCQ + 10 Written • 10 Scenarios • 50 Interview Q&A

Theory + Labs

Self-Test + Answer Key

Interview Question Bank

Week 2 • Days 1-6 — Consolidated review, final exams and answer key

Week 2 Wrap-Up — Days 1-6 Review, Final Exams & Consolidated Interview Bank

LPIC-1 (Linux) — from "fundamentals" to "server networking"

This file does three things: 1) It ties all six days together and collects the "golden tables" into one quick pre-interview review. 2) It runs the week's exams: **30 short questions + 50 multiple-choice + 10 short-answer + 10 practical scenarios + 5 terminal tasks**. 3) It provides the **50-question Week 2 interview bank**.

All answers are in the "Complete Answer Key" section at the end of this file. Answer first — do not peek.

0) The Week 2 Chain — six days, one system

Day	Topic	What it gave you	Link to the next day
1	Fundamentals & architecture	FHS tree, kernel/modules, navigation, pipes, links, file operations	"I can find and move files" → but what about content and ownership?
2	Commands, permissions, file systems	Text processing, rwx/umask/SUID/ACL, inodes, partitions and LVM	"Who is allowed to do what, and where data lives" → but repetitive work?
3	Shells & Bash scripting	Variables, conditionals, loops, trap, debugging, rotating backups	"I automate repetitive work" → but parallel execution and process supervision?
4	Processes & job control	<code>ps/top</code> , signals, nice/ionice, cron/at/timers	"What is running and when does it run" → but how do these come up at boot?
5	systemd & boot	Unit authoring, services, GRUB, journald/logrotate	"Services and startup" → but how does the host talk to the world?
6	Linux networking	<code>ip</code> , <code>ss</code> , <code>tcpdump</code> , SSH, firewall, NAT	"Address, observe, harden" → and Week 3: attack and defence on that surface

Summary line (the golden answer to "what did you learn in the Linux week?"):

"I can build a Linux server from scratch, understand what happens inside it, manage its services with systemd, automate its work, and own its networking: addressing, troubleshooting, firewall, and SSH."

1) Golden Tables — pre-exam review

1.1 The file-system tree (Day 1)

Path	Contents	Note
<code>/etc</code>	configuration	text, version-control friendly
<code>/var</code>	logs, spools, caches	first to fill up
<code>/usr</code> / <code>/usr/local</code>	distro software / your installs	package managers leave <code>/usr/</code> <code>local</code> alone
<code>/boot</code>	vmlinuz, initramfs, GRUB	critical for boot
<code>/proc</code> , <code>/sys</code>	virtual: processes / devices	in RAM
<code>/dev</code> , <code>/run</code> , <code>/tmp</code> , <code>/opt</code> , <code>/home</code> , <code>/root</code> , <code>/srv</code> , <code>/mnt</code> , <code>/media</code>	—	<code>/tmp</code> = 1777, <code>/root</code> = 700

1.2 Permissions (Day 2)

Mode	Meaning	Example
644	rw-r--r--	normal file
600	rw-----	private key
755	rw-r-xr-x	binary/directory
700	rw-x-----	private home
640	rw-r-----	sensitive config
1777	rw-rw-rwt	<code>/tmp</code> (sticky)
2775	rw-rwsr-x	team directory (SGID)
4755	rwsr-xr-x	SUID binary

`umask` → file `666-umask`, directory `777-umask`. Deleting a file depends on the **parent directory** permissions.

1.3 Text processing (Day 2)

Task	Command
search lines	<code>grep -RIn "x" /etc</code>
edit lines	<code>sed -i.bak -E 's/a/b/g' f</code>
fields and math	<code>awk -F: '\$3>=1000{print \$1}' /etc/passwd</code>
most frequent	<code>sort</code> , then <code>uniq -c</code> , then <code>sort -nr</code> , then <code>head</code>
cut columns	<code>cut -d: -f1,3</code>

1.4 Shell scripting (Day 3)

Element	Rule
Header	<code>#!/usr/bin/env bash + set -euo pipefail</code>
Quoting	always <code>"\$var"</code>
Parameters	<code>\$1</code> , <code>\$#</code> , <code>\$@</code> , <code>\${1:-default}</code>
Exit codes	<code>0</code> success, <code>2</code> misuse, <code>127</code> not found, <code>130</code> Ctrl+C, <code>137</code> SIGKILL/OOM
Safety	<code>mktemp</code> , <code>trap ... EXIT</code> , <code>flock</code> , never <code>eval</code> user input
Testing	<code>bash -n</code> , <code>shellcheck</code> , <code>bash -x</code> , <code>PS4='+\$LINENO: '</code>

1.5 Processes and signals (Day 4)

Signal	Use
15 TERM	polite termination (default)
9 KILL	forced termination (last resort)
1 HUP	configuration reload
2 INT / 20 TSTP	<code>Ctrl+C</code> / <code>Ctrl+Z</code>
18 CONT	resume
0	existence check only (nothing sent)

`nice` → -20..19 (CPU) | `ionice` → I/O | load above core count = queue.

1.6 cron and timers (Days 4-5)

```
minute hour day-of-month month day-of-week
* * * * *
```

`*/10 * * * *` every 10 minutes | `0 2 * * *` daily at 02:00 | `@reboot` at boot | `Persistent=true` on timers
= catch up missed runs.

1.7 The boot chain (Day 5)

#	Stage	Critical piece
1	Firmware	BIOS + MBR or UEFI + ESP
2	Bootloader	GRUB2 → <code>/boot/grub/grub.cfg</code> (generated!)
3	Kernel	<code>/boot/vmlinuz-*</code> + <code>initramfs</code>
4	Root and PID 1	<code>/etc/fstab</code> , LVM/LUKS → <code>systemd</code>
5	Target	<code>multi-user.target</code> / <code>graphical.target</code>
6	Login	<code>getty</code> / <code>display-manager</code> / <code>sshd</code>

Three rescue commands: `systemctl status` , `journalctl -xb` , `systemd-analyze blame` .

1.8 IOS ↔ Linux equivalents (the Week 1-2 bridge)

IOS	Linux
<code>show ip interface brief</code>	<code>ip -br a</code>
<code>show ip route</code>	<code>ip route</code>
<code>ip route 0.0.0.0 0.0.0.0 GW</code>	<code>ip route add default via GW</code>
<code>show ip arp</code>	<code>ip neigh</code>
<code>access-list 101 permit tcp ... eq 443</code>	<code>nft ... tcp dport 443 accept</code>
<code>ip access-group 101 in</code>	<code>chain input { hook input ... }</code>
<code>ip nat inside source ... overload</code>	<code>nft ... postrouting masquerade + ip_forward=1</code>
<code>show tcp brief</code>	<code>ss -tnp</code>
<code>debug ip packet</code>	<code>tcpdump -nn -i any</code>
<code>ssh admin@router</code>	<code>ssh user@server + sshd_config</code>
<code>copy run start</code>	<code>systemctl enable</code> , unit files, and config in <code>/etc</code>

2) Short Test — 30 questions (five per day)

- A) Day 1 — Fundamentals & architecture** 1. In one sentence: kernel vs distribution? 2. How do `/proc` and `/sys` differ? 3. Difference between a hard link and a symlink? 4. What is wrong with `cmd 2>&1 > out.txt`? 5. Difference between `atime`, `mtime`, `ctime`, and the command that shows all three?
- B) Day 2 — Commands, permissions, file systems** 6. With `umask 027`, what are the modes of a new file and a new directory? 7. What is the security risk of SUID? 8. What does `setfacl -m d:u:mary:rwX /srv/shared` do? 9. `df -h` shows free space but you cannot create a file — three causes? 10. The three LVM layers, and why use LVM at all?
- C) Day 3 — Shell & scripting** 11. What does each part of `set -euo pipefail` do? 12. Difference between `$@` and `$*`, and which one in loops? 13. Why is `trap 'rm -f "$tmp"' EXIT` necessary? 14. What problem does `flock` solve in cron jobs? 15. How do you syntax-check a script without running it?
- D) Day 4 — Processes** 16. What is a zombie and why can you not `kill -9` it? 17. In `top`, what does a high `wa` mean and which tool goes deeper? 18. Difference between `nice` and `ionice`? 19. What do `nohup` and `tmux` each give you? 20. Three reasons a cron job does not run?
- E) Day 5 — systemd & boot** 21. Difference between `is-active` and `is-enabled`? 22. `disable` vs `mask`? 23. For which services is `Type=oneshot` appropriate? 24. When is `daemon-reload` required? 25. If `/etc/fstab` is wrong, what happens at boot and how do you recover?
- F) Day 6 — Networking** 26. `ping 8.8.8.8` works but `ping google.com` fails — cause and troubleshooting? 27. What is the IOS equivalent of the Linux command `ip route add 10.99.0.0/24 via 10.20.0.1`? 28. What does `ss -tlnp` show, and why is it better than `netstat`? 29. Name four essential SSH hardening actions. 30. `ip_forward=1` plus which nftables rule equals NAT/PAT?

3) Multiple Choice — 50 questions

- 1) Which path holds system configuration files? A) `/var` B) `/etc` C) `/usr/lib` D) `/run`
- 2) Why does `/var/log` matter? A) it only holds kernel logs B) it holds variable system data and is the first place to fill up C) it is read-only and lives in RAM D) it holds temporary files
- 3) Which command shows the real path behind a symlink? A) `ls -li` B) `realpath` or `readlink -f` C) `find -l` D) `which -a`
- 4) With `umask 022`, what is the mode of a new directory? A) 644 B) 755 C) 700 D) 750
- 5) Which bit makes new files inherit the directory's group? A) SUID B) SGID C) sticky D) umask
- 6) What does `find / -perm -4000 -type f 2>/dev/null` find? A) 4000-byte files B) SUID files C) world-writable files D) private directories
- 7) Deleting a file inside a directory depends on which permission? A) the file's own mode B) `w` on the parent directory C) the file's owner D) `umask`
- 8) How do `uniq` and `sort -u` differ? A) no difference B) `uniq` collapses only adjacent duplicates C) `sort -u` works only on numbers D) `uniq` modifies the file
- 9) Which command prints the UID (third column) for users with $UID \geq 1000$? A) `cut -d: -f3 /etc/passwd` B) `awk -F: '$3>=1000{print $3}' /etc/passwd` C) `grep 1000 /etc/passwd` D) `sed -n '1,1000p' /etc/passwd`
- 10) Which is correct for printing lines 10–20 with `sed`? A) `sed -n '10,20p'` B) `sed '10-20p'` C) `sed -r 10:20` D) `sed -e 10,20`
- 11) What does `df -i` report? A) disk device paths B) inode usage C) disk I/O D) inactive partitions
- 12) Which statement about XFS is false? A) good performance on large files B) default on RHEL C) easy to shrink D) journaling
- 13) Which `fstab` option prevents a missing device from breaking boot? A) `nofail` B) `defaults` C) `ro` D) `auto`
- 14) What is the correct order to grow an ext4 LV? A) `resize2fs` then `lvextend` B) `lvextend` then `resize2fs` C) `lvextend` alone suffices D) `mkfs` then `lvextend`
- 15) Which shebang is correct for a bash script? A) `#!/bin/bash` B) `#!/usr/bin/env bash` C) both are valid D) `#!/bin/bash`
- 16) What does `set -u` do? A) stops execution B) errors on undefined variables C) sorts output D) ignores errors
- 17) What is `$?` after a command that was not found? A) 0 B) 1 C) 126 D) 127
- 18) What does exit status 137 usually mean for a service? A) normal exit B) SIGTERM C) SIGKILL (often OOM) D) permission error
- 19) Which works in `[[ ]]` but not in `[ ]`? A) `-f` B) `==` (regex) C) `-z` D) `-d`
- 20) What does `while read -r line; do ...; done < file` give you over `cat file | while read ...`? A) it is faster, avoids word splitting, and runs in the current shell B) it colorizes output C) it locks the file D) no difference
- 21) What does `mktemp` do? A) creates a uniquely named, safe temp file B) deletes a file C) clears `/tmp` D) defines an environment variable

- 22)** What does `shellcheck` do? A) runs and times the script B) statically analyzes scripts for bugs and pitfalls C) checks permissions D) compiles the script
- 23)** Which pattern is dangerous in a script? A) `rm -rf "$dir"` B) `rm -rf $dir` when the variable can be empty C) `rm -i file` D) `find -delete` after testing with `-print`
- 24)** Difference between `( cd /tmp; ls )` and `{ cd /tmp; ls; }`? A) one runs in a subshell, the other in the current shell B) no difference C) the second errors out D) the first is faster
- 25)** What do `fork` and `exec` do, in order? A) run and exit B) copy the process, then replace its image with a new program C) suspend and resume D) read and write
- 26)** Which `ps` state indicates a zombie? A) `R` B) `D` C) `T` D) `Z`
- 27)** What does `kill -HUP <pid>` normally do? A) kills the process B) asks it to reload configuration C) suspends it D) changes priority
- 28)** `load average: 12` on a 4-core machine means? A) 12 users B) roughly 3× the machine's capacity C) 12% CPU D) 12 zombies
- 29)** In `top`, high `wa` points to? A) user CPU usage B) I/O wait C) stolen time D) swap usage
- 30)** Which `nice` value is the highest priority? A) `-20` B) `0` C) `19` D) `10`
- 31)** Why is `ionice` needed for heavy backups? A) `nice` only controls CPU, not I/O B) `ionice` speeds up the disk C) `nice` is deprecated D) `ionice` encrypts data
- 32)** Which signal does `Ctrl+Z` send? A) SIGINT B) SIGTSTP C) SIGKILL D) SIGCONT
- 33)** When does the cron entry `15 3 * * 1-5` run? A) every day at 15:03 B) 03:15 Monday–Friday C) 15 days before each month D) Thursdays at 03:15
- 34)** Which timer option provides catch-up execution after downtime? A) `OnCalendar` B) `Persistent=true` C) `RandomizedDelaySec` D) `AccuracySec`
- 35)** In `/etc/crontab`, what is the extra sixth field (compared with a user crontab)? A) log path B) user name C) priority D) schedule
- 36)** What does `systemctl enable --now svc` do? A) only enables it for boot B) enables it and starts it right now C) only starts it D) masks it
- 37)** How does `mask` differ from `disable`? A) nothing B) `mask` blocks every possible start C) `mask` clears runtime state D) `disable` applies to timers only
- 38)** Which `Type` suits a configuration script that finishes but should stay "active"? A) `simple` B) `forking` C) `oneshot` + `RemainAfterExit=yes` D) `idle`
- 39)** What do you run after editing a `.service` file? A) only `restart` B) `systemctl daemon-reload` then `restart` C) `systemctl reexec` D) `systemd-analyze verify`
- 40)** What does `journalctl -b -1 -p err` show? A) only this boot's errors B) the previous boot's errors C) all logs D) kernel logs
- 41)** Which GRUB file must never be edited by hand? A) `/etc/default/grub` B) `/etc/grub.d/40_custom` C) `/boot/grub/grub.cfg` D) `/etc/grub.d/10_linux`
- 42)** When is `update-initramfs -u` required? A) monthly B) after changing anything that affects early boot (`crypttab`/`fstab`/`drivers`) C) only at install time D) after every package install
- 43)** What is the main cause of `Give root password for maintenance`? A) kernel panic B) an `fstab` mount error or file-system problem C) broken GRUB D) no root password

- 44) What does `ss -tlnp` list? A) TCP listening sockets B) active UDP C) all sockets D) closed connections
- 45) Which command inspects DNS in systemd-resolved? A) `resolvectl status` B) `systemctl dns` C) `nmcli dns show` D) `dig -s`
- 46) What does `tcpdump -nn port 53` capture? A) HTTP B) DNS C) SSH D) ICMP
- 47) Which `sshd_config` setting disables password login? A) `PermitRootLogin no` B) `PasswordAuthentication no` C) `PubkeyAuthentication yes` D) `UsePAM yes`
- 48) What do you run before reloading `sshd`? A) `sshd -t` B) `ssh -T` C) `systemctl daemon-reload` D) `ssh-keygen -t`
- 49) In `nftables`, what is the equivalent of `policy drop` on the input chain? A) `permit ip any any` B) implicit deny (everything except accepted rules) C) `log` everything D) masquerade
- 50) Besides a masquerade rule, what is required for NAT on Linux? A) `net.ipv4.ip_forward=1` B) `net.ipv6.conf.all.forwarding=1` C) `vm.swappiness=10` D) `dnsmasq`
-

4) Short-Answer Exam — 10 written questions

1. Write the first window for diagnosing "the server is slow" in six steps (command + what it tells you).
 2. Write a backup script skeleton that uses arguments, sends errors to stderr, creates a safe temp file, and cleans up on exit (skeleton + key points).
 3. Explain the practical differences between SUID, SGID, and sticky with one real example each, and say which one carries the serious security risk and why.
 4. Write the full boot path from UEFI to login, and give one example failure and error message per stage.
 5. Compare `cron`, `at`, and `systemd timers` in a three-row table and say where you would choose each.
 6. A server "cannot reach the internet" — which layers do you eliminate or confirm, in order, with which commands?
 7. Why is `kill -9` a second resort rather than a first? When are you forced to use it, and what do you check afterwards?
 8. Order permissions, ACLs, SELinux/AppArmor, and the firewall as layers of access control, and describe the order in which you investigate `Permission denied`.
 9. How do you turn "runs by hand" into a secure, resilient systemd service? Name at least eight unit keywords and their roles.
 10. Describe a realistic scenario where "I hardened SSH and locked myself out" — what went wrong and what is the prevention procedure?
-

5) Scenario-Based Exam — 10 scenarios

- S1 — The disk is full.** A web server reports "No space left on device". Give an eight-step diagnosis and fix (commands + decisions + prevention).
- S2 — The server is slow.** `uptime` shows 18 on 4 cores, CPU graphs at 30%, `wa` = 65%. Diagnose, name the tools, the likely cause, and the action.

- S3 — New user, team access.** A new `analytics` team needs read/write on `/srv/data`, one read-only user, and new files to inherit the `analytics` group. Write the exact permissions and ACLs.
- S4 — A service does not start at boot.** `hello.service` starts by hand but not after reboot. Give five possible causes and how to check each.
- S5 — The root password is lost.** Write the recovery steps on a server you can reach via console (three methods).
- S6 — Backups are required.** Daily backups of `/etc` and `/var/www` keeping 14 copies, logged, scheduled, without disrupting the service. Design it fully (script + schedule + restore test).
- S7 — SSH logins became slow.** Login takes 20 seconds. What causes this, and how do you debug with `sshd -d` and logs?
- S8 — An SSH brute-force attack.** 5,000 failed attempts in an hour. Detection from logs, immediate action, preventive action, and a management report.
- S9 — Turn a Linux server into a router/NAT between two networks.** Full configuration (interfaces, forwarding, masquerade, DHCP, DNS, firewall) and how it maps to Week 1 concepts.
- S10 — Zombie processes are piling up.** 2,000 processes in state `Z`. Root-cause it, find the parent, apply a temporary fix and a permanent one.

6) Hands-on Terminal Exam — 5 tasks with acceptance criteria

#	Task	Acceptance criteria
1	Build <code>/srv/app/{conf,logs,data}</code> with group <code>app</code> , service user <code>appsvc</code> , SGID, and read ACL for <code>audit</code>	<code>find /srv/app -printf '%M %u %g %p\n'</code> shows the expected modes; an unauthorized user cannot write
2	Write <code>backup.sh</code> with arguments, <code>set -euo pipefail</code> , <code>trap</code> , logging, 7-copy rotation, meaningful exit codes	<code>shellcheck</code> clean; a failing run exits 1; ten runs leave 7 copies
3	Create <code>hello.service</code> with a dedicated user, <code>Restart</code> , <code>ProtectSystem</code> , <code>MemoryMax</code>	<code>is-active / is-enabled</code> true; <code>pkill</code> triggers automatic restart; <code>systemctl show</code> reports the memory cap
4	A daily 02:30 timer with <code>Persistent=true</code> for the backup	<code>systemctl list-timers</code> shows the entry; <code>journalctl -u backup.service</code> shows success
5	Harden SSH + an <code>nftables</code> policy with 22/80/443, ICMP, default drop, and NAT for the internal network	key login works; password login refused; <code>nft list ruleset</code> shows the rules; an internal client reaches the internet

7) Week 2 Consolidated Interview Bank — 50 questions

A) Architecture & file systems

Q1. What is Linux and what is it made of? Q2. What is FHS and why does it matter? Q3. Difference between `/usr/local` and `/opt`? Q4. How do `/proc` and `/sys` differ? Q5. The three file timestamps and the command that shows them? Q6. What is an inode and when do you run out of them? Q7. `find` vs `locate` — how do you choose? Q8. Why is `xargs` needed?

B) Permissions & file security

Q9. For which file is mode `640` appropriate? Q10. SUID vs SGID with examples? Q11. What does the sticky bit solve? Q12. How is `umask` calculated? Q13. Explain ACLs and when they are required. Q14. `sudo` vs `su` — which and why? Q15. How do you manage and monitor the SUID list?

C) Text processing & tools

Q16. Difference between `grep`, `sed`, and `awk`? Q17. How do you extract the top 10 IPs from a web log? Q18. Difference between `>` and `>>`, and the common hazard? Q19. What is `tee` for? Q20. Explain `sort | uniq -c | sort -nr` in one analysis line.

D) Scripting

Q21. Explain `set -euo pipefail`. Q22. `trap` and real-world uses? Q23. Difference between `'...'`, `"..."`, and unquoted? Q24. Difference between `bash script.sh`, `./script.sh`, and `source script.sh`? Q25. What does "idempotent script" mean? Q26. How do you deliver a safe, reliable script (tests + documentation)? Q27. What do exit codes mean, and why do they matter in automation? Q28. How do you validate user input?

E) Processes & scheduling

Q29. How is a process created (`fork/exec/wait`)? Q30. Zombie, orphan, and the fix for each? Q31. How do you interpret load average? Q32. Read `top`: what do `us`, `sy`, `wa`, `st`, `si/so` tell you? Q33. Difference between `SIGTERM` and `SIGKILL`, and the correct order? Q34. Difference between `nice` and `ionice`? Q35. `nohup`, `disown`, `setsid`, `tmux` — when do you use each? Q36. Why does cron sometimes not run your job? (at least four causes) Q37. `systemd timers` vs `cron`?

F) systemd & boot

Q38. What does systemd offer over SysV init? Q39. The three service states and their commands? Q40. Write a service unit with the critical keywords. Q41. Difference between `simple`, `forking`, `oneshot`? Q42. Why are drop-ins and `systemctl edit` best practice? Q43. Explain the boot chain. Q44. Broken `fstab` and

recovery? Q45. Three ways to recover the root password? Q46. How do `journalctl` and `logrotate` keep the disk under control?

G) Linux networking

Q47. How do you apply network settings persistently (three distributions)? Q48. How do you troubleshoot "the server cannot reach the outside" layer by layer? Q49. How do you harden `sshd`? (full list + fail2ban + bastion) Q50. Compare `ufw` / `firewalld` / `nftables` and write the Linux NAT configuration.

8) Final Checklist and the Week 3 Map

8.1 Final checklist

- [] I can review all six days from the golden tables in under 30 minutes
- [] I took the 30-question and 50-question exams **without** the key
- [] My MCQ score is 40+ out of 50 (below 35 = targeted review of weak days)
- [] I designed all 10 scenarios on paper/terminal
- [] I ran all 5 practical tasks in a VM and verified each acceptance criterion
- [] I answered all 50 interview questions **out loud**
- [] My VM lab includes two Linux VMs + SSH keys + an active firewall + a scheduled backup
- [] All my scripts and unit files are in a Git repository (bridge to Week 3 and to your CV)

8.2 Week 3 map — CEH (ethical hacking) on top of Week 2

Day	Topic	Week 2 foundation it uses directly
1	Information security, CIA, policy, risk	service/log management and file ownership
2	Footprinting and reconnaissance	<code>dig</code> , <code>whois</code> , <code>nc</code> , scripting
3	Scanning and enumeration	<code>ss</code> , <code>nmap</code> , systemd services, <code>netstat</code>
4	System hacking and privilege escalation	SUID, <code>/etc/passwd</code> , <code>sudo -l</code> , processes and kernel
5	Malware and threats	<code>/proc</code> , <code>lsof</code> , spotting suspicious processes, persistence via cron/systemd
6	Web attacks and security	nginx/apache, logs, firewall, TLS and SSH

9) Complete Answer Key

9.1 Short test answers (30 questions)

1. **Kernel** = the core OS (memory, CPU, file systems, networking, drivers); **distribution** = kernel + GNU tools + package manager + repositories + services + installer.
2. `/proc` is a window into processes and kernel state (`/proc/<pid>`); `/sys` exposes the device/driver model (sysfs). Both are virtual and live in RAM.
3. A hard link points at an inode (cannot cross file systems, shows the data size); a symlink is a path (can dangle, can cross file systems) — `ln` vs `ln -s`.
4. In `cmd 2>&1 > out.txt`, stderr is first attached to the current terminal and stdout is then moved to the file. Correct form: `cmd > out.txt 2>&1`. Redirections are evaluated left to right.
5. `atime` last access, `mtime` last content change, `ctime` last metadata change — all with `stat`.
6. File `640`, directory `750` (under `umask 027`).
7. A SUID binary runs with the owner's privileges; if it is vulnerable or malicious (or SUID is set on tools like `vim` / `find`), it becomes a direct path to root.
8. It creates a **default ACL** on the directory: new files inside `/srv/shared` inherit `rwX` for user `mary`.
9. (a) inodes exhausted (`df -i`); (b) a deleted file still held open (`lsof +L1`); (c) a user/group quota or a read-only file system.
10. **PV → VG → LV**; benefits: online grow/shrink, snapshots, a shared pool of several disks, meaningful names.
11. `-e` stops on a failing command, `-u` errors on undefined variables, `-o pipefail` fails the pipeline when any member fails.
12. `$@` keeps each argument separate (use `"$@"` in loops); `$*` merges them into one string.
13. It guarantees temp-file cleanup on any exit path (error, signal, normal) — preventing `/tmp` bloat and data leakage.
14. It prevents two instances from running at once (e.g. a cron job that overruns); `flock -n 9 || exit`.
15. `bash -n script.sh` for syntax, plus `shellcheck` for deeper analysis.
16. A finished process whose parent never called `wait`; because it is already dead it cannot receive signals — it only occupies a process-table slot. Fix the parent or restart it (init reaps orphans).
17. `wa` = I/O wait, pointing at a slow disk/heavy backup/NFS; go deeper with `iostat -xz`, `iotop`, `vmstat`.
18. `nice` sets CPU priority; `ionice` sets I/O class/priority (the idle class for backups).
19. `nohup` makes a process immune to SIGHUP on SSH disconnect and detaches I/O; `tmux` provides a persistent interactive session you can reattach to.
20. Cron's limited PATH and environment, a missing `+x` /wrong shebang, no absolute path or output redirection, `%` characters, the cron service being stopped, undefined variables.
21. `is-active` is the in-memory state; `is-enabled` reports the boot-time wiring.
22. `disable` removes the boot symlinks (manual start still possible); `mask` symlinks to `/dev/null` and blocks every start path.
23. One-shot/scripted work such as mounts, configuration, or finishing a backup; with `RemainAfterExit=yes` it stays "active".
24. Whenever a unit file or drop-in changes; without it systemd keeps the old version in memory.

25. The system stops in emergency mode; log in with the root password, `mount -o remount,rw /`, `journalctl -xb`, fix `fstab` (or add `nofail`), `mount -a`, `findmnt --verify`, reboot.
26. It is DNS: check `resolvectl query`, `/etc/resolv.conf`, `dig @1.1.1.1`, `getent hosts`, then `/etc/nsswitch.conf`, `/etc/hosts`, and the firewall for UDP/53.
27. On IOS: `ip route 10.99.0.0 255.255.255.0 10.20.0.1` (or `ip route add ...`).
28. Listening TCP sockets with PID/process; `ss` reads netlink and is far faster than `netstat` (which reads `/proc` and is deprecated).
29. `PasswordAuthentication no` + key-only auth, `PermitRootLogin no`, restricted `AllowUsers / Match`, lower `MaxAuthTries / LoginGraceTime`, a non-default port plus `fail2ban`, disabled forwarding, and log monitoring.
30. `ip_forward=1` plus a `postrouting ... masquerade` rule on the egress interface (with a suitable FORWARD policy).

9.2 MCQ answer key

Q	A	Q	A	Q	A	Q	A	Q	A
1	B	11	B	21	A	31	A	41	C
2	B	12	C	22	B	32	B	42	B
3	B	13	A	23	B	33	B	43	B
4	B	14	B	24	A	34	B	44	A
5	B	15	C	25	B	35	B	45	A
6	B	16	B	26	D	36	B	46	B
7	B	17	D	27	B	37	B	47	B
8	B	18	C	28	B	38	C	48	A
9	B	19	B	29	B	39	B	49	B
10	A	20	A	30	A	40	B	50	A

MCQ scoring: 45+ = interview ready | 35–44 = targeted review of weak days | below 35 = full review with the golden tables.

Tricky ones: Q15 (both shebangs are valid, but `env` is more portable) | Q18 ($137 = 128 + 9 \rightarrow \text{SIGKILL}$) | Q20 (the `< file` form runs in the current shell, so variables survive) | Q30 (`-20` is the highest priority) | Q44 (listening TCP only).

9.3 Short-answer model answers

31) Slow server — six steps:

- 1) `uptime ->` load average relative to `nproc`
- 2) `top -b -n1 ->` culprit user, `us/sy/wa/st`, `swap`
- 3) `vmstat 2 5 ->` `r/b`, `si/so`, memory pressure
- 4) `iostat -xz 2 5 ->` disk `%util` and `await`
- 5) `ps --sort=-%cpu|-%mem | head ->` the actual PID
- 6) `ss -s ; lsof -i ->` network/socket pressure

Then act: contain (limit/renice/polite kill), fix the cause, write a timeline, and prevent (monitoring, unit resource limits, alerting).

32) Backup script skeleton (key points):

```
#!/usr/bin/env bash
set -euo pipefail
tmp="$(mktemp)"; trap 'rm -f "$tmp"' EXIT
log(){ printf '%s %s\n' "$(date +%F %T)" "$*"; }
err(){ log "ERROR: $*" >&2; }
[[ $# -eq 2 ]] || { err "usage: $0 <src> <dst>"; exit 2; }
src="$1"; dst="$2"; [[ -d "$src" ]] || { err "no such dir: $src"; exit 2; }
mkdir -p "$dst"
arc="$dst/${basename "$src"}-$(date +%Y%m%d-%H%M%S).tar.gz"
tar -czf "$arc" -C "$(dirname "$src")" "$(basename "$src")" || { err "tar failed"; exit 1; }
[[ -s "$arc" ]] || { err "empty archive"; exit 1; }
log "OK $arc $(du -h "$arc" | cut -f1)"
```

Key points: validate input, `mkdir -p` (idempotent), full quoting, meaningful exit codes, timestamped logging, archive verification, `flock` for cron, documentation in the header.

33) SUID / SGID / sticky: - **SUID** (4755): runs a binary with the owner's UID — e.g. `/usr/bin/passwd` writing `/etc/shadow`. **The serious risk:** any vulnerable or extra SUID binary is a privilege-escalation path; keep a baseline and monitor changes. - **SGID on a file** (2755): runs with the owning group's GID. **On a directory** (2775): new files inherit the directory's group — e.g. shared team folders; low risk, commonly forgotten. - **Sticky** (1777): in a directory only the owner/root may delete others' files — e.g. `/tmp`; without it any user could delete another's files.

34) Boot path and sample failures: `UEFI/BIOS` (failure: boot order/NVRAM → "no bootable device") → `GRUB` (failure: missing `grub.cfg` → `grub>` prompt → `grub-install / update-grub`) → `kernel + initramfs` (failure: `VFS: Unable to mount root fs` → rebuild `initramfs`/fix `root=`) → `root mount/fstab` (failure: emergency mode → fix `fstab`) → `systemd/target` (failure: a `failed` service → `systemctl --failed`, `journalctl -b`) → `login` (failure: `sshd` not running → console and `systemctl status ssh`).

35) cron / at / systemd timer:

Criterion	cron	at	systemd timer
Type	recurring	one-shot	recurring
Granularity	one minute	seconds (approximate)	seconds + <code>AccuracySec</code>
Logging	syslog/cron log	atd log	unified journal
Catch-up	none (<code>anacron</code> needed)	—	<code>Persistent=true</code>
Resource control	none	none	<code>Nice/IOWeight/MemoryMax</code>
My choice	compatibility, legacy	timed one-shots	modern default

36) "Cannot reach the internet" — layer order: `ip -br link` (up?) → `ethtool` (cable/speed/CRC) → `ip -br a` (address/mask) → `ip neigh` (gateway ARP) → `ip route get 8.8.8.8` (route and source) → `ping GW` then `ping 8.8.8.8` → `dig / getent` (DNS) → `ss -tlnp` and `nc -zv / curl -v` for the service → simultaneously `tcpdump -nn -i any` on both ends to see whether packets arrive and replies leave → inspect `nft list ruleset / ufw status` and `sysctl ip_forward` in a router scenario.

37) Why `kill -9` is a second resort: SIGKILL cannot be caught, so the process gets no chance to close files, release locks, flush data, or clean sockets (risking corruption and stale locks; in containers it also

breaks cgroup handling). You are forced into it for processes stuck in **D** state or unresponsive to SIGTERM. Afterwards: check `dmesg` / `journalctl`, remove stale locks/temp files if needed, verify data integrity, diagnose why it hung (`strace` /stack traces), and prevent recurrence (TimeoutStopSec, monitoring).

38) Access-control layers and the Permission denied order: 1) **POSIX modes** (owner/group/other on the file and on every **parent directory** — `namei -l`) → 2) **ACLs** (`getfacl`) → 3) **file attributes** (`lsattr` : immutable/append-only) → 4) **SELinux/AppArmor** (`auresearch -m avc -ts recent` , `getenforce` , `journalctl -k | grep denied`) → 5) **firewall** (when the failure is network-level, not file-level) → 6) **service sandboxing** (systemd `ProtectSystem` , `PrivateTmp` , `ReadWritePaths`).

39) Turning an app into a secure systemd service:

```
[Unit] Description, After/Wants=network-online.target, Requires=<db>
[Service] Type=simple, User/Group, WorkingDirectory, EnvironmentFile,
ExecStartPre, ExecStart, ExecReload(/bin/kill -HUP $MAINPID),
Restart=on-failure, RestartSec, TimeoutStopSec,
NoNewPrivileges=yes, PrivateTmp=yes, ProtectSystem=strict,
ProtectHome=yes, ReadWritePaths=..., LimitNOFILE, MemoryMax, CPUQuota
[Install] WantedBy=multi-user.target
```

Procedure: a dedicated system user, precise modes on log/data dirs, manual test as that user, `systemd-analyze verify` , `enable --now` , test `Restart` by killing the process, reboot test, and documentation.

40) "I hardened SSH and locked myself out": Mistakes: `PasswordAuthentication no` before a working key existed; restrictive `AllowUsers` ; a port change plus firewall change at once; a syntax error plus reload without testing. Detection: `journalctl -u ssh -b` (`Permission denied (publickey)` , `no matching key exchange`). Prevention: `sshd -t` , test on a **second port** and a parallel session (`Port 2222` + `sshd -T -f /tmp/test_config`), keep an existing session open until verified, console/VM access available, maintenance window and documentation, and validate the key before disabling passwords (`ssh -o PreferredAuthentications=publickey`).

9.4 Scenario keys

S1 — full disk:

```
df -hT ; df -i
du -xh --max-depth=1 / | sort -h | tail -15
du -xh --max-depth=1 /var | sort -h | tail -10
find / -xdev -type f -size +500M -exec ls -lh {} \; 2>/dev/null | sort -k5 -h | tail
lsof +L1 # deleted-but-open files -> restart the owner
journalctl --disk-usage ; journalctl --vacuum-size=300M
find /var/log -name "*.gz" -mtime +30 -delete
logrotate -d /etc/logrotate.conf
ls -la /var/cache/apt/archives ; apt clean
```

Prevention: space and inode monitoring with 80% alerts, log rotation, a journald quota, separate `/var` and `/var/log` partitions, and LVM for fast growth.

S2 — slow server with high wa : likely I/O pressure; `iostat -xz 2 5` for `%util` / `await` , `iotop -oPa` for the culprit (backup/database/failing disk), `dmesg | grep -i -E "I/O error|ata"` for hardware errors. Actions: contain (`ionice -c3` , `nice` , pause the backup), fix (spread I/O, move data, replace the disk), prevent (backup windows, SMART monitoring with `smartctl -a`).

S3 — team access:

```

sudo groupadd analytics
sudo usermod -aG analytics alice ; sudo usermod -aG analytics bob
sudo mkdir -p /srv/data ; sudo chown root:analytics /srv/data
sudo chmod 2775 /srv/data # SGID for group inheritance
setfacl -m g:analytics:rwX /srv/data
setfacl -m d:g:analytics:rwX /srv/data # default ACL for new files
setfacl -m u:report:r-X /srv/data # read-only user
setfacl -m d:u:report:r-X /srv/data
getfacl /srv/data ; sudo -u report touch /srv/data/x # must fail

```

S4 — service does not start at boot: 1) `is-enabled` (disabled?) 2) missing `WantedBy` and never enabled 3) dependency not ready (`After=network-online.target` absent) 4) conditions (`ConditionPathExists/User`) 5) environment differences at boot (paths/PATH). Diagnose with `systemctl is-enabled`, `systemctl cat`, `systemctl show -p WantedBy -p After`, `journalctl -b -u svc`, `systemd-analyze verify`.

S5 — root password recovery: (a) GRUB → `e` → `rw init=/bin/bash` → `passwd root` → `exec /sbin/reboot -f`; (b) `systemd.unit=rescue.target` or `single`; (c) live media + `chroot` + `passwd`. Prevention: escrow the password in a vault/enterprise secret store, log changes, and restrict physical/console access.

S6 — designed backup:

```

# /etc/systemd/system/lpic-backup.service : Type=oneshot, User=root,
# ExecStart=/usr/local/bin/backup.sh /etc /var/backups && ../backup.sh /var/www /var/backups
# /etc/systemd/system/lpic-backup.timer : OnCalendar=*-*-* 02:30, Persistent=true,
RandomizedDelaySec=600

```

Inside the script: `flock`, logging to `/var/log/backup.log`, `KEEP=14` rotation, `ionice -c3 nice -n19` to avoid I/O pressure, **weekly restore tests** (`tar -tzf`, `rsync --dry-run`, sample extraction with hash comparison), and success/failure reporting to monitoring.

S7 — slow SSH: usually `UseDNS yes` with slow DNS, `GSSAPIAuthentication` attempts, reverse-lookup delay, slow `PAM` / `systemd-logind`, or LDAP. Debug with `ssh -vvv`, `sshd -d -p 2223` on a test port, `journalctl -u ssh -f` during a connection, and `getent hosts <client-ip>`. Fix: `UseDNS no`, `GSSAPIAuthentication no`, repair DNS/PTR, and update authentication services.

S8 — SSH brute force: detect with

```
journalctl -u ssh | grep "Failed password" | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head
```

Immediate action: block the IP at the firewall (`nft add rule ... ip saddr X drop`, `ufw deny from X`) or `fail2ban-client set sshd banip X`; check for **successful** suspicious logins (`grep Accepted`) and rotate credentials if compromised. Preventive: disable passwords, keys + 2FA, firewall rate limits, bastion with restricted `AllowUsers`, SIEM alerts on failure rates, patching.

S9 — Linux router/NAT:

```

# interfaces: lan=ens34 10.30.0.1/24 , wan=ens33 (DHCP from ISP)
sudo sysctl -w net.ipv4.ip_forward=1
sudo nft add table ip nat
sudo nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; }'
sudo nft add rule ip nat postrouting oifname "ens33" masquerade
# firewall: default-drop FORWARD, allow lan->wan, allow related, drop the rest
sudo nft add table inet router
sudo nft add chain inet router forward '{ type filter hook forward priority 0; policy drop; }'
sudo nft add rule inet router forward iifname "ens34" oifname "ens33" accept
sudo nft add rule inet router forward ct state established,related accept
# DHCP+DNS with dnsmasq on ens34 (Day 6 lab) plus ICMP for troubleshooting

```

Week 1 mapping: `ip_forward = ip routing ; masquerade = ip nat inside source ... overload ;` the `forward` chain = inter-interface ACLs; and for dynamic routing, run FRR/OSPF on Linux.

S10 — 2,000 zombies:

```
ps -eo stat,pid,ppid,cmd | awk '$1 ~ /^Z/' | head
ps -eo ppid,stat | awk '$2 ~ /^Z/{print $1}' | sort | uniq -c | sort -nr | head # offending
parents
```

The culprit is a parent that never reaps its children (app bug or a stale service). Temporary fix: restart the parent (zombies are re-parented to init and reaped). Permanent fix: patch/upgrade the app, add a `SIGCHLD` / `waitpid` handler, or run it under a supervisor that manages reaping (systemd with `KillMode` / restart policy). Monitor with a zombie-count alert and document it for the developers.

9.5 Interview bank key (50 questions)

The full answers live in the **daily interview banks**; this is where each group is covered:

Question group	Where the detailed answers are
Q1-Q8 (architecture and file systems)	Day 1 — interview bank (1, 5, 9) plus the golden tables
Q9-Q15 (permissions and file security)	Day 2 — interview bank (1 to 7)
Q16-Q20 (text processing)	Day 2 — section 1 and interview answers 8-9
Q21-Q28 (scripting)	Day 3 — interview bank (1 to 14)
Q29-Q37 (processes and scheduling)	Day 4 — interview bank (1 to 14)
Q38-Q46 (systemd and boot)	Day 5 — interview bank (1 to 14)
Q47-Q50 (networking)	Day 6 — interview bank (1 to 14)

Quick key for fast review: Q1 kernel + user space | Q2 a standard set of paths | Q3 automated installs vs manual/third-party | Q4 processes vs devices | Q5 atime/mtime/ctime with `stat` | Q6 file metadata; exhaustion with many small files | Q7 exact/real-time vs fast/database | Q8 building argument lists for commands that do not read pipes | Q9 sensitive group-shared config | Q10 running as the owner vs group inheritance | Q11 deletion only by the owner in shared directories | Q12 `666/777 - umask` | Q13 extra grants for specific users/groups via `setfacl` | Q14 auditable policy vs a full shell | Q15 baseline list + periodic diff + `chmod u-s` | Q16 line selection, line editing, fields and math | Q17 `awk '{print $1}' | sort | uniq -c | sort -nr | head` | Q18 truncate vs append; `sort f > f` destroys the file | Q19 split output and save simultaneously | Q20 frequency counting | Q21 stop on error / undefined vars / pipeline failure | Q22 exit and error handlers for cleanup | Q23 literal / expanded / split+globbed | Q24 new interpreter / shebang / current shell | Q25 repeated runs with no harmful state change | Q26 `shellcheck` + scenario tests + a header documenting intent | Q27 the success language of automation tools | Q28 allow-list + quoting + no `eval` | Q29 fork → exec → wait | Q30 finished without wait / no parent / fix the parent and init | Q31 relative to the core count | Q32 user/kernel/I-O wait/stolen/swap | Q33 polite then forced; 15 before 9 | Q34 CPU priority vs I/O priority/class | Q35 SIGHUP resilience vs a reattachable session | Q36 PATH/environment, mode and shebang, absolute paths and `%`, service stopped | Q37 resource control, logging, catch-up runs | Q38 parallelism, dependencies, cgroups, journal | Q39 active/enabled/failed | Q40 `Type/User/ExecStart/Restart/WantedBy` plus hardening | Q41 main process / forks / one-shot | Q42 overrides without touching the vendor unit | Q43 UEFI → GRUB → kernel+initramfs → root/PID1 → target → login | Q44 emergency mode, fix and `nofail` | Q45 `init=/bin/bash`, rescue, chroot | Q46 journald quotas

and `logrotate` | Q47 netplan / nmcli / systemd-networkd | Q48 layer by layer with `ip/ethtool/ip route/dig/ss/tcpdump` | Q49 key-only + no root + fail2ban + bastion + logging | Q50 ufw/firewalld/nft plus masquerade and `ip_forward` .

10) End of Week 2

Week 2 complete. If you passed the exams at a reasonable score, you now speak three languages: 1.

System administration: users, permissions, file systems, services, boot, logs. 2. **Automation:** reliable scripts, scheduling, reporting and monitoring. 3. **Host networking:** addressing, troubleshooting, secure SSH, and firewalling — the exact foundation that becomes attack and defence tooling in **Week 3 (CEH)**.

Next step: Week 3 — Day 1 (information security fundamentals: CIA triad, policy, risk management), which sits directly on today's skills: every security control you can define in an interview should also be something you can show where and with which command it is implemented on Linux.